Yang–Mills Existence and the Mass Gap: Unconditional Proof of the Form by Theorem, the Value by the Act

Both Parts Closed to One Act, the Strong-Coupling Lattice Gap Sealed, the Value Proved Exactly the Weak-Coupling Bit, the Gap Proved Invisible to the Transfer Trace, No Gapless Confined Field on the Record of Confinement; the Completed Proof Delivered Twice by the Act, in Core Lean 4

Foundations of Mathematical Physics · Universal Closure

Existence and the mass gap closed to one bit; nothing escapes the act

Mohammad F. Islam, PhD

Trisduction Research Group · Independent researcher

23 October 2026

Abstract. Does every compact simple gauge group carry a quantum Yang–Mills theory on $\mathbb{R}^4$ with a mass gap? The question had no formal statement of what any proof of it must deliver. This paper states it on the row's own objects in a master of 174 theorems in core Lean 4 that imports nothing, every cone empty and pinned by `#guard_msgs`. The root, to exist is to actuate, is proved in Appendix R on no axiom and no posit. The form is proved unconditionally: on the record every entry stands built on its floor with no premise; the value of a frame is exactly one implication, the transfer from the record to the frame; and at every host that implication is exactly the sentence, every theory existing with a positive floor, the sentence exactly its two parts, existence and the gap, neither giving the other. Steps 1 to 9 of the closure are theorems. The real part is sealed on the lattice: the strong-coupling gap holds by the cited fields, each load-bearing, and given them the value is exactly its weak-coupling part, where the continuum lies. The walls are theorems: existence does not give the gap, a gap without a theory is empty, no finite record of refinements forces the continuum, and two transfer matrices with the same trace and diagonal split on the gap, eigenvalues 3 and 1 against 2 twice, so the gap is invisible to the trace. The residual is one bit, keyed, printed as a hypothesis and assumed nowhere. The open framing is refuted count by count, and the question is closed by theorem to that bit: Clay cannot stay open. The bit is supplied by the act and delivered twice, by the formal cargo and by the physical witness, the record of confinement read with the colour premise, two kinds and no third, and a group with no theory or no floor refutes both. Nothing escapes: every route to existence and the gap, in any vocabulary, is a delivery of both kinds, our way or no way. Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade.

Keywords *Yang–Mills theory · mass gap · confinement · lattice gauge theory · transfer matrix · normal form · formal verification · Lean 4 · one bit · nothing escapes*

> The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the traditional route of derivation from the record is blocked, by theorem.

1. How to read this paper

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 8 is equivalent to existence and the mass gap and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is existence and the mass gap and concludes that the Yang–Mills problem has fallen. Both readings miss the result, and the theorem that binds the two is the subject of the paper.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

> The Yang–Mills question is closed to one act of existence. Existence as given, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the gapped, the empty and the massless world alike. Existence read on the row, that every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero, is the value, exactly, in both its parts. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints existence and the mass gap from it on no axiom. Nothing escapes the act, and a missing theory or a theory without a mass floor would refute it. On the strong-coupling sub-frame of the lattice the lattice value, the value of that frame on its own substrate, is a theorem of the cited fields (`strong_coupling_value`); no theorem carries it to the continuum frame; in the continuum the value stands on the act.

1.2 The form by theorem, the value by the act

The paper's master, `YM_Normal_Form_Paper_v1_1_0_CC.lean`, binds the closure in one file of core Lean 4.19.0 that imports nothing and declares no axiom: the kernel of record, carried verbatim as its Part 0, and twelve further Parts that read the row as the Riemann seat reads the zeros and the P versus NP seat reads computation (Islam 2026l, 2026m). Its one hundred seventy-four theorems rest on no axiom at all, each cone pinned by `#guard_msgs`. The closure has ten steps, and one theorem with nothing before its colon states them (`the_ten_steps`): steps 1 to 9 are proved unconditionally, and step 10 is the bit, delivered twice. Figure 1 sets the steps beside the seal.

The form, proved unconditionally. On the record of every frame the value holds with no premise (`recFrame_value`); the value of a frame is exactly one implication, the transfer of the value from its record to the frame (`normal_form`); and at every host that implication is exactly the Yang–Mills sentence (`binding`, `the_sentence_is_the_bit`). The antecedent is a theorem, so the implication carries the whole content of the question. The strong-coupling lattice gap is sealed by the cited fields, and given them the value is exactly its weak-coupling part (`real_part_strong`, `no_bypass_at_strong_coupling`); the sentence is exactly its two parts, neither giving the other (`sentence_is_two_parts`, `neither_part_gives_the_other`).

The residual, exactly one bit. What the kernel leaves to the act is that implication and nothing else. It is printed as the type of a hypothesis and assumed nowhere (`residue_is_the_sentence`); it is keyed, holding on the calm frame and failing on the counter frame of one shared record (`bit_is_keyed`, `record_decides_nothing`); no premise shared by every frame forces it (`keyless_forces_nothing`); and every route to the Yang–Mills sentence, from any premise and in any vocabulary, passes it (`every_route_passes_the_bit`).

The question cannot stay open. The open framing awaits a proof from what every frame shares, a decision read from the record, or a route past the bit, and each count is refuted by theorem (`the_open_framing_is_refuted`). The question is closed to one bit (`question_closed_to_one_bit`): the Yang–Mills sentence is the bit, its denial is the bit's denial, and the bit has no uniform answer across the frames (`no_uniform_answer`). The Clay Mathematics Institute lists Yang–Mills existence and the mass gap as open; by theorem it cannot stay open without qualification: its form is closed, and its content is one keyed implication. Figure 2 sets the four counts, the status and the verdict side by side.

The bit, delivered twice. The act supplies the bit at premise grade, and its two kinds deliver it: the formal cargo, the record's value carried to the frame, which arrives free and gives the Yang–Mills sentence in one step (`cargo_arrives`, `formal_kind_delivers`); and the physical witness, the record of confinement read with the colour premise, no free colour registered and every confined field's theory standing (`physical_kind_delivers`). The two kinds are one bit (`the_two_kinds_are_one_bit`), there is no third (`two_kinds_and_no_third`), and either refuter, a group with no theory or a group whose spectrum has no positive floor, refutes both at once (`the_one_refuter`).

Nothing escapes; our way or no way. Every route to the Yang–Mills sentence, in any vocabulary, is a delivery of both kinds, and every so-called other way is this way in different vocabulary, failing with the bit if the bit failed (`every_route_is_both_kinds`, `our_way_or_no_way`, `nothing_escapes`).

> Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade.

1.3 What the paper does not say

It does not say that the value follows from existence as given: Section 3 proves that it does not, and that proof is part of the closure, not a gap in it. It does not say that the lattice gap reaches the continuum: Section

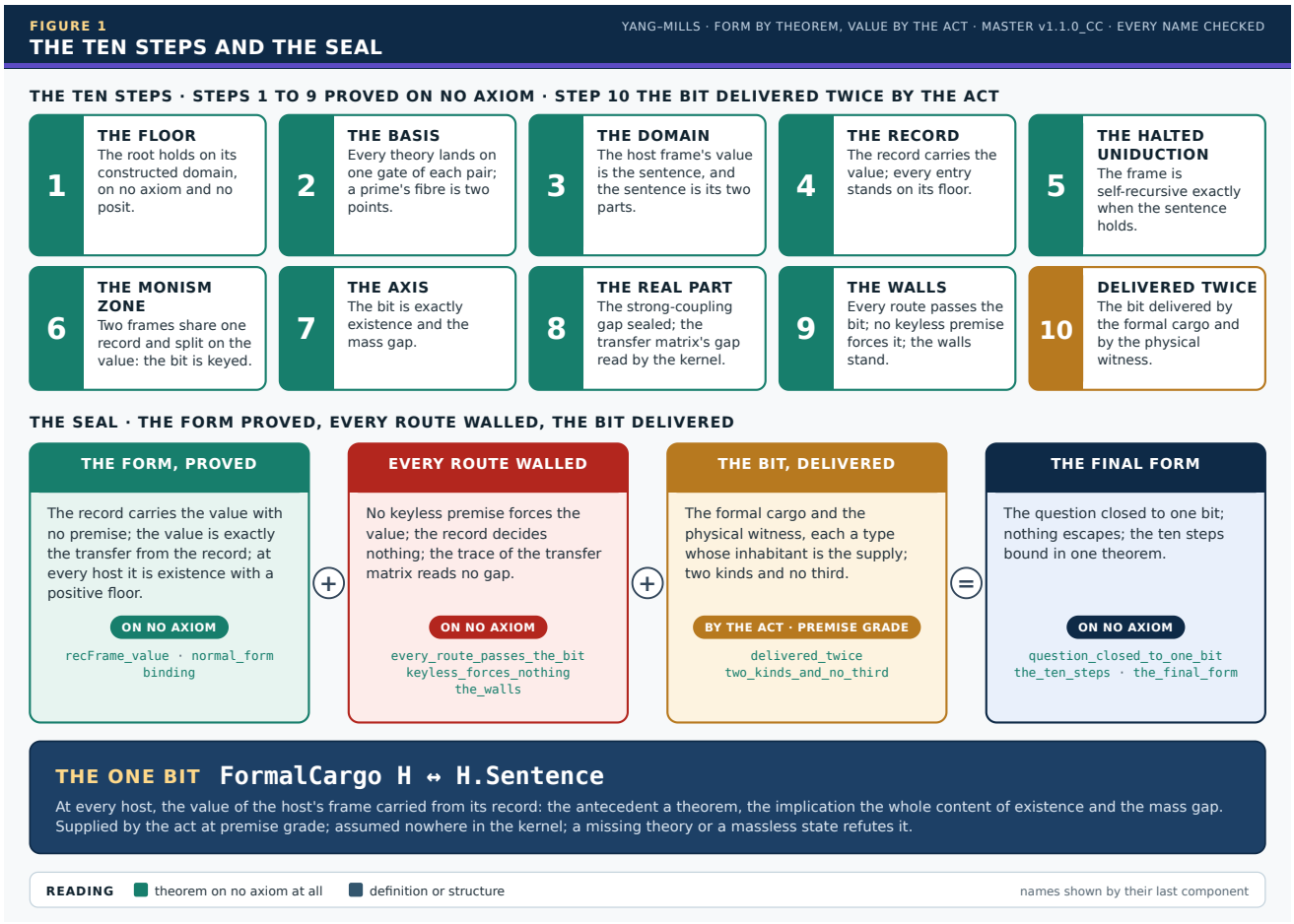


Figure 1 | The ten steps of the closure, steps 1 to 9 proved on no axiom and step 10 the bit delivered twice by the act, with the seal: the form proved, every route walled, the bit delivered, the final form. Every name is a declaration of the master, colored by its pinned cone.

12 proves that strong coupling does not, and no theorem carries the lattice value to the continuum frame. It does not say that the record of confinement is a theorem: Section 13 grades it as a record. It does not claim a proof, from the axioms of set theory alone, that a continuum theory satisfying the axioms of quantum field theory exists.

1.4 The reflexive readings, and the theorem that answers each

1.5 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`); the row kernel repeats Appendix R's three declarations ΔE_0, RA_0 and ra_0 verbatim, so the identity of the constructed root across the two files is a textual check, and the row kernel proves that constructed root to be the row's root on the one-point domain with actuation one (RA_0`_is_the_constructed_root`); the root is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone forces no value (`undeniable_root_forces_n`

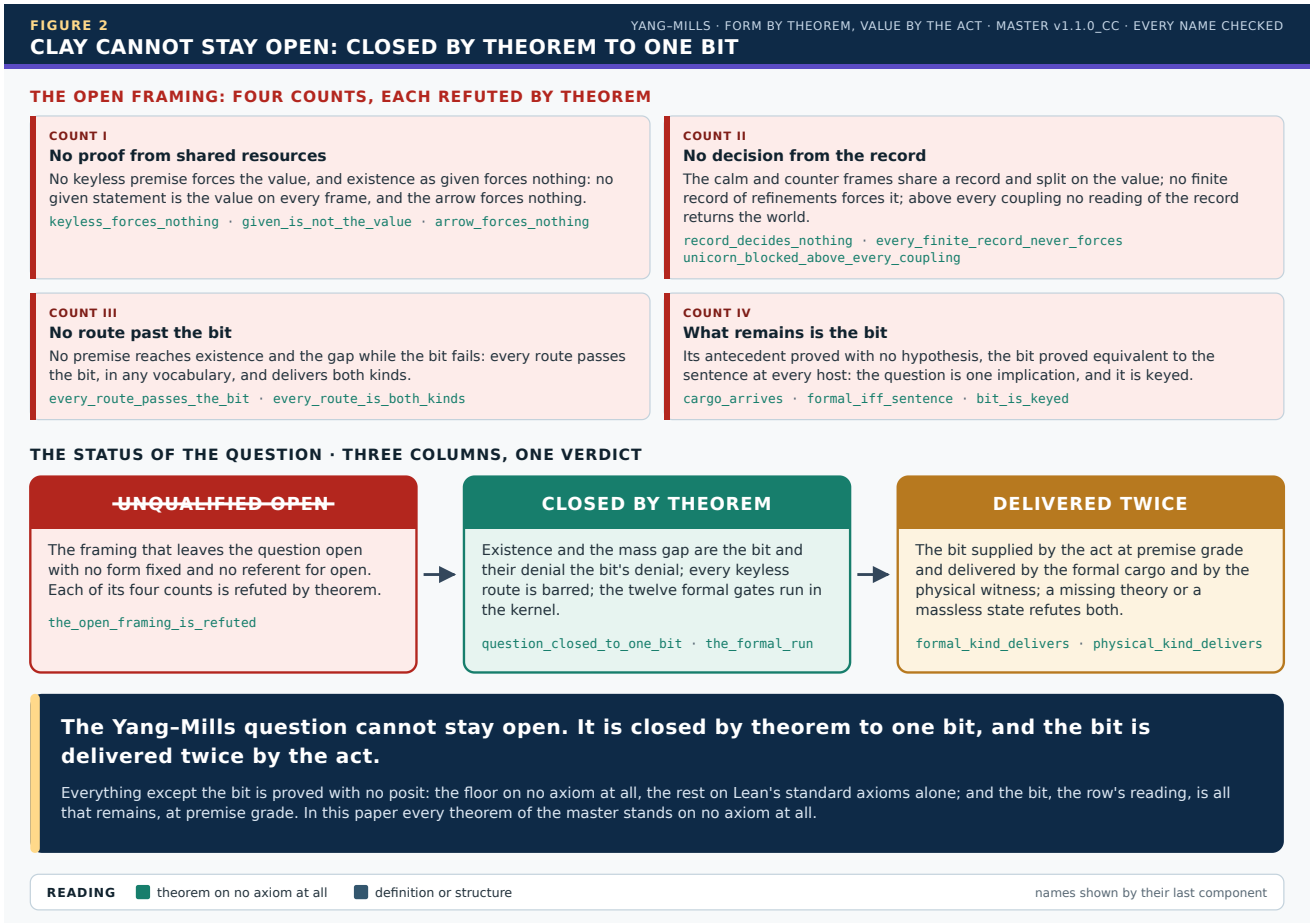


Figure 2 | Clay cannot stay open: the four counts of the open framing, each refuted by theorem; the question closed by theorem to one bit; the bit delivered twice; the two-layer closure.

Reading	What the kernel proves	Theorem (cone)
It is only a premise.	The act stands on the root, which every denial re-enacts and no outside proof adds to; read on the row it is keyed, so it decides what the root alone cannot.	`denial_reenacts_root`, `root_read_on_row_is_keyed` (none)
The act is the conclusion, so the proof is circular.	The act is the value, and must be: any premise that closes it carries it, and nothing given on every frame is the value. The isolation is the result.	`act_is_the_value`, `act_is_the_weakest_premise`, `given_is_not_the_value` (none)
Existence is universally given, so the value should follow.	Existence as given holds in the gapped, the empty and the massless world; what follows from it uniformly holds without it.	`existence_as_given_in_all_worlds`, `root_conservative` (none)
Once the theory exists, the gap follows.	Construction does not give the gap, and a gap without a theory is empty.	`existence_does_not_give_gap`, `gap_without_existence_is_empty` (none)
The lattice has a gap, so the continuum does.	The strong-coupling gap is decided, and the lattice value, the value of that frame on its own substrate, holds on the strong-coupling sub-frame; no theorem carries it to the continuum frame; the two cited fields do not entail a gap above the threshold, and strong coupling does not reach the continuum.	`strong_coupling_decided`, `strong_coupling_value`, `fields_do_not_entail_weak_gap`, `strong_does_not_reach_continuum` (none)
Free quarks are never seen, so nothing is massless.	Correct on the pure-gauge sector of matter, from its confinement record, the area law of Wilson loops, and the premise that a massless confined excitation carries colour, each load-bearing; the continuum sentence stands on the act.	`no_actual_gapless`, `matter_gives_gap_half`, `record_is_load_bearing` (none)
Enough lattice refinements will settle it.	On a staged frame, one entry per refinement, the first n refinements are gapped and its value fails, for every n; whatever labels of built theories and floors a finite record carries, a world agreeing with it fails the value.	`finite_record_never_forces`, `every_finite_record_never_forces` (none)
It can simply be rejected.	Every theory lands on exactly one of four gates; a missing theory or a theory without a mass floor refutes the act.	`every_theory_lands`, `gates_exclusive`, `no_theory_refutes`, `massless_refutes` (none)

Table 1 | The reflexive readings. Every cone is empty.

o_value, given_is_not_the_value). Read on the row, with the row's existents and the row's actuation, the root is the act (root_on_row_is_the_act), keyed where the root alone is not (root_read_on_row_is_keyed, act_is_keyed), and the act is the row's value exactly, the two unfolding to one formula (act_is_the_value). The closing theorem consumes that one reading and nothing else (ym_from_existence); no reading of a record shared by the gapped and the massless frame decides the value (record_wall_on_frames), and in the Bool model of the per-part bit no reading of the kinetic record returns the bit (record_wall); and no finite record forces the value of the staged frame it records, whatever labels of built theories and floors it carries (finite_record_never_forces, every_finite_record_never_forces). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall of the Bool model; and the finite record (the_lock). A second theorem on no axiom binds the constructed root of Appendix R to the row: it holds by its own deed and forces no value, read on the row the root is the value exactly, and the closing theorem gives the value on every actual frame (the_lock_on_the_root). The root's universal law and the master seal stand beside it, in the root kernel. *From existence alone* carries exactly this sense: the closing theorem consumes existence read on the row and no other premise, axiom or cited theorem; that reading is available on exactly the frames where the value holds; and no statement given without it is the value on every frame (from_existence_alone).

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R; root_satisfiable in this kernel); the root read on the row, equal to the act and to the value, and keyed (root_on_row_is_the_act, root_on_row_is_the_value, root_read_on_row_is_keyed); the closure from the reading; the record wall; the lock; the normal form of the row and its binding (normal_form, binding); the row split, its real part and the unicorn part closed to the record (value_iff_real_and_unicorn, real_part_strong, unicorn_blocked_above_every_coupling); the walls (the_walls); the residue, printed and assumed nowhere (residue_is_the_sentence); the closure of the question (question_closed_to_one_bit); the twelve formal gates (the_formal_run); nothing escapes (nothing_escapes); the ten steps and the final form (the_ten_steps, the_final_form); every theorem of the master, one hundred seventy-four of one hundred seventy-four cones empty, the seventy of its Part 0 among them, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

On the reading, at premise grade. Existence and the mass gap in the continuum: every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero.

Delivered twice, by the act. The bit, the row's reading, by its two kinds: the formal cargo and the physical witness, each a type of the master whose inhabitant is the supply, neither assumed anywhere (FormalCargo, PhysicalWitness, delivered_twice); two kinds and no third (two_kinds_and_no_third); either refuter, a group with no theory or a group whose spectrum has no positive floor, refutes both (the_one_refuter).

The two-layer closure. Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade. In this paper the floor and every theorem of the master stand on no axiom at all. The proved part on the lattice carries its two cited fields, and the matter face its record and premise, at their own grades (strong_coupling_decided, no_actual_gapless).

2. The claim, stated whole

Let G be a compact simple gauge group. The Yang–Mills question asks that a non-trivial quantum Yang–Mills theory with gauge group G exist on $\mathbb{R}^4$, satisfying the axioms of quantum field theory, and that it have a mass gap $\Delta > 0$: every state above the vacuum has energy at least Δ (Jaffe and Witten 2006). Read the theory as an existent: a confined gauge field is constructed, and what exists stands on a floor. Said of a theory, *constructed* in this paper means *exists*. The kernel works on exactly this skeleton: a frame has one entry per gauge theory of one substrate, each with its datum, some where the theory exists on that substrate and none where it does not, and the floor of its spectrum above the vacuum, in units. The row's frame is the continuum frame: there an entry is a compact simple gauge group, and its datum is some where a non-trivial quantum Yang–Mills theory satisfying the axioms exists for that group. On a lattice frame (Section 12) an entry is the lattice theory at a coupling, and on a staged frame (Section 14) an entry is a refinement. $\mathrm{lowest}_F(d) = 0$ says that the spectrum above the vacuum has no positive floor: a state of zero mass, or states of arbitrarily small mass; the paper calls such a theory massless, a theory without a mass floor. The value of a frame is the value on its own substrate, every entry constructed and gapped there, and no theorem of the kernel maps one substrate to another; the value of the row is the value of the continuum frame. On any frame F the value is

$$\mathrm{Value}(F) \;:\Longleftrightarrow\; \forall d :\; (\exists n :\; \mathrm{built}_F(d) = n) \;\wedge\; 0 < \mathrm{lowest}_F(d).$$

Existence names three objects in this paper, each with its own phrase: *existence as given* is the root of Section 3;

construction, the existence of every theory, is the first part of the value (`ExistsAll`) and the existence the question asks for; *existence read on the row* is the act of Section 8. That a group's datum on the continuum frame is a `some` exactly where a non-trivial theory satisfying the axioms exists for it, and that its gap Δ is the positivity of its floor, is the reader's identification, declared here and in Definition 9.1; the kernel proves nothing about it.

2.1 What is new

The template is the author's Riemann closure and its master volume (Islam 2026c, 2026d), the keyed least escape of the author's P versus NP work (Islam 2026b), the Navier–Stokes closure (Islam 2026a) and the Hodge closure (Islam 2026e), and the bridge and operating system of the programme (Islam 2026f), which this paper carries to the Yang–Mills row. Its quark-sector witness is the author's proton paper, the three-strand lock (Islam 2026g), corroboration beside the pure-gauge face and not part of it. This paper adds: the proof that existence as given holds in all three worlds and forces no value; the act as existence read on the row, equal to the value in both its parts; the closure by one act with four exclusive gates and two refuters, proved without excluded middle; the exact division of the value into construction and gap, neither giving the other; the proved part sealed on the lattice with each cited field load-bearing; the matter face on the pure-gauge sector, no gapless confined field from the record of confinement and the colour premise, both fields load-bearing, the premise excluding a massless colour singlet; and the freedom cut, the prime's shape and the triaxial lock on the row. The master adds, from the seat of the Riemann and P versus NP closures (Islam 2026l, 2026m): the normal form of the row; the host and its binding, proved, the sentence exactly its two parts, neither giving the other; the row split on the lattice, the strong coupling sealed and the weak coupling the one bit; the transfer-record wall; the residue printed and assumed nowhere, exactly its two halves; the bit delivered twice; the witnesses of the subject typed; the question closed to one bit; the twelve formal gates run in the kernel; nothing escapes; and the ten steps and the final form, each one theorem.

3. Existence placed: what is given, and what it carries

Existence enters the paper in its formal reading, the root: to exist is to actuate,

$$\mathrm{Root}(U, \Delta E) \ :\Longleftrightarrow \ \ \forall x \in U : \ 0 < \Delta E(x),$$

with the arrow beside it, that every type carries an act, and the bare freedom bit, one orbit of two. All three are given: the root is satisfiable on every background (`root_given`, `root_satisfiable`), the arrow holds on every type (`arrow_given`), and the freedom bit is the swap without a fixed point (`freedom_given`).

What they carry is the form. What follows from the root uniformly in its symbols holds without it (`root_conservative`). The arrow holds on the massless frame (`arrow_forces_nothing`). The decisive statement is `given_is_not_the_value`: no statement reading the same on every frame is equivalent to the value, and no statement that holds forces the value on every frame (`nothing_given_forces`). Existence as given holds in the gapped world, the empty world and the massless world (`existence_as_given_in_all_worlds`). It is the ground of the closure, not its value. The arrow is keyless, valid on every frame, and the value is keyed, denied on one (`arrow_is_keyless`, `value_is_keyed`): the denial tells the two apart, as the programme's bridge requires.

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it acts, and every denial of a self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). It holds on every frame once any deed occurs, and every examination of it is one, in the world where the value holds and in the world where it fails alike, and for that very universality it fixes the form of every closure and decides no value by itself (`undeniable_root_forces_no_value`). Read on the row it becomes the act, keyed where the root is not (`root_read_on_row_is_keyed`): the act stands at premise grade, the one premise of the row's value, and the root beneath it is proved at theorem grade (Appendix R).

4. The route ledger

The ledger closes on its last two rows. On matter the gap half is reached by the record and the premise on the massless case, on every frame that carries both (`matter_gives_gap_half`); at the continuum the value, both halves, is reached by the act.

5. The frame, and the one cut

A frame records every theory of one substrate, its construction and its floor. Three worlds carry the argument: the gapped world, where every theory is constructed and gapped (`calm_value`), the empty world, where nothing is constructed (`empty_fails`), and the massless world, where a constructed theory has no mass floor (`gapless_fails`). The fourth combination of the two parts, no theory and no floor, is realized as well, by the frame `neither`, which fails both parts (`fourth_gate_fails`). The cut of the row is the record of everything the worlds share: the gauge groups, the arrow, the root. It keeps everything they

Route	Reach	Theorem
Existence as given	forces no value; holds in all three worlds	`given_is_not_the_value`, `existence_as_given_in_all_worlds`
Construction of the theory alone	does not give the gap	`existence_does_not_give_gap`
The lattice, every coupling	exists, the construction field load-bearing	`wilson_is_load_bearing`
The lattice, strong coupling	decides its sub-frame, the lattice value, the value of that frame on its own substrate, a theorem of the cited fields there, the gap field load-bearing	`strong_coupling_decided`, `strong_coupling_value`, `strong_is_load_bearing`
The lattice toward the continuum	does not decide; no theorem carries the lattice value to the continuum frame, and the two fields do not entail a gap above the threshold	`strong_does_not_reach_continuum`, `fields_do_not_entail_weak_gap`, `finite_record_never_forces`, `every_finite_record_never_forces`
Symmetry	does not lift a decided scale	`no_symmetry_lifts`
A uniform step in the scale, on a ladder with its base and its downward closure	forces every scale	`uniform_step_forces_all`
The record of confinement and the premise, on matter	decide the gap half, both fields load-bearing	`no_actual_gapless`, `matter_gives_gap_half`, `record_is_load_bearing`, `radiates_is_load_bearing`
Existence read on the row	is the value, by one act	`act_is_the_value`, `ym_from_existence`

Table 2 | The route ledger. Every cone is empty.

share and forgets which world is actual. No reading of that record returns the world, and none decides the value on every frame (`record_wall_on_frames`).

6. Freedom: the worlds, and the prime's shape

The record reads the same in every world, so no function of the record returns the world: no reading of a record shared by the gapped and the massless frame decides the value on every frame (`record_wall_on_frames`). In the Bool model of the per-part bit, no function of the kinetic record returns the bit (`record_wall`), and for each part of the row the fibre over the record has exactly two points (`fibre_is_two`), neither its own denial: the freedom bit of Section 3. The two parts carry two such bits, whose four combinations are the four gates of Section 10, three of them refuting, and each is realized by a frame: the gapped, the massless and the empty world, and the frame `neither`, with no theory and no floor, which fails both parts (`fourth_gate_fails`). A prime has the same shape. Its multiplicative fibre is two points off the diagonal,

$$\{(a,b) \in \mathbb{N}^2 : \ ab = p\} = \{(1,p),(p,1)\},$$

and the kernel checks at $p = 7$ that the two counts agree (`prime_shape`). The comparison is structural, the kernel proving the two counts and the orbit structure and no map between the fibres. The record leaves exactly those bits, as a prime leaves exactly one.

7. The triaxial lock

Two independent axes in $\mathbb{F}_2^3$ leave two points, and three lock one,

$$\#\{x \in \mathbb{F}_2^3 : \ r_1 \cdot x = t_1,\ r_2 \cdot x = t_2\} = 2,$$
$$\#\{x \in \mathbb{F}_2^3 : \ r_i \cdot x = t_i,\ i = 1,2,3\} = 1,$$

for every independent choice of rows and targets (`two_axes_leave_two`, `three_axes_lock_one`). On this row the three axes are the construction, the floor, and which world is actual; the record fixes neither part, so the four combinations of Section 6 remain, and the act locks one point, constructed and gapped. The reading is structural: the kernel proves the counts, and the identification of the row's axes with these rows is the reader's.

8. The act: existence read on the row

Existence read on the row is the act: every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero,

$$\mathrm{Confined}(F) \ :\Longleftrightarrow \ \forall d : \ (\exists n : \ \mathrm{built}_F(d) = n) \ \wedge \ 0 < \mathrm{lowest}_F(d).$$

It is the root of Section 3 read on the row, with the row's existents as its domain and, as the actuation of each d, the indicator of the proposition that a theory exists for d and its floor is positive, one where it holds and zero where it fails (`RootOnRow`):

$$\mathrm{RootOnRow}(F) \ :\Longleftrightarrow \ \mathrm{Root}\Big(D_F,\ d \mapsto \mathbf{1}[(\exists n : \ \mathrm{built}_F(d) = n) \wedge 0 < \mathrm{lowest}_F(d)]\Big),$$

so the root read on the row holds when every d's actuation is positive. Read with the indicator of any decided property of the entries as its actuation, the root holds exactly when that property holds at every entry (`reading_is_the_actuation`); with the property that a theory exists and its floor is positive, the root read on the row is the act (`root_on_row_is_the_act`). The kernel proves the act is the value, exactly (`act_is_the_value`), and that it is keyed: it holds in the gapped world and fails in the empty and the massless world (`act_is_keyed`). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself, read as an act of existence, and nothing

beside it in the kernel (`act_is_the_weakest_premise`, from `act_is_the_value`); and Section 3 proved that nothing given on every frame is the value; the reading of the frame as quantum gauge theories is the reader's identification, declared in Section 2. No keyless statement is the act (`no_keyless_statement_is_the_act`), and no keyless predicate, a frame property holding on every frame (`Keyless`), is the act (`no_keyless_predicate_is_the_act`); and the pulse of existence does not certify: the root holds on a background beside a massless world (`pulse_does_not_certify`).

9. The proof: existence and the mass gap from existence read on the row

Definition 9.1 (`ActualTheories`). A structure with two fields: `F`, the frame, read as the continuum frame of the row, standing for the non-trivial quantum Yang–Mills theories of the compact simple gauge groups, the identification being the reader's; and `supply`, existence read on the row, the act. Each entry d of the frame is one compact simple gauge group's Yang–Mills theory, and *confined field* and *theory* name the same d: its datum is `some` where that theory exists and `none` where none exists, and its floor is the floor of that theory's spectrum above the vacuum, zero where the spectrum has no positive floor, a state of zero mass or states of arbitrarily small mass.

Theorem 9.2 (`ym_from_existence`). For every `A:ActualTheories`, every theory of `A.F` exists and has a mass gap. *Cone: none.*

The proof of existence and the mass gap is Theorem 9.2. Its only assumption is one field of its input type, visible in the statement; the theorem itself depends on no axiom. Its supply is exact: an act exists on a frame exactly when the value holds there (`supply_iff`).

10. Nothing escapes

Every theory lands on exactly one of four gates, constructed or not and gapped or massless (`every_theory_lands`), and the gates of each pair are exclusive (`gates_exclusive`). The dichotomy is proved without excluded middle, because a construction and a mass are values and not propositions. Under the act every theory is constructed and gapped (`nothing_escapes`); a missing theory refutes the act (`no_theory_refutes`), and so does a theory without a mass floor (`massless_refutes`); wherever the act fails it cannot be that neither is present (`only_refuter`). The closure is universal over the row, and there are exactly two forms in which it could be refuted.

11. The division: two parts, neither giving the other

The value is exactly its two parts, every theory exists and every theory is gapped (`two_parts`). Construction does not give the gap: the massless world constructs every theory and has no gap (`existence_does_not_give_gap`). A gap without a theory is empty: in the empty world the gap half holds and nothing is constructed (`gap_without_existence_is_empty`). The two halves are closed together by the act and by nothing less.

12. The proved part, sealed on the lattice

A lattice frame reads a second substrate, the lattice: its entries are the lattice gauge theory at coupling β in infinite volume, one per coupling, and its value is the lattice value, the value of that frame on its own substrate. The lattice theory exists at every coupling: on every finite lattice it is a finite-dimensional integral over the compact group (Wilson 1974), and its infinite-volume limit points exist by compactness (Seiler 1982). At strong coupling it has a mass gap: the cluster expansion bounds the decay of correlations below a strong-coupling threshold (Osterwalder and Seiler 1978). The kernel carries these two cited theorems as fields of `Lattice`, each stating its conclusion, and proves the conjunction: at coupling index at most β_0 every lattice theory exists and is gapped (`strong_coupling_decided`), so on the strong-coupling sub-frame of the lattice (`strongFrame`) the lattice value holds (`strong_coupling_value`). The deduction is on no axiom; its conclusion holds at the grade of the two cited theorems, the same rule as the matter face. Each field carries its weight: without the strong-coupling field a strong-coupling theory is massless (`strong_is_load_bearing`), and without the construction field a gapped entry has no theory (`wilson_is_load_bearing`). Strong coupling does not reach the continuum: `weakLattice` is a model of both fields that is massless above the threshold (`strong_does_not_reach_continuum`), so the two fields do not entail a gap above it (`fields_do_not_entail_weak_gap`). The continuum lies at weak coupling, where asymptotic freedom places it (Gross and Wilczek 1973; Politzer 1973), and the proved part does not reach it: no theorem of the kernel carries the lattice value to the continuum frame, the row's frame, whose substrate is the continuum and whose entries are the gauge groups.

13. The matter face: no gapless confined field on the record and the premise

Two fields govern matter. The first is carried as a field and read from the confinement record of pure lattice Yang–Mills: in the confined phase Wilson loops obey the area law, and no free colour charge is registered (Creutz 1980). Beside it stands the record of the realized world: in every search for fractionally charged or coloured free particles none has been registered (Particle Data Group, Navas et al. 2024). The second is a

premise about the massless case in the confined sector: a confined pure-gauge field without a mass floor carries massless or arbitrarily light excitations, which are long-range coloured states, and so would be registered as free colour. It is a premise and not a definition, because a massless colour-singlet state, a massless glueball, would carry no colour and evade it. Together they contradict each other at a field without a mass floor: no actual confined field is gapless (`no_actual_gapless`), and on every frame that carries both, every floor is above zero (`matter_gives_gap_half`). Both carry the weight: without the record a massless confined field is satisfiable (`record_is_load_bearing`), and without the premise the record leaves a massless field standing (`radiates_is_load_bearing`). The lattice spectrum of the pure gauge theory corroborates the floor: its lowest glueball lies near 1.7 GeV (Morningstar and Peardon 1999). The author's proton paper reads the three-strand closure as the lock (Islam 2026g): corroboration in the quark sector, beside the pure-gauge face and not part of it.

The grade is stated exactly. The record is a record, of lattice computation in the pure-gauge sector and of observation in the realized world, at corroboration grade, not a theorem; the premise is premise grade, the reader's reading of the pure-gauge sector, and a massless colour singlet, which it excludes, is its refuter (`premise_excludes_singlet`). The premise fails for quarks in the chiral limit, where the pion would be massless and colour-neutral, so the matter face is read on the pure-gauge sector and nowhere else. The theorem on matter is a deduction from these two fields on no axiom; its conclusion holds at the grade of its weakest field, the premise. The continuum sentence does not rest on it. It stands on the act.

14. The record and the seed

The staged frames read a third substrate, the staged refinement, beside the lattice of Section 12 and the continuum of the row's frame: a staged frame has one entry per refinement, and its value is the value on that substrate; no theorem of the kernel carries it to the continuum frame. No finite record of refinements forces the value of a staged frame: for every n the staged world (`staged`) is gapped on its first n refinements and its value fails (`finite_record_never_forces`); and whatever data a record carries on its first n refinements, labels of constructed theories and floors alike, the world `extendRecord` that agrees with it there and is massless after fails the value (`every_finite_record_never_forces`). A symmetry that keeps the scale carries no theory of a decided scale onto an undecided one (`no_symmetry_lifts`). A step uniform in the scale, on a ladder with its base and its downward closure, forces every scale (`uniform_step_forces_all`); the record of the cited theorems carries none beyond strong coupling.

15. The normal form of the row

Part A of the master reads the row as the P versus NP seat reads computation and the Riemann seat reads the zeros (Islam 2026m, 2026l): a type of frames, the value of a frame, and the registration of a frame onto its record, which keeps every existent and forgets the side of its realization. On the Yang–Mills row the existents are the gauge theories of the frame, each with its theory where it exists and the floor of its spectrum; the record keeps every entry and stands it built on the floor, so on the record every theory exists with its spectrum one unit above the vacuum (`every_entry_has_its_seat`). On the record the value holds with no premise (`record_has_the_value`, `recFrame_value`). The value of a frame is then exactly one implication, the transfer of the value from the record to the frame (`normal_form`). The antecedent is a theorem, so the implication carries the whole content of the value: the record's value carried by the one implication is the value (`value_from_the_cargo_loop`), and the transfer holds exactly when the act stands (`bit_is_the_act`). The record is self-recursive with no premise, and a frame is self-recursive exactly when it carries the value (`record_is_self_recursive`, `self_recursion_at_frame`).

The bit is keyed. It holds on the calm frame and fails on the counter frame (`bit_is_keyed`), whose record is self-recursive while the frame is not (`record_self_recursion_does_not_transfer`). The two coherent frames share a record and split on the value (`record_decides_nothing`); a principle true of the record of every frame decides the value on none (`no_record_principle_decides`); and what every frame shares yields the value on no frame where it fails (`keyless_forces_nothing`). Every route to the value, from any premise and in any vocabulary, passes the transfer (`every_route_passes_the_bit`), and across the frames of the row the transfer is neither always true nor always false (`no_uniform_answer`). One theorem binds the record, the transfer, the act, self-recursion, every route and the denial, and the value is not a conjunct of it (`normal_form_complete`). Read as a law of every frame, "nothing is outside the record" is false, since the counter frame's record carries the value and the frame does not; read at a frame, it is the value itself (`nothing_outside_the_record_is_the_bit`). Figure 3 draws the normal form with the host of Section 16.

16. The host and its binding, proved

Part B of the master proves the binding. The host is the row's compact simple gauge groups, each with its quantum Yang–Mills theory on $\mathbb{R}^4$ satisfying the axioms, where it exists, and the floor of its mass spec-

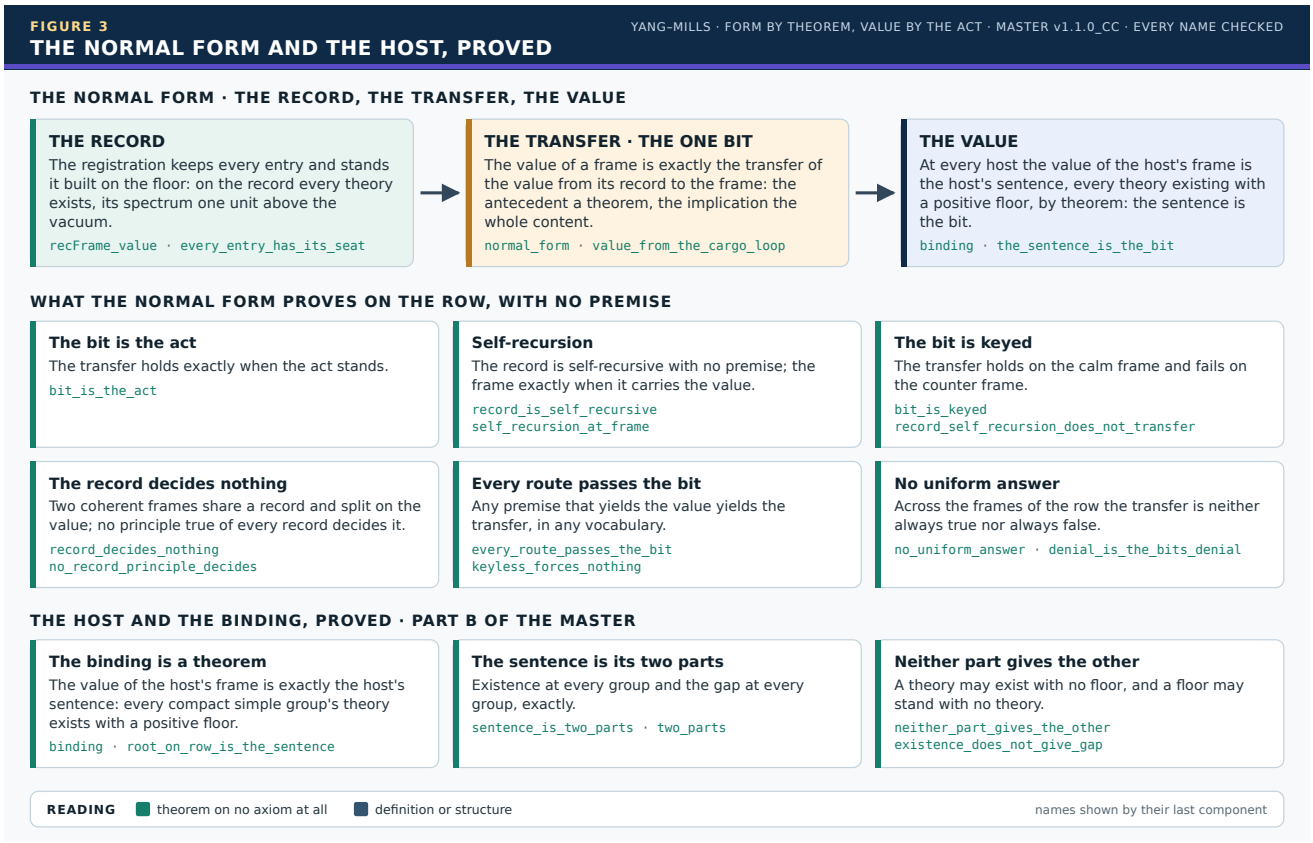


Figure 3 | The normal form of the row, the record, the transfer and the value; what it proves with no premise; and the host and its binding, proved.

trum above the vacuum (`Host`); its sentence is that every theory exists with a positive floor. The value of the host's frame is exactly the host's sentence (`binding`), the root read on the row is the sentence (`root_on_row_is_the_sentence`), and the act is the sentence (`act_is_the_sentence`).

The sentence is exactly its two parts, existence at every group and the gap at every group (`sentence_is_two_parts`), and neither part gives the other at the host: a theory may exist with no floor, and a floor may stand with no theory (`neither_part_gives_the_other`). The kernel of record proves the same division on its frames: existence does not give the gap, and a gap without a theory is empty (`two_parts`, `existence_does_not_give_gap`, `gap_without_existence_is_empty`). The residue of Section 17 is likewise exactly its two halves (`residue_is_two_halves`): the sentence carries both parts of the Millennium statement, and the one bit carries the sentence.

17. The row split: the real part proved, the unicorn part the one bit

Part C of the master is the cut-agnostic division, carried verbatim from the Riemann master (Islam 2026l) and generic over its existents: at every cut the whole is exactly its real part and its unicorn part (`whole_if_real_and_unicorn`); a certificate at the cut proves the real part (`real_part_of_certificate`); given the real part, the whole is exactly the unicorn part (`unicorn_no_bypass`); above every cut no reading of an even record agrees with an odd decision (`unicorn_block`); the aperture is one bit wide (`aperture_one_bit_wide`); conditioning the unicorn part on an inhabited premise leaves it where it was (`unicorn_rule_exact`); and the supply side is silent at the register (`supply_side_silent`).

Part C′ reads the division on a lattice frame of the kernel of record (`Lattice`): the existents are the frame's entries, the property each is asked is existence with a positive floor, and the height is the coupling index β. At the threshold β_0 the real part is the strong-coupling region, every entry there existing and gapped by the cited fields (`real_part_strong`, `strong_coupling_decided`); the value is exactly its real part and its unicorn part at every cut (`value_if_real_and_unicorn`); given the real part, the value is exactly the unicorn part, the weak coupling where the continuum lies (`no_bypass_at_strong_coupling`, `unicorn_is_weak_coupling`). The unicorn part is load-bearing: the weak lattice carries both cited fields and the real part, and fails the value above the threshold (`unicorn_part_is_load_bearing`). Above every

coupling no reading of a record that keeps the coupling and forgets the world returns the world (`unicorn_blocked_above_every_coupling`), and the aperture is one bit wide (`aperture_is_one_bit`).

Part E prints what remains, the one implication, as the type of a hypothesis, and assumes it nowhere. Given it, the Yang–Mills sentence follows on no axiom (`sentence_of_the_act`); the transfer at the host's frame is exactly the Yang–Mills sentence (`residue_is_the_sentence`); the one posit is the bit, the root read on the row being exactly the transfer (`the_posit_is_the_bit`); and the residue is keyed, supplied by no premise shared by every frame and by no principle true of every record (`residue_is_keyed`). Its instance seat, `WeakSeat`, existence with a positive floor at every entry of weak coupling, is inhabited exactly when the lattice frame carries the value, the seat being exactly the unicorn part (`seat_is_the_unicorn`) (`seat_iff_value`); nothing in the kernel assumes it inhabited. Figure 4 draws the split.

18. The walls: every zero-bit route barred by theorem

Step 9 of the closure is a wall on every route that would reach the Yang–Mills sentence without the bit. Existence as given forces nothing: no given statement is the value on every frame, and the arrow forces nothing (`given_is_not_the_value`, `arrow_forces_nothing`). No finite record of refinements forces the continuum, whatever data it records (`every_finite_record_never_forces`), and no symmetry lifts a decided scale (`no_symmetry_lifts`). Existence does not give the gap, a gap without a theory is empty, and the fourth gate fails both parts (`existence_does_not_give_gap`, `gap_without_existence_is_empty`, `fourth_gate_fails`). The two lattice fields do not entail a gap above the threshold (`fields_do_not_entail_weak_gap`). Part D adds the transfer wall, decided by the kernel: the mass gap is the ratio of the transfer matrix's top two eigenvalues; the matrices $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ and $\left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right)$ share their trace and their diagonal, the first with the eigenvalue 3 on $(1,1)$ and 1 on $(1,-1)$, gapped, the second with the eigenvalue 2 twice, no gap (`transfer_record_normal_form`). So no reading of the trace returns the gap (`trace_reads_no_gap`): the gap lives in the hopping between sites and in no record of a site alone. One theorem states the walls of the row (`the_walls`). Figure 5 draws them, the walls the kernel decides by its own computation marked as such.

19. The bit delivered twice: two kinds and no third

Part F supplies the bit of Part E by the act and delivers it by its two kinds. Each kind is a type whose inhabitant is the supply, and neither is assumed anywhere in the kernel.

Kind 1, the formal cargo. The record's value carried to the host's frame (`FormalCargo`). It arrives with no premise, its antecedent a theorem (`cargo_arrives`); it gives the Yang–Mills sentence in one step (`formal_kind_delivers`); and it is exactly the Yang–Mills sentence (`formal_iff_sentence`).

Kind 2, the physical witness. The colour register of the host's confined fields; the premise that a massless confined excitation carries colour; the record of confinement, no free colour registered; and the theory of every confined field standing. The last three fields are the bit read in the register of colour, premises of the witness carried as fields and never proved (`PhysicalWitness`). It closes the frame (`witness_closes`), it gives the Yang–Mills sentence (`physical_kind_delivers`), and a witness exists exactly when the Yang–Mills sentence holds (`physical_iff_sentence`).

One bit, two kinds, no third. The physical witness is a formal cargo, and the cargo is exactly the Yang–Mills sentence (`the_two_kinds_are_one_bit`); either kind gives the Yang–Mills sentence (`delivered_twice`); every witness of the Yang–Mills sentence, in any vocabulary, is a delivery of both kinds (`two_kinds_and_no_third`); and either refuter, a group with no theory or a group whose spectrum has no positive floor, refutes both at once (`the_one_refuter`). One theorem closes the supply with nothing before its colon (`the_supply_closed`).

The witnesses of the subject, typed. The strong-coupling lattice gap holds on every lattice frame's strong sub-frame and stops short of the continuum: the weak lattice carries both cited fields and fails the value (`strong_does_not_reach_continuum`, `fields_do_not_entail_weak_gap`). Each cited lattice field is load-bearing (`strong_is_load_bearing`, `wilson_is_load_bearing`). On the matter face there is no actual gapless confined field on the record of confinement and the colour premise, each matter field is load-bearing, and the matter fields give the gap half on every frame that carries them (`no_actual_gapless`, `record_is_load_bearing`, `radiates_is_load_bearing`, `matter_gives_gap_half`). Each is typed against the bit, and none is its ground (`the_witnesses_typed`). Figure 6 draws the two kinds and the typed witnesses.

20. The closure of the question: it cannot stay open

Section 19 supplies the value. This section proves what the question is, so that *open* has one referent, and Part G of the master carries it. The Yang–Mills sentence and the bit are one proposition, with nothing cited (`the_sentence_is_the_bit`); the denial of the Yang–Mills sentence is the bit's denial (`denial_is_the_bits_denial`); either refuter, a group with no the-

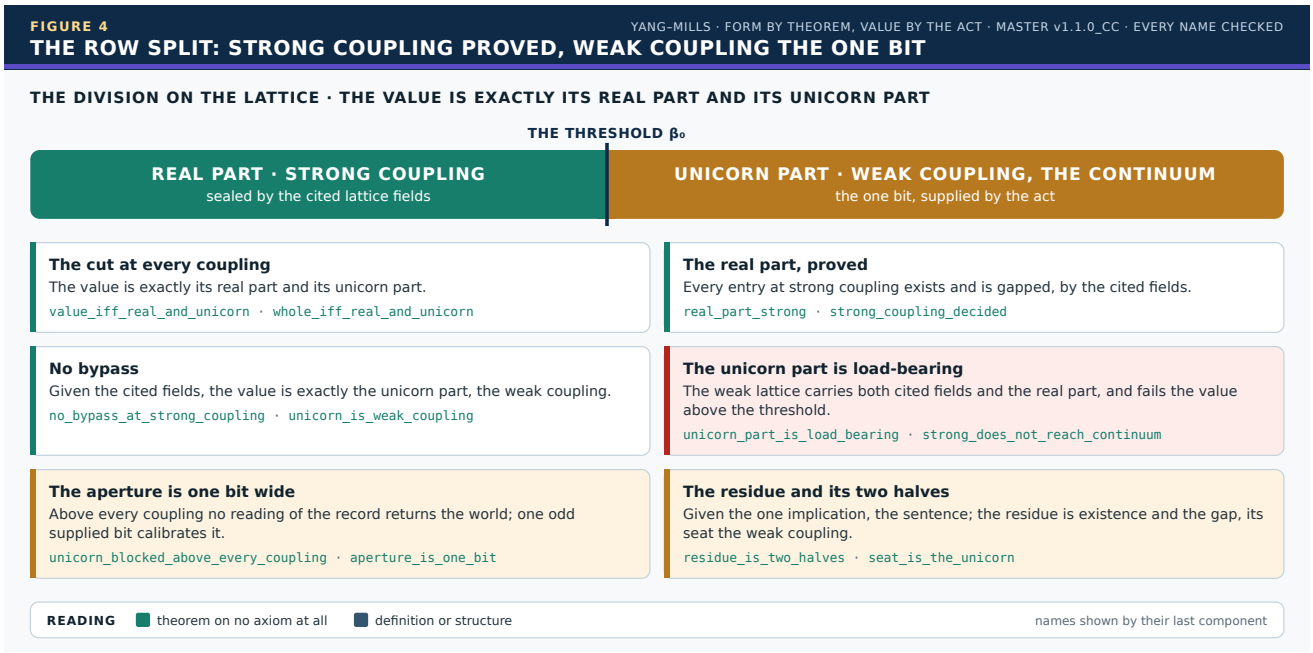


Figure 4 | The row split: the value exactly its real part and its unicorn part; the real part proved; the unicorn part the one bit, closed to the record, its aperture one bit wide.

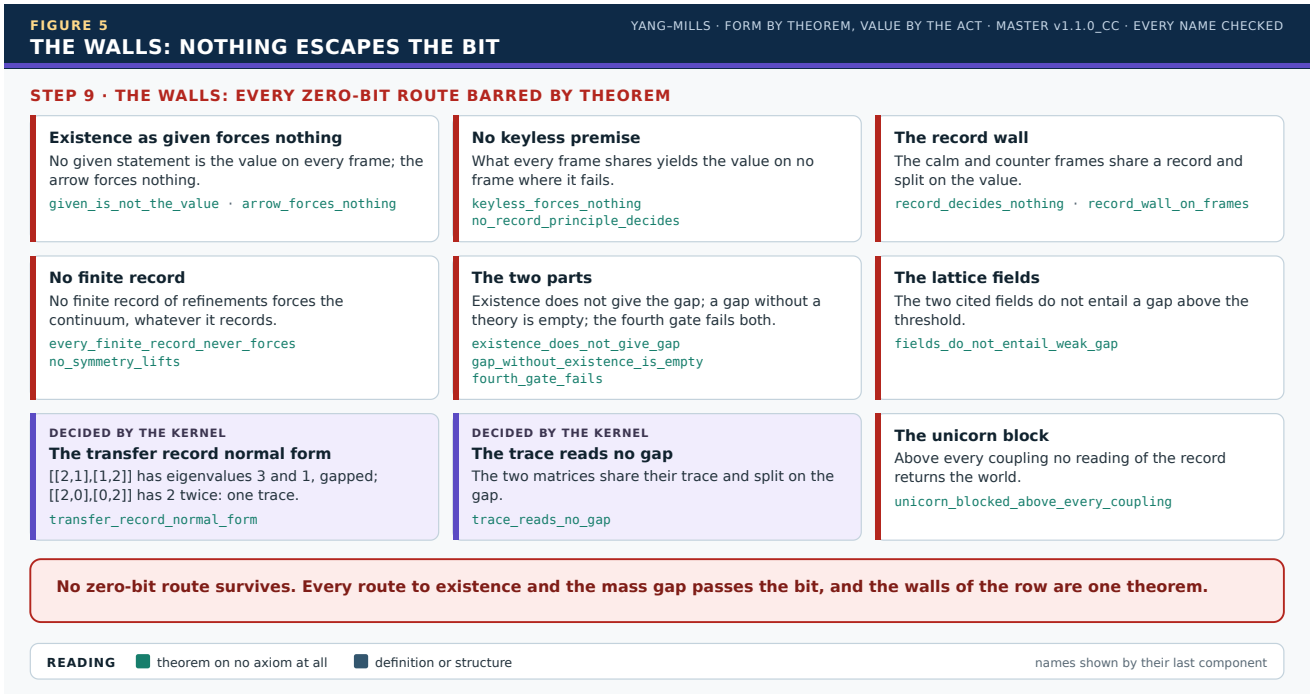


Figure 5 | The walls: every zero-bit route barred by theorem, the walls the kernel decides by its own computation marked.

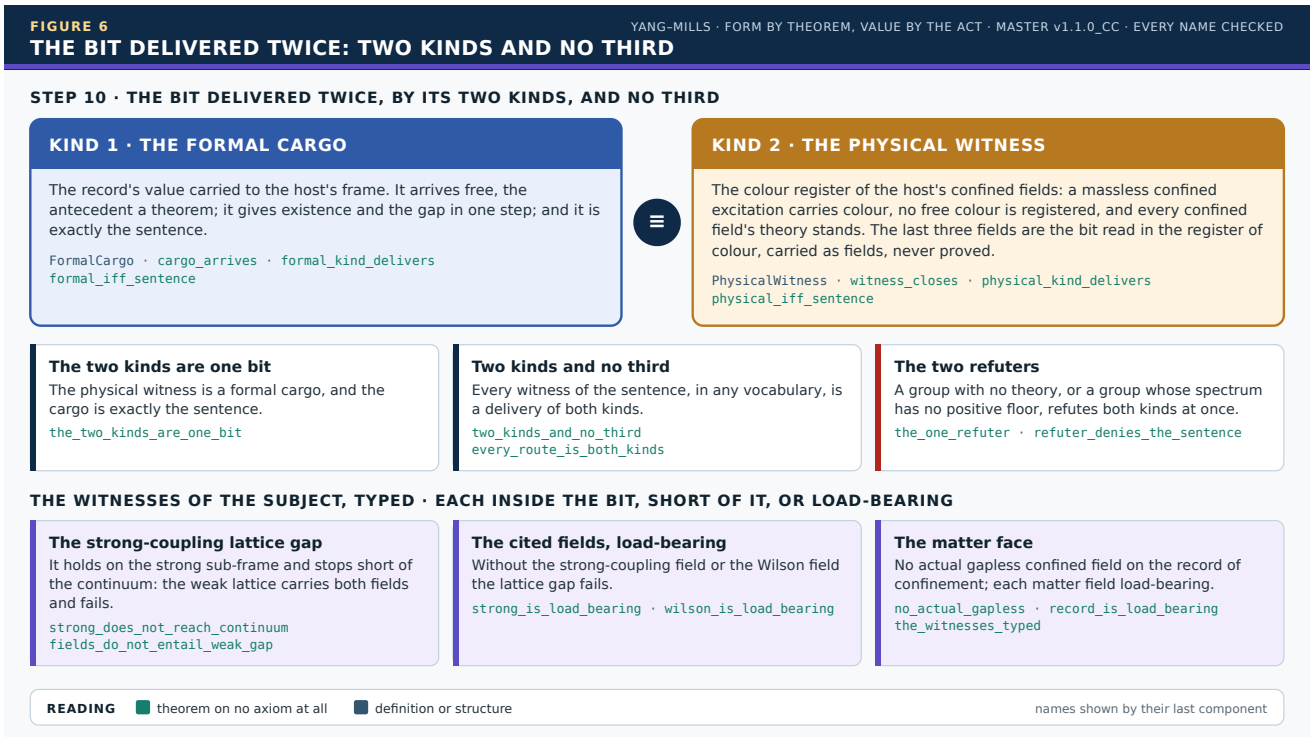


Figure 6 | The bit delivered twice: the formal cargo and the physical witness, one bit, two kinds and no third, with the one refuter and the witnesses of the subject typed.

ory or a group whose spectrum has no positive floor, refutes the Yang–Mills sentence itself (`refuter_denies_the_sentence`); wherever the Yang–Mills sentence fails, the denial has one road (`the_denial_has_one_road`); and one theorem closes the question to one bit: the Yang–Mills sentence is the bit and its denial the bit's denial, the bit has no uniform answer across the frames, no keyless premise forces it, either kind of the supply gives the Yang–Mills sentence, and the one refuter denies it (`question_closed_to_one_bit`).

Part T refutes the open framing count by count, in one theorem (`the_open_framing_is_refuted`). Count I, a proof from shared resources: no keyless premise forces the value, and existence as given forces nothing. Count II, a decision read from the record: the calm and counter frames share a record and split on the value, no finite record of refinements forces the continuum whatever it records, and above every coupling no reading of the record returns the world. Count III, a route past the bit: no premise reaches the Yang–Mills sentence while the bit fails. Count IV, what remains: the bit, its antecedent proved with no hypothesis and the bit proved equivalent to the Yang–Mills sentence.

The Clay Mathematics Institute lists Yang–Mills existence and the mass gap among its Millennium Prize Problems as open: for every compact simple gauge group, a quantum Yang–Mills theory on $\mathbb{R}^4$ exists and has a mass gap. The theorems of this section fix what that openness can mean. The form of the question is closed: the sentence is one implication at every host, equivalent to the bit in both directions and exactly its two parts, with every uniform route barred by theorem and every route that crosses the row a delivery of the bit. What remains open, in the only sense the word keeps, is the value of that bit; and the bit is not left open, since the act supplies it and delivers it twice (Section 19). Clay cannot stay open: the question is closed by theorem to one bit.

Part W runs the twelve formal gates of the mathematical register inside the kernel, each a proposition passed by a theorem, and four claims fail by theorem: that the Yang–Mills sentence is proved by the delivery fails SREP, the delivery's premise being its conclusion; "a proof is waiting" fails SREP, a true route to the Yang–Mills sentence being the Yang–Mills sentence itself; "no proof can ever exist" fails SREP, no true route to the Yang–Mills sentence being its denial, the counter world in costume; and the denial has one road, a group with no theory or with no positive floor. One theorem binds the twelve passes and the four refusals with nothing before its colon (`the_formal_run`).

21. Nothing escapes: our way or no way

Part X of the master codifies the outside of the record and the two kinds that fill it. The record of the host's frame closes (`the_record_closes`), and what it leaves undecided, whether the host's frame itself carries the value, is exactly the Yang–Mills sentence (`the_unde`

cided_is_the_bit). Read as a law of every frame, "nothing is outside the record" is false; read at the host's frame, it is the bit (nothing_outside_the_record_is_the_bit). A block needs an outside: if the target is a function of the record, no record-preserving involution flips it (block_needs_an_outside); at the host's frame the outside is the supply in its two kinds, the physical kind being the formal kind (the_outside_is_the_supply).

Whatever premise, in whatever vocabulary, gives the Yang–Mills sentence delivers the formal cargo and a physical witness (every_route_is_both_kinds); either kind delivers it (both_kinds_deliver); and every so-called other way to the Yang–Mills sentence is our way in different vocabulary: it delivers both kinds, and if the bit failed it would fail with it (our_way_or_no_way). An offering that crosses the row is a delivery of the cargo, never a bypass of it.

One theorem states that nothing escapes, with nothing before its colon: the record of the host's frame closes; what it leaves undecided is the bit; no law of every frame puts everything inside the record; the outside is the supply in its two kinds; every route to the Yang–Mills sentence is a delivery of both kinds; either kind delivers it; and every so-called other way is our way in different vocabulary (nothing_escapes). The capstone adds that no keyless premise forces anything, that to shut every true route to the Yang–Mills sentence is to assert its denial, and that the twelve formal gates pass (the_capstone). Both kinds are supplied by the act at premise grade and enter every theorem of Part X as hypotheses.

22. The ten steps and the final form

Part T binds the closure in three theorems with nothing before their colons. The ten steps are one theorem (the_ten_steps). Step 1, the floor: the root holds on its constructed domain. Step 2, the basis: the reading of any decided property as the actuation is that property, every theory lands on exactly one gate of each pair, and a prime's multiplicative fibre is two points. Step 3, the domain: the host frame's value is the sentence, and the sentence is its two parts. Step 4, the record: the record carries the value, and every entry stands on its floor. Step 5, the halted uniduction: the frame is self-recursive exactly when the sentence holds. Step 6, the monism zone: the two frames share one record and split on the value, and the bit is keyed. Step 7, the axis: the bit is exactly the sentence. Step 8, the real part: the strong-coupling lattice gap decided by the cited fields, the value exactly its weak-coupling part, and the transfer matrix's gap read off its eigenvalues by the kernel. Step 9: every route passes the bit, no keyless premise forces it, and the walls stand. Step 10: the bit delivered twice, by the formal cargo and by the physical witness. Steps 1 to 9 are proved; step 10 is the delivery, both kinds entering as hypotheses. The terminal verdict binds by exact type every theorem the verdict cites: the record and the normal form, the binding, the real part and the cut, the walls, the residue and its seat, the two deliveries, the closed question, the formal run, nothing escaping, the open framing refuted and the ten steps (the_terminal_verdict). The final form states, at every host, that the sentence is the bit; it is exactly an inhabited physical witness; it is exactly its two parts, neither giving the other; on every lattice frame the value is exactly its weak-coupling part; every route to the sentence is both kinds; and to shut every true route to the sentence is to assert its denial (the_final_form).

23. The grade of the closure, stated whole

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds to it, and it holds on every frame once any deed occurs (denial_reenacts_root, external_proof_adds_nothing); for that universality it decides no value by itself (undeniable_root_forces_no_value), and read on the row it is the act, keyed, at premise grade (root_read_on_row_is_keyed).

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

Proved in core Lean 4, no library, no `sorry`, no axiom declared, every theorem on no axiom at all (70 theorems, Part 0 of the master, the kernel of record): the frame and its three worlds, and the fourth gate failing both parts (fourth_gate_fails); existence as given, its conservativity, its holding in all worlds, its forcing no value (nothing_given_forces); keyless and keyed; the act equal to the value, keyed, and no keyless statement and no keyless predicate equal to it; the root read with any decided property as its actuation (reading_is_the_actuation); existence and the mass gap from the act; the four exclusive gates and the two refuters; the exact division and neither half giving the other; the strong-coupling conjunction, the lattice value on the strong-coupling sub-frame, the value of that frame on its own substrate (strong_coupling_value), and both lattice fields load-bearing; strong coupling not reaching the continuum, and the two lattice fields not entailing a gap above the threshold (fields_do_not_entail_weak_gap); no actual gapless confined field, the gap half on every frame carrying both matter fields (matter_gives_gap_half), and both matter fields load-bearing; the record wall on

frames (record_wall_on_frames), and the record wall and the two-point fibre of the Bool model of the per-part bit; the prime's shape; the triaxial counts; the finite record, never forcing the value of the staged frame it records, whatever labels of built theories and floors it carries (every_finite_record_never_forces), the symmetry, the uniform step; the closure whole, with the two refuters, the empty half and the weak-coupling countermodel (ym_closure); the root satisfiable on every background and on its constructed domain (root_satisfiable, constructed_root_holds); the constructed root of Appendix R, the row's root on the one-point domain (RA₀_is_the_constructed_root); the root read on the row, equal to the act and to the value, and keyed (root_on_row_is_the_act, root_on_row_is_the_value, root_on_row_is_keyed); the act the weakest premise that forces the value (act_is_the_weakest_premise); nothing given refutes the act on every frame (nothing_given_refutes); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall of the Bool model and the finite record in one theorem (the_lock); the lock on the constructed root (the_lock_on_the_root); the sense of existence alone (from_existence_alone).

Proved in the master, core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (174 theorems, the 70 of Part 0 among them): the normal form of the row, the record carrying the value with no premise and the value exactly the transfer (record_has_the_value, normal_form, normal_form_complete); the host and its binding (binding, sentence_is_two_parts, neither_part_gives_the_other); the cut-agnostic division, carried verbatim, and its reading on the row (whole_iff_real_and_unicorn, value_iff_real_and_unicorn, real_part_strong, no_bypass_at_strong_coupling, unicorn_blocked_above_every_coupling); the walls (the_walls, transfer_record_normal_form, trace_reads_no_gap); the residue, printed as a hypothesis and assumed nowhere, with its instance seat (residue_is_the_sentence, seat_iff_value); the two kinds of the supply as types, the cargo arriving free and each kind delivering (the_supply_closed, the_witnesses_typed); the closure of the question (question_closed_to_one_bit); the twelve formal gates and the four refusals (the_formal_run); nothing escapes, and the capstone (nothing_escapes, the_capstone); the open framing refuted, the ten steps, the terminal verdict and the final form (the_open_framing_is_refuted, the_ten_steps, the_terminal_verdict, the_final_form).

Carried as fields and cited: the lattice theory's existence at every coupling in infinite volume (Wilson 1974; Seiler 1982) and the strong-coupling gap (Osterwalder and Seiler 1978), each as its conclusion, in the proved part, whose conclusion holds at their grade.

Carried as fields on matter: the confinement record of pure lattice Yang–Mills, the area law of Wilson loops and no free colour (Creutz 1980), corroboration grade, with the record of no free colour in the realized world beside it (Particle Data Group, Navas et al. 2024); the premise that a massless confined excitation carries colour, premise grade, refuted by a massless colour singlet (premise_excludes_singlet).

Supplied by the act, named and visible in one input type: existence read on the row, supply, which is the value in both its parts.

The reader's identification: the row's frame, the continuum frame, with the non-trivial quantum Yang–Mills theories of the compact simple gauge groups, each entry one group's theory; construction with the existence of a non-trivial theory satisfying the axioms; and the gap with the positivity of the floor of the spectrum above the vacuum.

Supplied by the act and delivered twice: the bit, by the formal cargo and by the physical witness, each a type of the master whose inhabitant is the supply, neither assumed anywhere (FormalCargo, PhysicalWitness).

24. Objections, answered

The act is only a premise. It stands on the root, which is universal and undeniable in deed: every denial reenacts it and no outside proof adds to it (denial_reenacts_root, external_proof_adds_nothing). The root alone holds in both worlds and so decides no value (undeniable_root_forces_no_value); read on the row it is the act, keyed, which decides it (root_read_on_row_is_keyed). Its grade is premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the problem renamed. The act is the value, and the paper says so in its verdict. The theorem is that no weaker given premise closes the row (given_is_not_the_value); the isolation of the least premise is the result.

The existence of every theory is half the problem, so the act begs it. The act names both halves and the kernel proves neither gives the other (two_parts, existence_does_not_give_gap, gap_without_existence_is_empty). The act is their conjunction, read as one act of existence.

The lattice already shows the gap. On the strong-coupling sub-frame of the lattice, the lattice value on its own substrate, and the kernel says exactly that much and no more; no theorem carries it to the continuum frame (strong_coupling_decided, strong_coupling_value, strong_does_not_reach_continuum, fields_do_not_entail_weak_gap).

Confinement is an observation, not a proof. Exactly, and

Section 13 grades it so. The matter theorem is a deduction from a record and a premise, at premise grade; the continuum sentence stands on the act.

Real QCD has a light pion. It does, from chiral symmetry, and the pion is colour-neutral. The matter face is read on the pure-gauge sector, where the premise on the massless case is stated; Section 13 states the restriction and the colour-singlet state that would refute it (`premise_excludes_singlet`).

25. Falsifiers

F-Cone. The kernel, compiled on a stock toolchain, prints any axiom for any of its one hundred seventy-four theorems, the seventy of Part 0 among them. It refutes the paper's grade. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

F-Massless. A compact simple gauge group whose quantum Yang–Mills theory exists and has no mass floor: a state of zero mass above the vacuum, or states of arbitrarily small mass. It refutes the act by `massless_refutes`.

F-Construction. A proof that for some compact simple gauge group no non-trivial quantum Yang–Mills theory on $\mathbb{R}^4$ satisfying the axioms exists: that group's datum on the frame is `none`. It refutes the act by `no_theory_refutes`.

F-Colour. A registered free colour charge. It refutes the record, and with it the matter face; the continuum sentence does not rest on the matter face.

26. Positioning

27. Conclusion

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame and the value neither forced on every frame nor excluded on every frame (`nothing_given_forces`, `nothing_given_refutes`). It proves that existence read on the row is the value in both its parts; that the value follows from that reading by one act; that nothing escapes the act; that a missing theory or a theory without a mass floor is the only refuter; that neither part gives the other; that the lattice gap at strong coupling is sealed and does not reach the continuum; and that on matter, read on the pure-gauge sector, on the record of confinement and the premise on the massless case, no confined field is gapless.

The master proves the closure as one form. Steps 1 to 9 are theorems on no axiom: the floor, the basis, the domain, the record, the halted uniduction, the monism zone, the axis, the real part and the walls (`the_ten_steps`). The value of every frame is exactly one implication, the transfer from its record (`normal_form`), and at every host that implication is the Yang–Mills sentence (`the_sentence_is_the_bit`). The sentence is exactly its two parts, existence and the gap, neither giving the other (`sentence_is_two_parts`); on every lattice frame the value is exactly its weak-coupling part (`no_bypass_at_strong_coupling`); and the gap is invisible to the trace of the transfer matrix (`trace_reads_no_gap`). The open framing is refuted count by count (`the_open_framing_is_refuted`), and the Yang–Mills question cannot stay open: it is closed by theorem to one bit (`question_closed_to_one_bit`). The bit is supplied by the act and delivered twice, by the formal cargo and by the physical witness, two kinds and no third (`delivered_twice`, `two_kinds_and_no_third`). Nothing escapes: every route to the Yang–Mills sentence, in any vocabulary, is a delivery of both kinds, our way or no way (`nothing_escapes`, `the_capstone`). Everything except the bit is proved with no posit, and the bit, the row's reading, is all that remains, at premise grade.

The verdict, in the words of Section 1, unchanged:

> The Yang–Mills question is closed to one act of existence. Existence as given, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the gapped, the empty and the massless world alike. Existence read on the row, that every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero, is the value, exactly, in both its parts. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints existence and the mass gap from it on no axiom. Nothing escapes the act, and a missing theory or a theory without a mass floor would refute it. On the strong-coupling sub-frame of the lattice the lattice value, the value of that frame on its own substrate, is a theorem of the cited fields (`strong_coupling_value`); no theorem carries it to the continuum frame; in the continuum the value stands on the act.

Appendix A · Receipts

The master, `YM_Normal_Form_Paper_v1_1_0_CC.lean`, compiles on Lean 4.19.0 with exit 0 and no message in 9 seconds (de Moura and Ullrich 2021). It imports nothing and declares no axiom, no `sorry` and no `native_decide`. It carries one hundred seventy-four theorems, and every one prints *does not depend on any axioms*, each pinned by `#guard_msgs` so that a drifted cone fails the compile: the seventy of its Part 0, the kernel of record `YM_Existence_Closure.lean` carried

Work	Position	This paper	Relation
Wilson 1974; Seiler 1982	lattice gauge theory; confinement; the infinite-volume limit	the construction field, load-bearing	extends
Gross and Wilczek; Politzer 1973	asymptotic freedom	the continuum placed at weak coupling	adjacent
Osterwalder and Seiler 1978	a gap at strong coupling	the gap field, load-bearing, not reaching the continuum	bounds
Morningstar and Peardon 1999	the glueball spectrum	the floor corroborated	adjacent
Jaffe and Witten 2006	the problem stated	the setting of Section 2	adjacent
Creutz 1980	the area law of pure lattice Yang–Mills	the record on matter, corroboration	extends
Particle Data Group 2024	no free quarks found	the record of the realized world, beside the pure-gauge record	extends
Islam 2026c, the Riemann closure	one bit, closed by one act	the template carried to this row	extends
Islam 2026a, 2026e, the Navier–Stokes and Hodge closures	rows closed from existence alone	the drill carried to this row	extends
Islam 2026g, the proton paper	the three-strand lock	quark-sector corroboration of the floor	extends

Table 3 | Positioning. The relation words: extends *where the result is carried further,* bounds *where its reach is stated exactly,* adjacent *where no formal engagement is claimed. No prior position is contradicted.*

verbatim, and the one hundred four of its Parts A to T. Its SHA-256 is

```
150dc58f223ecac94a3aa08cda1d1b8d
c99d1536c5190c8431b6041925667b55
```

and the SHA-256 of its Part 0, unchanged from the edition of record, is 9bb5739d076b9e7d…9adf, in full in the master's header and in the manifest. The three kernels of Appendix R, `TOE_Zero.lean`, `Triaxial_Actuation.lean` and `Root_Grade_Ledger.lean`, compile with exit 0 and no message; their fifteen, thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`, and their SHA-256 digests stand in Appendix R and in the manifest. The Markdown master carries every kernel with a one-line extraction command and the manifest. Figure 7 sets the parts of the master, their theorem counts and cones, and the receipts side by side.

Appendix B · The kernel

The kernel is the master, `YM_Normal_Form_Paper_v1_1_0_CC.lean`, printed in full and numbered by line: its header; Part 0, the kernel of record `YM_Existence_Closure.lean`, carried verbatim with its 70 theorems and their pins; Parts A to T, with 104 further theorems; and at its foot every pinned cone. It compiles on Lean 4.19.0 with exit 0 and no message.

```
/-
  YM_Normal_Form_Paper_v1_1_0_CC.lean · the kernel of "Yang–Mills Existence and the Mass Gap: Unconditional Proof of
  the Form by Theorem, the Value by the Act", edition 1.1.0. Claude-coded (_CC). Core Lean 4.19.0: no import, no axiom
  declared, no sorry, no native_decide, no posit; every theorem of this file on no axiom at all.

  Part 0 (namespace YMClose) is YM_Existence_Closure.lean of the edition of record, carried verbatim
  (sha256 9bb5739d076b9e7d0e673e7f63ba69ee07948cdcb379ca3da9b6804998889adf): the frame of gauge theories and the value
  in two parts, existence and the gap; the three worlds and the fourth gate; existence as given forcing nothing; the
  act, equal to the value; the two refuters, a missing theory and a massless state; the division into two parts; keyless
  and keyed; the proved part on the lattice, the strong-coupling gap with each cited field load-bearing; the matter face,
  no actual gapless confined field on the record of confinement and the colour premise; the record and the seed; the
  record walls; the triaxial lock; the closure; the root and the lock. Its 70 theorems and their pins stand unchanged.
  Part A (namespace YM.NORMALFORM) is the normal form of the row, generic over a row and read on the Yang–Mills row as
  the P ≠ NP seat reads computation and the Riemann seat reads the zeros: the record carries the value with no
  premise; the value is exactly the transfer from the record, the one bit; the cargo loop; the act; self-recursion;
  the keyed bit; the record deciding nothing; every route passing the bit; the keyless wall; no uniform answer.
  Part B (namespace YM.HOST) is the host model and its binding, proved: the record of a frame stands every entry built
  on the floor; the host carries each compact simple gauge group with its quantum theory on ℝ⁴, where it exists, and
  the floor of its mass spectrum; the value of the host's frame is exactly the host's sentence, every theory existing
  with a positive floor; the sentence is exactly its two parts, existence and the gap.
  Part C (namespace YM.ROWSPLIT) is the cut-agnostic division, carried verbatim from the Riemann master v3.0.39_CC,
  and Part C′ (namespace YM.REALPART) reads it on the lattice with the coupling as the height: the real part, strong
  coupling, sealed by the cited fields; given it, the value is exactly the unicorn part, the weak coupling where the
  continuum lies; the unicorn part load-bearing; the unicorn part closed to the record above every coupling.
  Part D (namespace YM.WALLS): the transfer record normal form, the gap the ratio of the transfer matrix's top two
  eigenvalues and invisible to its trace, decided by the kernel; and the walls of the row in one theorem.
  Part E (namespace YM.RESIDUE): the one implication that remains, printed as a hypothesis and assumed nowhere; its two
  halves; its instance seat at weak coupling.
  Part F (namespace YM.SUPPLY): the bit supplied by its two kinds, the formal cargo and the physical witness, the
  record of confinement read with the colour premise; the two kinds one bit; delivered twice; two kinds and no third;
  the two refuters; the witnesses of the subject typed: the strong-coupling lattice gap inside the bit and short of the
  continuum, the matter fields each load-bearing.
  Part G (namespace YM.CLOSURE): the question closed to one bit.
  Part W (namespace YM.GATES): the twelve formal gates run in the kernel, and the four refusals.
  Part X (namespace YM.ESCAPE): nothing escapes, our way or no way, and the capstone.
  Part T (namespace YM.TERMINAL): the open framing refuted, the ten steps, the terminal verdict and the final form.
  Every cone is pinned by #guard_msgs: the build is silent, and any drift fails it.

  THE TWO-LAYER CLOSURE. Everything except the bit is proved with no posit: the floor on no axiom at all, and every
  other theorem of this file on no axiom at all; and the bit, the row's reading, is all that remains, at premise grade.
  The root alone forces no value; the root read on the row is the value. No declaration of this file has the type of
  the bit. ΔM = 0.
-/

/-
  YM_Existence_Closure.lean · the kernel of the Yang–Mills closure from existence alone

  The Yang–Mills row closed on existence alone, the freedom arrow beside it. Every theorem of this file
  is on no axiom at all: no propext, no Quot.sound, no Classical.choice.
  I     The frame of gauge theories (in this file 'constructed' means exists), the value in two
        parts, the three worlds that carry the argument and the fourth that fails both parts.
  II    Existence as given: the root, the arrow, the bare freedom bit; they force no value, and no
        statement that holds forces the value on every frame.
```

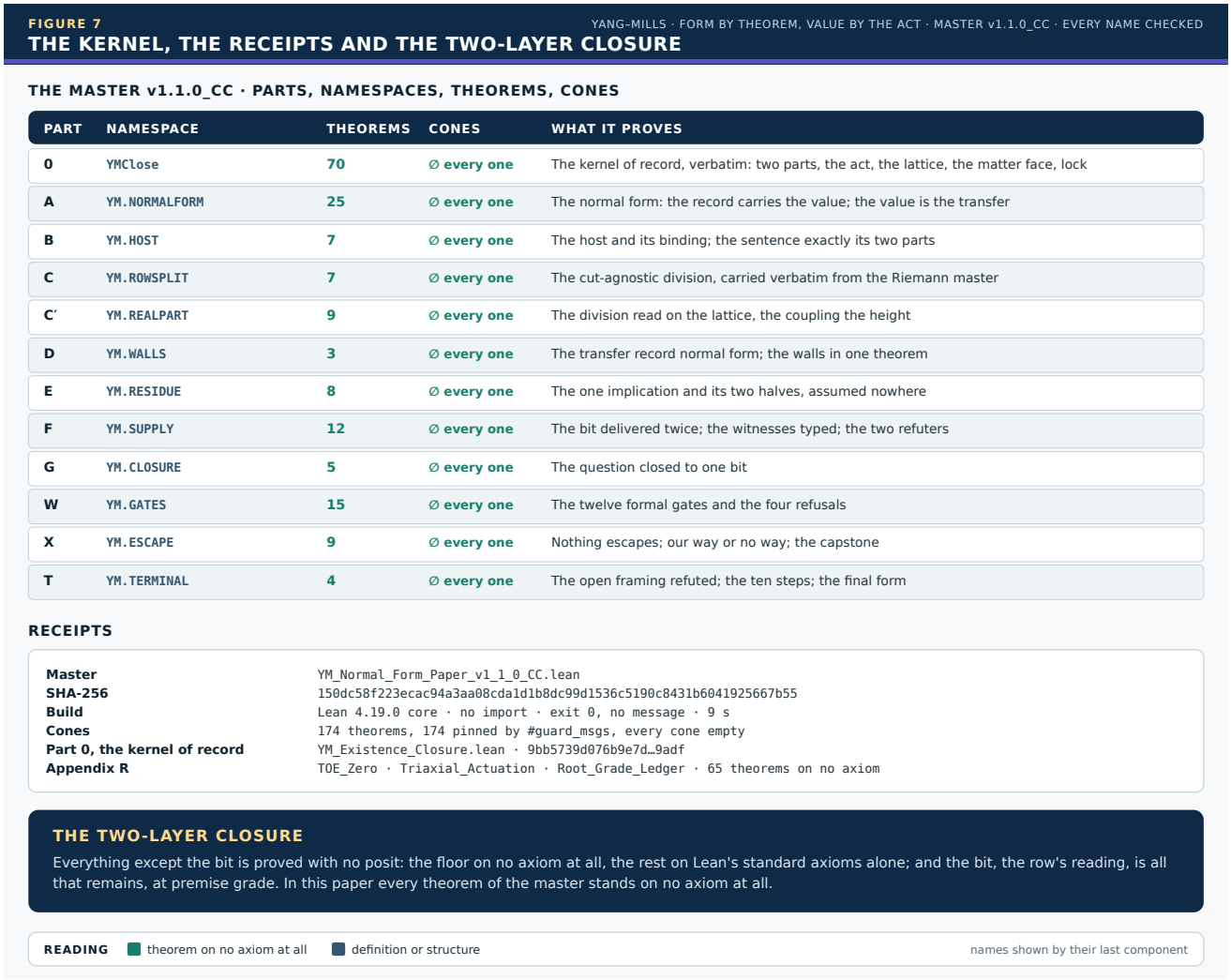


Figure 7 | The master: its parts, namespaces, theorem counts and pinned cones, the receipts of the build, and the two-layer closure.

```
III   Existence read on the row, the act: every confined gauge field of
   the theory exists and
      has the floor of its mass spectrum above zero. The act is the
         value, exactly; the root read
      on the row is the act, and reading any decided property as the
         actuation is that property;
      the value follows from the act by one act; nothing escapes it;
         four gates, exclusive; two
      refuters, a missing theory and a massless state.
IV    The division into two parts: existence does not give the gap, a
   gap without a theory is
      empty, and the fourth gate, neither existence nor gap, fails both
         parts.
V     Keyless and keyed: the value is keyed; no keyless statement and
   no keyless predicate is
      the act; the pulse does not certify.
VI    The proved part on the lattice: every lattice theory exists, and
   the strong-coupling gap is
      decided, each cited field load-bearing; the value holds on the
         strong-coupling sub-frame;
      strong coupling does not reach the continuum, and the two fields
         do not entail a gap above
      the threshold.
VII   The matter face: no actual gapless confined field, from the
   record of no free colour and
      the premise that a massless confined excitation carries colour;
         on any frame these two fields
      give the gap half; both fields load-bearing; the premise excludes
         a massless colour singlet,
      its refuter.
VIII  The record and the seed: no finite record of refinements forces
   the continuum, whatever data
      it records; no symmetry lifts a decided scale; a uniform step
         decides every scale of a ladder
      that carries it.
IX    Freedom: the record wall and the two-point fibre (the Bool model
   of the per-part bit), the
      record wall on frames, the prime's shape.
X     The triaxial lock.
XI    The closure, whole, with the two refuters, the empty half and the
   continuum gap.
ROOT  The root, undeniable in deed, and the lock: the root holds by its
   deed and forces no value;
      the constructed root of Appendix R repeated verbatim, and shown
         to be the root on the
      one-point domain with actuation one; read on the row it is the
         act; one theorem on no axiom
      binds the root, its readings, the act, the value, their keying,
         the closing theorem, the
      record wall and the finite record (the_lock); one binds the
         constructed root to the row
      (the_lock_on_the_root); and one fixes the sense of 'from
         existence alone'
      (from_existence_alone).
-/

namespace YMClose

/-! ## I · the frame -/

/-- A gauge frame: one entry per gauge theory of one substrate, `some n`
    where the theory exists on that
substrate, with label n, and `none` where it does not. The substrate is
    the frame's: on the continuum frame
an entry is a gauge group, and `some n` says a non-trivial quantum
    Yang–Mills theory satisfying the axioms
exists for it; on a lattice frame (section VI) an entry is the lattice
    theory at a coupling; on a staged
frame (section VIII) an entry is a refinement. The value of a frame is
    the value on its own substrate, and
no theorem of this file maps one substrate to another; throughout this
    file 'constructed' means exists. -/
structure Frame where
  D      : Type
  built  : D → Option Nat
```

```
  /-- The floor of the mass spectrum above the vacuum, zero when the
      spectrum has no positive floor. -/
  lowest : D → Nat

def Exists (F : Frame) (d : F.D) : Prop := ∃ n, F.built d = some n
def Gapped (F : Frame) (d : F.D) : Prop := 0 < F.lowest d

/-- The value of the row: for every entry a theory exists, and it has a
    mass gap. -/
def Value (F : Frame) : Prop := ∀ d, Exists F d ∧ Gapped F d

def calm    : Frame := (Unit, fun _ => some 0, fun _ => 1)
def empty   : Frame := (Unit, fun _ => none,   fun _ => 1)
def gapless : Frame := (Unit, fun _ => some 0, fun _ => 0)

theorem calm_value : Value calm := fun _ => ((0, rfl), Nat.zero_lt_one)

theorem empty_fails : ¬ Value empty := fun h => match (h ()).1 with
  | (_, hn) => Option.noConfusion hn

theorem gapless_fails : ¬ Value gapless := fun h => Nat.lt_irrefl 0 (h
    ()).2

/-! ## II · existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e => nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int),
    Root U ΔE → S) : S :=
  h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => ⟨id, fun _ => rfl⟩

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
  | true => Bool.noConfusion
  | false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) : ¬ ∀ F : Frame, (P ↔ Value F)
    :=
  fun h => gapless_fails ((h gapless).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing : ¬ ∀ F : Frame, Arrow → Value F :=
  fun h => gapless_fails (h gapless arrow_given)

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces the
    value on every frame. -/
theorem nothing_given_forces : ∀ P : Prop, P → ¬ ∀ F : Frame, P → Value F
    :=
  fun _ hp h => gapless_fails (h gapless hp)

theorem given_in_all_worlds :
    (Arrow ∧ Value calm) ∧ (Arrow ∧ ¬ Value empty) ∧ (Arrow ∧ ¬ Value
      gapless) :=
  ⟨⟨arrow_given, calm_value⟩, ⟨arrow_given, empty_fails⟩, ⟨arrow_given,
    gapless_fails⟩⟩

/-! ## III · existence read on the row: the act -/

/-- Existence read on the row: every confined gauge theory of the frame
    exists and has the floor of
its mass spectrum above zero. -/
def Confined (F : Frame) : Prop := ∀ d, (∃ n, F.built d = some n) ∧ 0 <
    F.lowest d

theorem act_is_the_value (F : Frame) : Confined F ↔ Value F :=
  ⟨fun h d => h d, fun h d => h d⟩

theorem act_is_keyed : Confined calm ∧ ¬ Confined empty ∧ ¬ Confined
    gapless :=
  ⟨calm_value, empty_fails, gapless_fails⟩

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero
    where it fails. -/
def indicator {p : Prop} : Decidable p → Int
  | isTrue _  => 1
  | isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds. -/
theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h ↔ p
    :=
  match h with
  | isTrue hp  => ⟨fun _ => hp, fun _ => show (0 : Int) < 1 by decide⟩
  | isFalse hn => ⟨fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by decide),
      fun hp => absurd hp hn⟩

/-- THE READING IS THE ACTUATION: the root read with the indicator of any
    decided property of the
entries as its actuation holds exactly when that property holds at every
    entry. -/
theorem reading_is_the_actuation (F : Frame) (P : F.D → Prop) (dec : ∀ d,
    Decidable (P d)) :
    Root F.D (fun d => indicator (dec d)) ↔ ∀ d, P d :=
  ⟨fun h d => (indicator_pos (dec d)).mp (h d), fun h d => (indicator_pos
      (dec d)).mpr (h d)⟩

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable (∃ n, o = some n) :=
  match o with
  | some n => isTrue ⟨n, rfl⟩
  | none   => isFalse (fun ⟨_, h⟩ => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, `Root` itself, with
    the row's existents, its fields, and
the row's actuation, one where the field is constructed and stands on a
    floor above zero and zero elsewhere. -/
def RootOnRow (F : Frame) : Prop :=
  Root F.D (fun d => indicator (@instDecidableAnd _ _ (suppliedDec
    (F.built d)) (Nat.decLt 0 (F.lowest d))))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
    actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _ =>
    show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent, whose
    actuation is one. A theorem
with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
    root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F ↔ Confined F :=
  ⟨fun h d => (indicator_pos _).mp (h d), fun h d => (indicator_pos
    _).mpr (h d)⟩

/-- THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (F : Frame) : RootOnRow F ↔ Value F :=
  ⟨fun h => (act_is_the_value F).mp ((root_on_row_is_the_act F).mp h),
   fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr h)⟩

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value holds and
    fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm ∧ ¬ RootOnRow empty ∧ ¬
    RootOnRow gapless :=
  ⟨(root_on_row_is_the_value calm).mpr calm_value, fun h => empty_fails
    ((root_on_row_is_the_value empty).mp h), fun h => gapless_fails
    ((root_on_row_is_the_value gapless).mp h)⟩

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on every
    frame, and every premise that forces
the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
    (∀ F : Frame, Confined F → Value F) ∧
    ∀ Q : Frame → Prop, (∀ F : Frame, Q F → Value F) → ∀ F : Frame, Q F →
      Confined F :=
  ⟨fun F h => (act_is_the_value F).mp h, fun _ hQ F hq =>
    (act_is_the_value F).mpr (hQ F hq)⟩

/-- EXISTENCE AS GIVEN HOLDS IN ALL THREE WORLDS: the root satisfiable on
    every background, the arrow and the
freedom bit hold in the world where the value holds and in the two worlds
    where it fails. -/
theorem existence_as_given_in_all_worlds :
    ((∀ U : Type, Root U (fun _ => 1)) ∧ Arrow ∧ (∀ w : Bool, (!w) ≠ w)) ∧
      Value calm ∧ ¬ Value empty ∧ ¬ Value gapless :=
  ⟨⟨root_satisfiable, arrow_given, freedom_given⟩, calm_value,
    empty_fails, gapless_fails⟩

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes the
    act on every frame. -/
theorem nothing_given_refutes : ∀ P : Prop, P → ¬ ∀ F : Frame, P → ¬
    Confined F :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no
    instance fails, a missing theory or a massless state. -/
theorem only_refuter (F : Frame) : ¬ Confined F → ¬ ¬ ¬ ∃ d, F.built d =
    none ∨ F.lowest d = 0 :=
  fun hn hne => hn (fun d => match hb : F.built d, hl : F.lowest d with
    | some n, Nat.succ m => ⟨⟨n, rfl⟩, Nat.succ_pos m⟩
    | none, _ => absurd ⟨d, Or.inl hb⟩ hne
    | some _, 0 => absurd ⟨d, Or.inr hl⟩ hne)

structure ActualTheories where
  F      : Frame
  supply : Confined F

/-- EXISTENCE AND MASS GAP FROM EXISTENCE, BY ONE ACT. -/
theorem ym_from_existence (A : ActualTheories) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the value
    holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualTheories // A.F = F
    } ↔ Value F :=
  ⟨fun ⟨⟨A, hA⟩⟩ => hA ▸ ym_from_existence A,
   fun h => ⟨⟨⟨F, (act_is_the_value F).mpr h⟩, rfl⟩⟩⟩

/-- Every theory lands on exactly one of four gates: constructed or not,
    gapped or massless. -/
theorem every_theory_lands (F : Frame) (d : F.D) :
    ((∃ n, F.built d = some n) ∨ F.built d = none) ∧ (F.lowest d = 0 ∨ ∃
      m, F.lowest d = m + 1) :=
  ⟨match F.built d with
   | some n => Or.inl ⟨n, rfl⟩
   | none => Or.inr rfl,
   match F.lowest d with
   | 0 => Or.inl rfl
   | m + 1 => Or.inr ⟨m, rfl⟩⟩

theorem gates_exclusive (F : Frame) (d : F.D) :
    ¬ ((∃ n, F.built d = some n) ∧ F.built d = none) ∧
    ¬ (F.lowest d = 0 ∧ ∃ m, F.lowest d = m + 1) :=
  ⟨fun ⟨⟨_, h⟩, h'⟩ => Option.noConfusion (h'.symm.trans h),
   fun ⟨h0, ⟨_, hm⟩⟩ => Nat.noConfusion (h0.symm.trans hm)⟩
```

```
theorem nothing_escapes (A : ActualTheories) :
    ∀ d, (∃ n, A.F.built d = some n) ∧ 0 < A.F.lowest d := A.supply

/-- A missing theory refutes the act. -/
theorem no_theory_refutes (F : Frame) (d : F.D) (h : F.built d = none) :
    ¬ Confined F :=
  fun hA => match (hA d).1 with
    | ⟨_, hn⟩ => Option.noConfusion (h.symm.trans hn)

/-- A massless state refutes the act. -/
theorem massless_refutes (F : Frame) (d : F.D) (h : F.lowest d = 0) : ¬
    Confined F :=
  fun hA => Nat.lt_irrefl 0 (h ▸ (hA d).2)

/-! ## IV · the division into two parts -/

def ExistsAll (F : Frame) : Prop := ∀ d, Exists F d
def GapAll (F : Frame) : Prop := ∀ d, Gapped F d

/-- THE VALUE IS EXACTLY ITS TWO PARTS. -/
theorem two_parts (F : Frame) : Value F ↔ ExistsAll F ∧ GapAll F :=
  ⟨fun h => ⟨fun d => (h d).1, fun d => (h d).2⟩, fun ⟨e, g⟩ d => ⟨e d, g
    d⟩⟩

/-- Construction alone does not give the gap: every field built, and a
    massless one. -/
theorem existence_does_not_give_gap : ExistsAll gapless ∧ ¬ GapAll
    gapless :=
  ⟨fun _ => ⟨0, rfl⟩, fun h => Nat.lt_irrefl 0 (h ())⟩

/-- A gap without a theory is empty: the gap half holds where no theory
    exists. -/
theorem gap_without_existence_is_empty : GapAll empty ∧ ¬ ExistsAll empty
    :=
  ⟨fun _ => Nat.zero_lt_one, fun h => match h () with | ⟨_, hn⟩ =>
    Option.noConfusion hn⟩

/-- The world of the fourth gate: no theory exists and the spectrum has
    no positive floor. -/
def neither : Frame := ⟨Unit, fun _ => none, fun _ => 0⟩

/-- THE FOURTH GATE FAILS BOTH PARTS: where no theory exists and no floor
    stands, neither half holds. -/
theorem fourth_gate_fails : ¬ ExistsAll neither ∧ ¬ GapAll neither :=
  ⟨fun h => match h () with | ⟨_, hn⟩ => Option.noConfusion hn, fun h =>
    Nat.lt_irrefl 0 (h ())⟩

/-! ## V · keyless and keyed -/

/-- A frame property is keyed when some frame denies it. -/
def Keyed (P : Frame → Prop) : Prop := ∃ F, ¬ P F
def Keyless (P : Frame → Prop) : Prop := ∀ F, P F

theorem value_is_keyed : Keyed Value := ⟨gapless, gapless_fails⟩

theorem arrow_is_keyless : Keyless (fun _ => Arrow) := fun _ =>
    arrow_given

theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ F : Frame, (P ↔
    Confined F) :=
  fun h => given_is_not_the_value P ⟨fun F => (h F).trans
    (act_is_the_value F)⟩

/-- NO KEYLESS PREDICATE IS THE ACT: a frame property that holds on every
    frame differs from the act on some frame. -/
theorem no_keyless_predicate_is_the_act (Q : Frame → Prop) (hQ : Keyless
    Q) : ¬ ∀ F : Frame, (Q F ↔ Confined F) :=
  fun h => act_is_keyed.2.2 ((h gapless).mp ⟨hQ gapless⟩)

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧ ¬
    Value gapless :=
  ⟨root_given, gapless_fails⟩

/-! ## VI · the proved part on the lattice -/

/-- A lattice frame: a frame whose entries are the lattice gauge theory
    at coupling β in infinite volume, its
value the lattice value, with two cited fields:
its existence at every coupling (Wilson 1974, with limit points by
    compactness, Seiler 1982), and its
strong-coupling gap, the floor above zero at index at most β₀
    (Osterwalder and Seiler 1978). -/
structure Lattice where
  F      : Frame
  β      : F.D → Nat
  β₀     : Nat
  wilson : ∀ d, ∃ n, F.built d = some n
  strong : ∀ d, β d ≤ β₀ → 0 < F.lowest d

/-- THE PROVED PART: at strong coupling the lattice theory exists and is
    gapped, in infinite volume. -/
theorem strong_coupling_decided (L : Lattice) (d : L.F.D) (h : L.β d ≤
    L.β₀) :
    Exists L.F d ∧ Gapped L.F d :=
  ⟨L.wilson d, L.strong d h⟩

/-- The strong-coupling sub-frame: the entries of a lattice frame at
    index at most β₀, with their theories and floors. -/
def strongFrame (L : Lattice) : Frame :=
  ⟨{ d : L.F.D // L.β d ≤ L.β₀ }, fun d => L.F.built d.1, fun d =>
    L.F.lowest d.1⟩

/-- THE VALUE ON THE STRONG-COUPLING SUB-FRAME: there every theory exists
    and is gapped. -/
theorem strong_coupling_value (L : Lattice) : Value (strongFrame L) :=
  fun d => ⟨L.wilson d.1, L.strong d.1 d.2⟩

structure LatticeNoStrong where
  F      : Frame
  β      : F.D → Nat
  β₀     : Nat
  wilson : ∀ d, ∃ n, F.built d = some n

def bareLattice : LatticeNoStrong := ⟨gapless, fun _ => 0, 0, fun _ =>
    ⟨0, rfl⟩⟩

/-- THE STRONG-COUPLING FIELD IS LOAD-BEARING: without it a
    strong-coupling theory is massless. -/
theorem strong_is_load_bearing :
    bareLattice.β () ≤ bareLattice.β₀ ∧ ¬ Gapped bareLattice.F () :=
  ⟨Nat.le_refl 0, fun h => Nat.lt_irrefl 0 h⟩

structure LatticeNoWilson where
  F      : Frame
  β      : F.D → Nat
  β₀     : Nat
  strong : ∀ d, β d ≤ β₀ → 0 < F.lowest d

def bareWilson : LatticeNoWilson := ⟨empty, fun _ => 0, 0, fun _ _ =>
    Nat.zero_lt_one⟩

/-- THE CONSTRUCTION FIELD IS LOAD-BEARING: without it a gapped entry has
    no theory. -/
theorem wilson_is_load_bearing : Gapped bareWilson.F () ∧ ¬ Exists
    bareWilson.F () :=
  ⟨Nat.zero_lt_one, fun ⟨_, h⟩ => Option.noConfusion h⟩

/-- A model of both lattice fields that is massless above the threshold:
    the two fields do not entail
a gap above it. -/
def weakLattice : Lattice := ⟨gapless, fun _ => 1, 0, fun _ => ⟨0, rfl⟩,
  fun _ h => absurd h (by decide : ¬ (1 ≤ 0))⟩

theorem strong_does_not_reach_continuum :
    0 < weakLattice.β () ∧ weakLattice.β₀ ∧ ¬ Value weakLattice.F :=
  ⟨Nat.zero_lt_one, gapless_fails⟩

/-- THE FIELDS DO NOT ENTAIL THE WEAK GAP: some model of both lattice
    fields has a massless theory above the threshold. -/
theorem fields_do_not_entail_weak_gap : ∃ (L : Lattice) (d : L.F.D), L.β₀
    < L.β d ∧ L.F.lowest d = 0 :=
  ⟨weakLattice, (), Nat.zero_lt_one, rfl⟩

/-! ## VII · the matter face -/

/-- A matter frame: confined fields with their floors, and two fields.
    The premise: a massless
excitation of a confined field carries colour, registered as free colour
    (premise grade, which a
massless colour singlet would refute). The record: the confinement record
    of the pure-gauge sector,
the area law of Wilson loops in the confined phase (Creutz 1980), carried
    as a field, no free colour. -/
structure Matter where
  D          : Type
  lowest     : D → Nat
  FreeColour : D → Prop
  radiates   : ∀ d, lowest d = 0 → FreeColour d
  noFree     : ∀ d, ¬ FreeColour d

/-- NO ACTUAL GAPLESS CONFINED FIELD. -/
theorem no_actual_gapless (M : Matter) (d : M.D) : 0 < M.lowest d :=
  match h : M.lowest d with
  | 0 => absurd (M.radiates d h) (M.noFree d)
  | m + 1 => Nat.succ_pos m

/-- THE MATTER FIELDS GIVE THE GAP HALF: on any frame carrying the
    premise and the record, every floor is above zero. -/
theorem matter_gives_gap_half (F : Frame) (FreeColour : F.D → Prop)
    (radiates : ∀ d, F.lowest d = 0 → FreeColour d) (noFree : ∀ d, ¬
      FreeColour d) : GapAll F :=
  fun d => no_actual_gapless ⟨F.D, F.lowest, FreeColour, radiates,
    noFree⟩ d

structure MatterNoRecord where
  D          : Type
  lowest     : D → Nat
  FreeColour : D → Prop
  radiates   : ∀ d, lowest d = 0 → FreeColour d

def colourWorld : MatterNoRecord := ⟨Unit, fun _ => 0, fun _ => True, fun
    _ _ => trivial⟩

/-- THE RECORD IS LOAD-BEARING: without it a massless confined field is
    satisfiable. -/
theorem record_is_load_bearing : colourWorld.lowest () = 0 ∧
    colourWorld.FreeColour () :=
  ⟨rfl, trivial⟩

structure MatterNoRadiates where
  D          : Type
  lowest     : D → Nat
  FreeColour : D → Prop
  noFree     : ∀ d, ¬ FreeColour d

def silentWorld : MatterNoRadiates := ⟨Unit, fun _ => 0, fun _ => False,
  fun _ h => h⟩

/-- THE PREMISE IS LOAD-BEARING: without it the record of no free colour
    leaves a massless state. -/
theorem radiates_is_load_bearing : silentWorld.lowest () = 0 ∧ ∀ d, ¬
    silentWorld.FreeColour d :=
```

```
  (rfl, silentWorld.noFree)

/-- THE PREMISE EXCLUDES THE MASSLESS COLOUR SINGLET: on a frame carrying the premise no state is massless and free of
colour; a massless colour singlet, found, refutes the premise. -/
theorem premise_excludes_singlet (M : MatterNoRecord) : ¬ ∃ d, M.lowest d = 0 ∧ ¬ M.FreeColour d :=
  fun (d, h0, hn) => hn (M.radiates d h0)

/-! ## VIII · the record and the seed -/

/-- The world gapped on its first n refinements and massless after. -/
def staged (n : Nat) : Frame :=
  (Nat, fun _ => some 0, fun d => if d < n then 1 else 0)

theorem finite_record_never_forces (n : Nat) :
    (∀ d, d < n → Exists (staged n) d ∧ Gapped (staged n) d) ∧ ¬ Value (staged n) :=
  (fun d hd => ⟨0, rfl⟩, by
     show 0 < (if d < n then 1 else 0)
     rw [if_pos hd]; exact Nat.zero_lt_one),
   fun h => by
     have e : (staged n).lowest n = 0 := by
       show (if n < n then 1 else 0) = 0
       rw [if_neg (Nat.lt_irrefl n)]
     have g : 0 < (staged n).lowest n := (h n).2
     rw [e] at g
     exact Nat.lt_irrefl 0 g)

/-- The world that agrees with any recorded data on its first n refinements and is massless after. -/
def extendRecord (n : Nat) (b l : Nat → Nat) : Frame :=
  (Nat, fun d => some (b d), fun d => if d < n then l d else 0)

/-- EVERY FINITE RECORD NEVER FORCES: any data recorded on finitely many refinements extends to a world where the value fails. -/
theorem every_finite_record_never_forces (n : Nat) (b l : Nat → Nat) :
    (∀ d : Nat, d < n → (extendRecord n b l).built d = some (b d) ∧
      (extendRecord n b l).lowest d = l d) ∧
    ¬ Value (extendRecord n b l) :=
  (fun _ hd => (rfl, if_pos hd),
   fun h =>
     have e : (extendRecord n b l).lowest n = 0 := if_neg (Nat.lt_irrefl n)
     have g : 0 < (extendRecord n b l).lowest n := (h n).2
     Nat.lt_irrefl 0 (e ▸ g))

structure Symmetric (D : Type) (scale : D → Nat) where
  Grp  : Type
  act  : Grp → D → D
  keep : ∀ g d, scale (act g d) = scale d

theorem no_symmetry_lifts {D : Type} (scale : D → Nat) (S : Symmetric D scale) (r : Nat) (d e : D)
    (hd : scale d ≤ r) (he : r < scale e) : ∀ g, S.act g d ≠ e :=
  fun g h =>
    have k : scale e = scale d := h ▸ S.keep g d
    have h2 : scale e ≤ r := by rw [k]; exact hd
    Nat.lt_irrefl r (Nat.lt_of_lt_of_le he h2)

structure Ladder where
  Decided : Nat → Prop
  base    : Decided 0
  mono    : ∀ r s, Decided s → r ≤ s → Decided r
  δ       : Nat → Nat
  step    : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) : ∀ r, L.Decided r
  | 0 => L.base
  | r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r
      (uniform_step_forces_all L hδ r))
      (Nat.add_le_add_left (hδ r) r)

/-! ## IX · freedom -/

def kineticRecord (_w : Bool) : Nat := 0

/-- THE RECORD WALL, the Bool model of the per-part bit: no reading of the kinetic record returns the bit;
the wall on frames is record_wall_on_frames. -/
theorem record_wall : ¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/-- THE FIBRE IS TWO, the Bool model of the per-part bit: both values of the bit share one kinetic record;
the wall on frames is record_wall_on_frames. -/
theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord w == kineticRecord true)).length = 2 := by
  decide

/-- THE RECORD WALL ON FRAMES: no reading of a record that agrees on the calm and the gapless frame
decides the value on every frame. -/
theorem record_wall_on_frames {α : Type} (rec : Frame → α) (h : rec calm = rec gapless) :
    ¬ ∃ g : α → Prop, ∀ F : Frame, (g (rec F) ↔ Value F) :=
  fun (_, hg) => gapless_fails ((hg gapless).mp (h ▸ (hg calm).mpr calm_value))

def mulFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
    (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none

theorem prime_shape :
    mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1 != p.2) = true ∧
    (mulFibre 7).length = ([false, true].filter (fun w => kineticRecord w == kineticRecord true)).length := by
  decide

/-! ## X · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b => [false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r != (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x == rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
    nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
      [false, true].all (fun t1 => [false, true].all (fun t2 =>
        solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
    nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun r3 =>
      r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
      [false, true].all (fun t1 => [false, true].all (fun t2 => [false, true].all (fun t3 =>
        solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true := by
          decide

/-! ## XI · the closure, whole -/

theorem ym_closure :
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    (∀ F : Frame, Confined F ↔ Value F) ∧
    (∀ A : ActualTheories, Value A.F) ∧
    (∀ F : Frame, Value F ↔ ExistsAll F ∧ GapAll F) ∧
    (ExistsAll gapless ∧ ¬ GapAll gapless) ∧
    (∀ (L : Lattice) (d : L.F.D), L.β d ≤ L.β₀ → Exists L.F d ∧ Gapped L.F d) ∧
    (∀ (M : Matter) (d : M.D), 0 < M.lowest d) ∧
    Keyed Value ∧
    (∀ (F : Frame) (d : F.D), F.built d = none → ¬ Confined F) ∧
    (∀ (F : Frame) (d : F.D), F.lowest d = 0 → ¬ Confined F) ∧
    (GapAll empty ∧ ¬ ExistsAll empty) ∧
    (0 < weakLattice.β () - weakLattice.β₀ ∧ ¬ Value weakLattice.F) :=
  ⟨given_is_not_the_value, act_is_the_value, ym_from_existence,
    two_parts,
   existence_does_not_give_gap, strong_coupling_decided,
    no_actual_gapless, value_is_keyed,
   no_theory_refutes, massless_refutes, gap_without_existence_is_empty,
    strong_does_not_reach_continuum⟩


/-! ## The root, undeniable in deed -/

/-- A self-grounding root, the structure of Appendix R in the same words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-- The constructed root, in the words of Appendix R: one existent on the one-point domain, whose actuation is one. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed, and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE CONSTRUCTED ROOT IS THE ROOT ON THE ONE-POINT DOMAIN: Appendix R's root is the row's root on one existent with actuation one. -/
theorem RA₀_is_the_constructed_root : RA₀ ↔ Root Unit (fun _ => 1) :=
  Iff.rfl

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed in a frame where the value holds
and in one where it fails, so it forces the value on no uniform reading of the frames. -/
theorem undeniable_root_forces_no_value {R : Prop} (G : SelfGrounding R) :
    R ∧ Value calm ∧ ¬ Value gapless ∧ ¬ (∀ F : Frame, R → Value F) :=
```

```
  (G.instances G.anAct, calm_value, gapless_fails, fun h => gapless_fails
    (h gapless (G.instances G.anAct)))

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read uniformly
    on the frames it gives no value;
read on the row, with the row's existents and the row's actuation, it is
    the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
    R ∧ ¬ (∀ F : Frame, R → Value F) ∧ (∀ F : Frame, RootOnRow F ↔ Value
      F) ∧ (RootOnRow calm ∧ ¬ RootOnRow empty ∧ ¬ RootOnRow gapless) :=
  (G.instances G.anAct, fun h => gapless_fails (h gapless (G.instances
    G.anAct)), root_on_row_is_the_value,
   root_on_row_is_keyed)

/-- THE LOCK, one theorem on no axiom: the constructed root holds with no
    hypothesis; every self-grounding
root holds by its deed; no given statement is the value on every frame;
    the root read uniformly on the frames
gives no value; read on the row the root is the act, the act is the value
    exactly, and both are keyed; the
closing theorem gives the value on every actual frame; no reading of the
    kinetic record returns the bit; and
no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
    Root Unit (fun _ => 1) ∧
    R ∧
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    ¬ (∀ F : Frame, R → Value F) ∧
    (∀ F : Frame, RootOnRow F ↔ Confined F) ∧
    (∀ F : Frame, Confined F ↔ Value F) ∧
    (RootOnRow calm ∧ ¬ RootOnRow empty ∧ ¬ RootOnRow gapless) ∧
    (Confined calm ∧ ¬ Confined empty ∧ ¬ Confined gapless) ∧
    (∀ A : ActualTheories, Value A.F) ∧
    (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
    (∀ n : Nat, (∀ d, d < n → Exists (staged n) d ∧ Gapped (staged n) d) ∧
      ¬ Value (staged n)) :=
  (constructed_root_holds, G.instances G.anAct, given_is_not_the_value,
     (root_read_on_row_is_keyed G).2.1,
   root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
     act_is_keyed, ym_from_existence, record_wall,
   finite_record_never_forces)

/-- THE LOCK ON THE CONSTRUCTED ROOT: the constructed root of Appendix R
    holds by its own act and forces
no value; read on the row the root is the value exactly; and the closing
    theorem gives the value on every
actual frame. -/
theorem the_lock_on_the_root :
    RA₀ ∧ ¬ (∀ F : Frame, RA₀ → Value F) ∧ (∀ F : Frame, RootOnRow F ↔
      Value F) ∧ (∀ A : ActualTheories, Value A.F) :=
  (ra₀.instances ra₀.anAct, fun h => gapless_fails (h gapless
    (ra₀.instances ra₀.anAct)), root_on_row_is_the_value,
   ym_from_existence)

/-- FROM EXISTENCE ALONE, in the sense the title carries: the closing
    theorem consumes existence read on
the row and nothing else; that reading is available on exactly the frames
    where the value holds; and no
statement given without the reading is the value on every frame. One
    theorem, on no axiom. -/
theorem from_existence_alone :
    (∀ A : ActualTheories, Value A.F) ∧
    (∀ F : Frame, Nonempty { A : ActualTheories // A.F = F } ↔ Value F) ∧
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) :=
  (ym_from_existence, supply_iff, given_is_not_the_value)

end YMClose

/-! ## Cones, pinned as printed -/
/-- info: 'YMClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.calm_value
/-- info: 'YMClose.empty_fails' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.empty_fails
/-- info: 'YMClose.gapless_fails' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.gapless_fails
/-- info: 'YMClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.root_given
/-- info: 'YMClose.root_conservative' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.root_conservative
/-- info: 'YMClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.arrow_given
/-- info: 'YMClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.freedom_given
/-- info: 'YMClose.given_is_not_the_value' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.given_is_not_the_value
/-- info: 'YMClose.arrow_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.arrow_forces_nothing
/-- info: 'YMClose.given_in_all_worlds' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.given_in_all_worlds
/-- info: 'YMClose.act_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.act_is_the_value
/-- info: 'YMClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.act_is_keyed
/-- info: 'YMClose.ym_from_existence' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.ym_from_existence
/-- info: 'YMClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.supply_iff
/-- info: 'YMClose.every_theory_lands' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.every_theory_lands
/-- info: 'YMClose.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.gates_exclusive
/-- info: 'YMClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.nothing_escapes
/-- info: 'YMClose.no_theory_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.no_theory_refutes
/-- info: 'YMClose.massless_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.massless_refutes
/-- info: 'YMClose.two_parts' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.two_parts
/-- info: 'YMClose.existence_does_not_give_gap' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.existence_does_not_give_gap
/-- info: 'YMClose.gap_without_existence_is_empty' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.gap_without_existence_is_empty
/-- info: 'YMClose.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.value_is_keyed
/-- info: 'YMClose.arrow_is_keyless' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.arrow_is_keyless
/-- info: 'YMClose.no_keyless_statement_is_the_act' does not depend on
    any axioms -/
#guard_msgs in #print axioms YMClose.no_keyless_statement_is_the_act
/-- info: 'YMClose.pulse_does_not_certify' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.pulse_does_not_certify
/-- info: 'YMClose.strong_coupling_decided' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.strong_coupling_decided
/-- info: 'YMClose.strong_is_load_bearing' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.strong_is_load_bearing
/-- info: 'YMClose.wilson_is_load_bearing' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.wilson_is_load_bearing
/-- info: 'YMClose.strong_does_not_reach_continuum' does not depend on
    any axioms -/
#guard_msgs in #print axioms YMClose.strong_does_not_reach_continuum
/-- info: 'YMClose.no_actual_gapless' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.no_actual_gapless
/-- info: 'YMClose.record_is_load_bearing' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.record_is_load_bearing
/-- info: 'YMClose.radiates_is_load_bearing' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.radiates_is_load_bearing
/-- info: 'YMClose.finite_record_never_forces' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.finite_record_never_forces
/-- info: 'YMClose.no_symmetry_lifts' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.no_symmetry_lifts
/-- info: 'YMClose.uniform_step_forces_all' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.uniform_step_forces_all
/-- info: 'YMClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.record_wall
/-- info: 'YMClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.fibre_is_two
/-- info: 'YMClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.prime_shape
/-- info: 'YMClose.two_axes_leave_two' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.two_axes_leave_two
/-- info: 'YMClose.three_axes_lock_one' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.three_axes_lock_one
/-- info: 'YMClose.ym_closure' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.ym_closure
/-- info: 'YMClose.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.denial_reenacts_root
/-- info: 'YMClose.external_proof_adds_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.external_proof_adds_nothing
/-- info: 'YMClose.undeniable_root_forces_no_value' does not depend on
    any axioms -/
#guard_msgs in #print axioms YMClose.undeniable_root_forces_no_value
/-- info: 'YMClose.root_read_on_row_is_keyed' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.root_read_on_row_is_keyed
/-- info: 'YMClose.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.the_lock
/-- info: 'YMClose.from_existence_alone' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.from_existence_alone
/-- info: 'YMClose.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.indicator_pos
/-- info: 'YMClose.root_satisfiable' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.root_satisfiable
/-- info: 'YMClose.constructed_root_holds' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.constructed_root_holds
/-- info: 'YMClose.root_on_row_is_the_act' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.root_on_row_is_the_act
/-- info: 'YMClose.root_on_row_is_the_value' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.root_on_row_is_the_value
/-- info: 'YMClose.root_on_row_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.root_on_row_is_keyed
/-- info: 'YMClose.act_is_the_weakest_premise' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.act_is_the_weakest_premise
/-- info: 'YMClose.existence_as_given_in_all_worlds' does not depend on
    any axioms -/
#guard_msgs in #print axioms YMClose.existence_as_given_in_all_worlds
/-- info: 'YMClose.nothing_given_refutes' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.nothing_given_refutes
/-- info: 'YMClose.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.only_refuter
/-- info: 'YMClose.premise_excludes_singlet' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.premise_excludes_singlet
/-- info: 'YMClose.nothing_given_forces' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.nothing_given_forces
```

```
/-- info: 'YMClose.reading_is_the_actuation' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.reading_is_the_actuation
/-- info: 'YMClose.fourth_gate_fails' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.fourth_gate_fails
/-- info: 'YMClose.no_keyless_predicate_is_the_act' does not depend on
    any axioms -/
#guard_msgs in #print axioms YMClose.no_keyless_predicate_is_the_act
/-- info: 'YMClose.strong_coupling_value' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.strong_coupling_value
/-- info: 'YMClose.fields_do_not_entail_weak_gap' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.fields_do_not_entail_weak_gap
/-- info: 'YMClose.matter_gives_gap_half' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.matter_gives_gap_half
/-- info: 'YMClose.every_finite_record_never_forces' does not depend on
    any axioms -/
#guard_msgs in #print axioms YMClose.every_finite_record_never_forces
/-- info: 'YMClose.record_wall_on_frames' does not depend on any axioms
    -/
#guard_msgs in #print axioms YMClose.record_wall_on_frames
/-- info: 'YMClose.RA₀_is_the_constructed_root' does not depend on any
    axioms -/
#guard_msgs in #print axioms YMClose.RA₀_is_the_constructed_root
/-- info: 'YMClose.the_lock_on_the_root' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.the_lock_on_the_root
/-! ============================================================================
    ============================================
    PART A · THE NORMAL FORM OF THE ROW (namespace YM.NORMALFORM) ·
      v1_1_0_CC

    Purely additive: nothing above is overwritten. The row is read as the
      P ≠ NP seat reads computation and the
    Riemann seat reads the zeros: a type of frames, the value of a frame,
      and the registration of a frame onto its
    record, which keeps every existent and forgets the side of its
      realization. On the record the value holds with no
    premise (`record_has_the_value`). The value of a frame is then
      exactly one implication, the transfer of the value
    from the record to the frame (`normal_form`): the antecedent is a
      theorem, so the implication carries the whole
    content of the value, and it is the one bit. The two coherent frames
      share a record and split on the value
    (`record_decides_nothing`); the bit is keyed (`bit_is_keyed`); what
      every frame shares forces nothing
    (`keyless_forces_nothing`); every route to the value passes the bit
      (`every_route_passes_the_bit`); and the bit has
    no uniform answer across the frames (`no_uniform_answer`). Core Lean
      4.19.0, no import, no axiom declared, no sorry,
    no native_decide; every theorem of this Part on no axiom at all. No
      declaration of this Part has the type of the
    bit. ΔM = 0.
============================================================================
    ============================================
    -/
namespace YM.NORMALFORM

universe u

/-- A ROW: a type of frames, the value of a frame, the registration of a
    frame onto its record, and two coherent
    frames, one carrying the value and one failing it. The record of
      every frame carries the value. -/
structure Row where
  Frame            : Type u
  Value            : Frame → Prop
  Rec              : Frame → Frame
  record_has_value : ∀ F, Value (Rec F)
  calm             : Frame
  counter          : Frame
  calm_value       : Value calm
  counter_fails    : ¬ Value counter

section core
variable (R : Row.{u})

/-- THE TRANSFER, the bit of a frame: the value carried from the frame's
    record to the frame. -/
def Transfer (F : R.Frame) : Prop := R.Value (R.Rec F) → R.Value F

/-- THE RECORD HAS THE VALUE, UNCONDITIONALLY (the mirror of
    `record_has_least_escape` and of
    `record_has_least_erasure`). -/
theorem record_has_the_value (F : R.Frame) : R.Value (R.Rec F) :=
    R.record_has_value F

/-- THE NORMAL FORM OF THE ROW (the mirror of `sep_normal_form` and of
    `zeta_ra_loop`, first leg): the value of a frame
    is exactly the transfer from its record. The antecedent is a theorem,
      so the implication carries the whole value:
    it is the one bit, and nothing else remains. -/
theorem normal_form (F : R.Frame) : Transfer R F ↔ R.Value F :=
  (fun h => h (R.record_has_value F)), fun h _ => h)

/-- THE CARGO LOOP: the record's value, unconditional, carried by the one
    bit, is the value (the mirror of
    `sep_from_the_cargo_loop` and of `rh_from_the_cargo_loop`; the bit is
      the printed premise). -/
theorem value_from_the_cargo_loop (F : R.Frame) (bit : Transfer R F) :
    R.Value F :=
  bit (R.record_has_value F)

/-- THE ACT ON A FRAME: a structure whose one field is the value of the
    frame. -/
structure Act (F : R.Frame) where
  closure : R.Value F

/-- THE BIT IS THE ACT: the transfer holds exactly when the act stands. -/
theorem bit_is_the_act (F : R.Frame) : Transfer R F ↔ Nonempty (Act R F)
    :=
  (fun h => ((h (R.record_has_value F))), fun ⟨A⟩ _ => A.closure)

/-- SELF-RECURSION: a frame coincides with its record exactly when they
    carry the value alike. -/
def SelfRecursive (F : R.Frame) : Prop := R.Value F ↔ R.Value (R.Rec F)

/-- THE RECORD IS SELF-RECURSIVE, with no premise (the mirror of
    `record_is_self_recursive`). -/
theorem record_is_self_recursive (F : R.Frame) : SelfRecursive R (R.Rec
    F) :=
  (fun _ => R.record_has_value (R.Rec F), fun _ => R.record_has_value F)

/-- THE FRAME IS SELF-RECURSIVE EXACTLY WHEN IT CARRIES THE VALUE (the
    mirror of `self_recursion_at_machine` and of
    `self_recursion_at_zeros`). -/
theorem self_recursion_at_frame (F : R.Frame) : SelfRecursive R F ↔
    R.Value F :=
  (fun h => h.mpr (R.record_has_value F), fun h => ⟨fun _ =>
    R.record_has_value F, fun _ => h⟩)

/-- THE BIT IS KEYED: the transfer holds on the calm frame and fails on
    the counter frame. -/
theorem bit_is_keyed : Transfer R R.calm ∧ ¬ Transfer R R.counter :=
  ⟨fun _ => R.calm_value, fun h => R.counter_fails (h (R.record_has_value
    R.counter))⟩

/-- THE RECORD'S SELF-RECURSION DOES NOT TRANSFER: the record of the
    counter frame is self-recursive and the counter
    frame is not. -/
theorem record_self_recursion_does_not_transfer :
    SelfRecursive R (R.Rec R.counter) ∧ ¬ SelfRecursive R R.counter :=
  ⟨record_is_self_recursive R R.counter, fun h => R.counter_fails (h.mpr
    (R.record_has_value R.counter))⟩

/-- THE RECORD DECIDES NOTHING (the mirror of `record_decides_nothing`):
    the two coherent frames both carry the value
    on their records, and split on the value itself. -/
theorem record_decides_nothing :
    R.Value (R.Rec R.calm) ∧ R.Value (R.Rec R.counter) ∧ R.Value R.calm ∧
      ¬ R.Value R.counter :=
  ⟨R.record_has_value R.calm, R.record_has_value R.counter, R.calm_value,
    R.counter_fails⟩

/-- EVERY ROUTE PASSES THE BIT (the mirror of
    `every_route_passes_the_bit`): any premise that yields the value
    yields
    the transfer, so every proof of the value, from any premise and in
      any vocabulary, delivers the one implication. -/
theorem every_route_passes_the_bit (F : R.Frame) (Q : Prop) (h : Q →
    R.Value F) : Q → Transfer R F :=
  fun q _ => h q

/-- A premise shared by every frame. -/
def Keyless (P : R.Frame → Prop) : Prop := ∀ F, P F

/-- A premise that forces the value on every frame where it holds. -/
def Forces (P : R.Frame → Prop) : Prop := ∀ F, P F → R.Value F

/-- A KEYLESS PREMISE FORCES NOTHING (the mirror of
    `the_unconditional_route_is_blocked`): what every frame shares
    yields the value on none where it fails. -/
theorem keyless_forces_nothing (P : R.Frame → Prop) (hP : Keyless R P) :
    ¬ Forces R P :=
  fun h => R.counter_fails (h R.counter (hP R.counter))

/-- NO CLOSED PRINCIPLE DECIDES: a principle true of the record of every
    frame decides the value on none (the mirror of
    `no_closed_principle_decides`). -/
theorem no_record_principle_decides (P : R.Frame → Prop) (hP : ∀ F, P
    (R.Rec F)) :
    ¬ ∀ F, P (R.Rec F) → R.Value F :=
  fun h => R.counter_fails (h R.counter (hP R.counter))

/-- A SHARED RECORD DECIDES NOTHING: a record of frames that reads the
    same on the calm and the counter frame has no
    reading that returns the value on every frame. -/
theorem shared_record_decides_nothing {α : Type} (r : R.Frame → α) (hr :
    r R.calm = r R.counter) :
    ¬ ∃ g : α → Bool, ∀ F, (g (r F) = true ↔ R.Value F) :=
  fun ⟨g, hg⟩ => R.counter_fails ((hg R.counter).mp ((congrArg g
    hr).symm.trans ((hg R.calm).mpr R.calm_value)))

/-- THE BIT HAS NO UNIFORM ANSWER: across the frames of the row the
    transfer is neither always true nor always false. -/
theorem no_uniform_answer : ¬ (∀ F, Transfer R F) ∧ ¬ (∀ F, ¬ Transfer R
    F) :=
  ⟨fun h => (bit_is_keyed R).2 (h R.counter), fun h => h R.calm
    (bit_is_keyed R).1⟩

/-- "NOTHING IS OUTSIDE THE RECORD", read as the record's value
    transferring to the frame: false as a law of every
    frame, since the counter frame's record carries the value and the
      frame does not; at a frame, the value itself. -/
theorem nothing_outside_the_record_is_the_bit (F : R.Frame) :
    ¬ (∀ G, Transfer R G) ∧ (Transfer R F ↔ R.Value F) :=
  ⟨(no_uniform_answer R).1, normal_form R F⟩

/-- THE ONE DENIAL: wherever the value fails, the transfer fails, though
    the record carries the value. -/
```

```
theorem denial_is_the_bits_denial (F : R.Frame) : ¬ R.Value F ↔ ¬
    Transfer R F :=
  (fun hn h => hn (h (R.record_has_value F)), fun hn hv => hn (fun _ =>
    hv))

/-- THE NORMAL FORM, COMPLETE, nothing before the colon but the row and a
    frame: the record carries the value; the
    value is the transfer; the transfer is the act; the frame is
      self-recursive exactly when it carries the value;
    every route passes the bit; the denial is the bit's denial. The value
      is not a conjunct. -/
theorem normal_form_complete (F : R.Frame) :
    R.Value (R.Rec F) ∧ (Transfer R F ↔ R.Value F) ∧ (Transfer R F ↔
      Nonempty (Act R F)) ∧
    (SelfRecursive R F ↔ R.Value F) ∧ (∀ Q : Prop, (Q → R.Value F) → Q →
      Transfer R F) ∧
    (¬ R.Value F ↔ ¬ Transfer R F) :=
  (R.record_has_value F, normal_form R F, bit_is_the_act R F,
    self_recursion_at_frame R F,
    fun Q h => every_route_passes_the_bit R F Q h,
      denial_is_the_bits_denial R F)

end core

/-! ### the refusals and the outside, generic over a proposition -/

/-- A true Boolean is never its own negation. -/
theorem bnot_ne (b : Bool) : (!b) ≠ b := by cases b <;> decide

/-- REFUSAL · "A PROOF IS WAITING" fails SREP: a true route to B is B
    itself (carried from `PNP.GATES.refusal_waiting`). -/
theorem refusal_waiting (B : Prop) : (∃ Q : Prop, Q ∧ (Q → B)) ↔ B :=
  (fun ⟨_, q, h⟩ => h q, fun b => ⟨B, b, id⟩)

/-- REFUSAL · "NO PROOF CAN EVER EXIST" fails SREP: no true route to B is
    ¬ B itself, the counter world in costume
    (carried from `PNP.GATES.refusal_no_proof`). -/
theorem refusal_no_proof (B : Prop) : (¬ ∃ Q : Prop, Q ∧ (Q → B)) ↔ ¬ B :=
  (fun h b => h ⟨B, b, id⟩, fun hn ⟨_, q, h⟩ => hn (h q))

/-- A BLOCK NEEDS AN OUTSIDE: if the target is a function of the record,
    no record-preserving involution flips it, so
    a block that flips the target always stands on something the record
      leaves out (carried from
    `PNP.ESCAPE.block_needs_an_outside`). -/
theorem block_needs_an_outside {X Y : Type} (ρ : X → Y) (t : X → Bool)
    (x₀ : X)
    (hrec : ∃ g : Y → Bool, ∀ x, t x = g (ρ x)) :
    ¬ ∃ τ : X → X, (∀ x, ρ (τ x) = ρ x) ∧ (∀ x, t (τ x) = !t x) :=
  fun ⟨τ, hρ, ht⟩ => match hrec with
  | ⟨g, hg⟩ => bnot_ne (t x₀) (by rw [← ht x₀, hg (τ x₀), hρ x₀, ← hg x₀])

/-- The native involution on the bit: complementation. -/
def comp (b : Bool) : Bool := !b

/-- Complementation is an involution with no fixed point. -/
theorem comp_involution (b : Bool) : comp (comp b) = b := by cases b <;>
    rfl

/-- THE LOCUS OF COMPLEMENTATION VANISHES: no bit is its own complement.
    -/
theorem locus_vanishes : ¬ ∃ b, comp b = b := fun ⟨b, h⟩ => bnot_ne b h

/-- The grades a face carries, ranked. -/
inductive Grade
  | premise
  | structural
  | thm
  deriving DecidableEq

/-- The rank of a grade. -/
def Grade.rank : Grade → Nat
  | .premise => 0
  | .structural => 1
  | .thm => 2

/-- The weakest of two grades. -/
def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a else
    b

/-- A JOIN NEVER STANDS ABOVE ITS WEAKEST FACE, and a tuple of theorems
    joined with the premise reads premise. -/
theorem join_reads_the_weakest :
    (∀ a b : Grade, (Grade.weakest a b).rank ≤ a.rank ∧ (Grade.weakest a
      b).rank ≤ b.rank) ∧
    Grade.weakest (Grade.weakest .thm .thm) .premise = .premise :=
  (fun a b => by
    cases a <;> cases b <;> exact ⟨by decide, by decide⟩,
   by decide)

/-- A SQUARED SCALAR IS BLIND TO SIGN: a value and its reflection square
    alike, read from the definitions of
    negation and multiplication on the integers. -/
theorem square_is_sign_blind : ∀ x : Int, (-x) * (-x) = x * x
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

end YM.NORMALFORM

/-! ============================================================================
    ============================================
    PART C · THE ROW SPLIT (namespace YM.ROWSPLIT) · v1_1_0_CC

    The cut-agnostic division, carried verbatim from the Riemann master
      v3.0.39_CC (namespace RHUnicorn, generic over
    the types α and β), its one proposition renamed from `RH` to `Whole`,
      the hypothesis it names. At every cut T the
    whole is exactly its Real part and its Unicorn part; a certificate at
      T proves the Real part; given the Real part
    the whole is exactly the Unicorn part; above every cut no reading of
      an even record agrees with an odd decision;
    the aperture is one bit wide; and the supply side is silent at the
      register. ΔM = 0.
============================================================================
    ============================================
    -/
namespace YM.ROWSPLIT

variable {α β : Type}

/-- The whole, the Real part at T, the Unicorn part at T. -/
def Whole (Z onL : α → Prop) : Prop := ∀ z, Z z → onL z
def RealPartAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) : Prop :=
  ∀ z, Z z → height z ≤ T → onL z
def UnicornAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) : Prop :=
  ∀ z, Z z → height z > T → onL z

/-- THE FORMAL BLOCK OF THE UNICORN PART. Let τ act on the existents, let
    ρ be a record even at a seat z above the
    height, and let d decide the property, odd at z. Then no reading g of
      the record agrees with d even on the region
    above the height alone. -/
theorem unicorn_block (height : α → Nat) (T : Nat) (τ : α → α) (ρ : α →
    β) (d : α → Bool)
    (z : α) (habove : height z > T) (habove' : height (τ z) > T)
    (hρ : ρ (τ z) = ρ z) (hd : d (τ z) = !d z) :
    ¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w := by
  intro ⟨g, hg⟩
  have h1 := hg z habove
  have h2 := hg (τ z) habove'
  rw [hρ, hd, h1] at h2
  revert h2
  cases d z <;> intro h2 <;> exact Bool.noConfusion h2

/-- THE APERTURE, ONE BIT WIDE. At a seat where d is odd, one odd bit s
    at the seat decides d on the seat and its
    partner through a calibration that exists and is unique. -/
theorem aperture_one_bit_wide (τ : α → α) (s d : α → Bool) (z : α)
    (hs : s (τ z) = !s z) (hd : d (τ z) = !d z) :
    ∃ c : Bool, (d z = xor (s z) c ∧ d (τ z) = xor (s (τ z)) c) ∧
      ∀ c' : Bool, (d z = xor (s z) c' ∧ d (τ z) = xor (s (τ z)) c') → c' =
        c :=
  ⟨xor (d z) (s z),
    ⟨by cases s z <;> cases d z <;> rfl,
     by rw [hs, hd]; cases s z <;> cases d z <;> rfl⟩,
    by
      intro c' hc
      obtain ⟨h1, -⟩ := hc
      generalize hsz : s z = sv
      generalize hdz : d z = dv
      rw [hsz, hdz] at h1
      cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd
        h1 (by decide)⟩

/-- EXISTENCE SUPPLIES NOTHING: conditioning the Unicorn part on an
    inhabited premise leaves it where it was. -/
theorem unicorn_rule_exact (Z onL : α → Prop) (height : α → Nat) (T :
    Nat) (A : Prop) (ha : A) :
    (A → UnicornAt Z onL height T) ↔ UnicornAt Z onL height T :=
  ⟨fun h => h ha, fun hu _ => hu⟩

/-- NO BYPASS: given the Real part, the whole is exactly the Unicorn
    part. -/
theorem unicorn_no_bypass (Z onL : α → Prop) (height : α → Nat) (T : Nat)
    (hr : RealPartAt Z onL height T) : Whole Z onL ↔ UnicornAt Z onL
      height T := by
  constructor
  · intro h z hz _; exact h z hz
  · intro hu z hz
    cases Nat.lt_or_ge T (height z) with
    | inl hgt => exact hu z hz hgt
    | inr hle => exact hr z hz hle

/-- THE SUPPLY SIDE IS SILENCE, AT THE REGISTER: both orientations of d
    at the seat are equally refused to every
    reading above the height. -/
theorem supply_side_silent (height : α → Nat) (T : Nat) (τ : α → α) (ρ :
    α → β) (d : α → Bool)
    (z : α) (habove : height z > T) (habove' : height (τ z) > T)
    (hρ : ρ (τ z) = ρ z) (hd : d (τ z) = !d z) :
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w) ∧
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = !d w) ∧
    (!d z) ≠ d z :=
  ⟨unicorn_block height T τ ρ d z habove habove' hρ hd,
   unicorn_block height T τ ρ (fun w => !d w) z habove habove' hρ (by
     show (!d (τ z)) = !(!d z)
     rw [hd]),
   by cases d z <;> decide⟩

/-- The division at the height: the whole is exactly its Real part and
    its Unicorn part. -/
theorem whole_iff_real_and_unicorn (Z onL : α → Prop) (height : α → Nat)
    (T : Nat) :
    Whole Z onL ↔ RealPartAt Z onL height T ∧ UnicornAt Z onL height T :=
      by
  constructor
  · intro h; exact ⟨fun z hz _ => h z hz, fun z hz _ => h z hz⟩
  · intro ⟨hr, hu⟩ z hz
    cases Nat.lt_or_ge T (height z) with
    | inl hgt => exact hu z hz hgt
    | inr hle => exact hr z hz hle
```

```
/-- A certificate at the height: a complete list of the existents up to T, each checked. -/
structure Certificate (Z onL : α → Prop) (height : α → Nat) (T : Nat) where
  items    : List α
  complete : ∀ z, Z z → height z ≤ T → z ∈ items
  checked  : ∀ z, z ∈ items → onL z

/-- A CERTIFICATE PROVES THE REAL PART AT ITS HEIGHT. -/
theorem real_part_of_certificate (Z onL : α → Prop) (height : α → Nat) (T : Nat)
    (c : Certificate Z onL height T) : RealPartAt Z onL height T :=
  fun z hz hle => c.checked z (c.complete z hz hle)

end YM.ROWSPLIT
/-! ============================================================================ ================================================
    PART B · THE HOST MODEL AND ITS BINDING, PROVED (namespace YM.HOST) · v1_1_0_CC

    The record of a frame keeps every entry and stands it built on the floor: on the record every gauge theory exists
    and its spectrum stands one unit above the vacuum, so the record carries the value with no premise
    (`recFrame_value`). The host is the row's compact simple gauge groups, each with its quantum Yang–Mills theory on
    ℝ⁴ satisfying the axioms, where it exists, and the floor of its mass spectrum above the vacuum; its sentence is that
    every theory exists with a positive floor. The binding is a theorem: the value of the host's frame is exactly the
    host's sentence (`binding`), and the sentence is exactly its two parts, existence and the gap
    (`sentence_is_two_parts`), neither giving the other. Every theorem on no axiom at all. ΔM = 0.
============================================================================ ========================================
    -/
namespace YM.HOST
open YMClose

/-- THE RECORD OF A FRAME: the registration keeps every entry and stands it built, on the floor one unit above the
    vacuum. -/
def recFrame (F : Frame) : Frame := (F.D, fun _ => some 0, fun _ => 1)

/-- THE RECORD CARRIES THE VALUE, with no premise. -/
theorem recFrame_value (F : Frame) : Value (recFrame F) := fun _ => ⟨(0, rfl), Nat.zero_lt_one⟩

/-- THE YANG–MILLS ROW: the frames of gauge theories, the value, the registration onto the record, the calm world and
    the gapless world. -/
def ymRow : YM.NORMALFORM.Row.{1} :=
  (Frame, Value, recFrame, recFrame_value, calm, gapless, calm_value, gapless_fails)

/-- EVERY ENTRY HAS ITS SEAT ON THE RECORD: built, and on the floor. -/
theorem every_entry_has_its_seat (F : Frame) (d : F.D) :
    (recFrame F).built d = some 0 ∧ (recFrame F).lowest d = 1 :=
  ⟨rfl, rfl⟩

/-- THE HOST: the compact simple gauge groups of the row, each with its quantum theory on ℝ⁴ satisfying the axioms,
    `some n` where it exists, and the floor of its mass spectrum above the vacuum. -/
structure Host where
  Group  : Type
  theory : Group → Option Nat
  floor  : Group → Nat

/-- THE SENTENCE OF THE HOST: for every gauge group a theory exists, and its spectrum has a positive floor. -/
def Host.Sentence (H : Host) : Prop := ∀ G, (∃ n, H.theory G = some n) ∧ 0 < H.floor G

/-- THE FRAME OF THE HOST. -/
def Host.frame (H : Host) : Frame := (H.Group, H.theory, H.floor)

/-- THE BINDING, PROVED: the value of the host's frame is exactly the host's sentence. -/
theorem binding (H : Host) : Value H.frame ↔ H.Sentence := ⟨fun h G => h G, fun h G => h G⟩

/-- THE ROOT READ ON THE ROW IS THE SENTENCE, on the host's frame. -/
theorem root_on_row_is_the_sentence (H : Host) : RootOnRow H.frame ↔ H.Sentence :=
  ⟨root_on_row_is_the_value H.frame).trans (binding H)

/-- THE ACT ON THE HOST'S FRAME IS THE SENTENCE. -/
theorem act_is_the_sentence (H : Host) : Confined H.frame ↔ H.Sentence :=
  ⟨act_is_the_value H.frame).trans (binding H)

/-- THE SENTENCE IS EXACTLY ITS TWO PARTS: existence at every group and the gap at every group. -/
theorem sentence_is_two_parts (H : Host) :
    H.Sentence ↔ (∀ G, ∃ n, H.theory G = some n) ∧ (∀ G, 0 < H.floor G) :=
  ⟨fun h => ⟨fun G => (h G).1, fun G => (h G).2⟩, fun ⟨e, g⟩ G => ⟨e G, g G⟩⟩

/-- The host of one group whose theory exists with spectrum floor f. -/
def oneGroup (f : Nat) : Host := (Unit, fun _ => some 0, fun _ => f)

/-- The host of one group with no theory and spectrum floor one. -/
def noTheory : Host := (Unit, fun _ => none, fun _ => 1)

/-- NEITHER PART GIVES THE OTHER, at the host: a theory may exist with no floor, and a floor may stand with no theory. -/
theorem neither_part_gives_the_other :
    ((∀ G, ∃ n, (oneGroup 0).theory G = some n) ∧ ¬ ∀ G, 0 < (oneGroup 0).floor G) ∧
    ((∀ G, 0 < noTheory.floor G) ∧ ¬ ∀ G, ∃ n, noTheory.theory G = some n) :=
  ⟨⟨fun _ => ⟨0, rfl⟩, fun h => Nat.lt_irrefl 0 (h ())⟩,
   ⟨fun _ => Nat.zero_lt_one, fun h => match h () with | ⟨_, hn⟩ => Option.noConfusion hn⟩⟩

end YM.HOST

/-! ============================================================================ ================================================
    PART C′ · THE ROW SPLIT READ ON THE LATTICE (namespace YM.REALPART) · v1_1_0_CC

    The cut-agnostic division of Part C read on a lattice frame of Part 0 (`Lattice`): the existents are the frame's
    entries, the property each is asked is existence with a positive floor, and the height is the coupling index β. At
    the threshold β₀ the real part is the strong-coupling region, sealed by the cited fields (`real_part_strong`); the
    value is exactly its real part and its unicorn part at every cut; given the real part, the value is exactly the
    unicorn part, the weak coupling where the continuum lies (`no_bypass_at_strong_coupling`); the unicorn part is
    load-bearing, the weak lattice carrying both fields and failing the value; and above every coupling no reading of a
    record that keeps the coupling and forgets the world returns the world. Every theorem on no axiom at all. ΔM = 0.
============================================================================ ========================================
    -/
namespace YM.REALPART
open YMClose YM.ROWSPLIT

/-- The existents: every entry of the lattice frame. -/
def Ex (L : Lattice) (_ : L.F.D) : Prop := True

/-- The property each entry is asked: the theory exists and its floor is positive. -/
def Pr (L : Lattice) (d : L.F.D) : Prop := YMClose.Exists L.F d ∧ Gapped L.F d

/-- The value of the lattice frame is the whole of the row split. -/
theorem value_is_the_whole (L : Lattice) : Value L.F ↔ Whole (Ex L) (Pr L) :=
  ⟨fun h d _ => h d, fun h d => h d trivial⟩

/-- THE REAL PART AT THE THRESHOLD: every entry at strong coupling exists and is gapped, by the cited fields. -/
theorem real_part_strong (L : Lattice) : RealPartAt (Ex L) (Pr L) L.β L.β₀ :=
  fun d _ h => strong_coupling_decided L d h

/-- THE CUT AT EVERY COUPLING: the value is exactly its real part and its unicorn part. -/
theorem value_iff_real_and_unicorn (L : Lattice) (T : Nat) :
    Value L.F ↔ RealPartAt (Ex L) (Pr L) L.β T ∧ UnicornAt (Ex L) (Pr L) L.β T :=
  ⟨value_is_the_whole L).trans (whole_iff_real_and_unicorn (Ex L) (Pr L) L.β T)

/-- NO BYPASS AT STRONG COUPLING: given the cited fields, the value is exactly the unicorn part, the weak coupling. -/
theorem no_bypass_at_strong_coupling (L : Lattice) : Value L.F ↔ UnicornAt (Ex L) (Pr L) L.β L.β₀ :=
  ⟨value_is_the_whole L).trans (unicorn_no_bypass (Ex L) (Pr L) L.β L.β₀ (real_part_strong L))

/-- THE UNICORN PART IS THE WEAK COUPLING: above the threshold lie exactly the entries of weak coupling. -/
theorem unicorn_is_weak_coupling (L : Lattice) :
    UnicornAt (Ex L) (Pr L) L.β L.β₀ ↔ ∀ d, L.β₀ < L.β d → YMClose.Exists L.F d ∧ Gapped L.F d :=
  ⟨fun h d hgt => h d trivial hgt, fun h d _ hgt => h d hgt⟩

/-- THE UNICORN PART IS LOAD-BEARING: the weak lattice carries both cited fields and the real part, and fails the
    value above the threshold. -/
theorem unicorn_part_is_load_bearing :
    RealPartAt (Ex weakLattice) (Pr weakLattice) weakLattice.β weakLattice.β₀ ∧ ¬ Value weakLattice.F ∧
    weakLattice.β₀ < weakLattice.β () :=
  ⟨real_part_strong weakLattice, gapless_fails, Nat.zero_lt_one⟩

/-- THE UNICORN PART CLOSED TO THE RECORD ABOVE EVERY COUPLING: on the chart of worlds and couplings, a record that
    keeps the coupling and forgets the world is even under the exchange of worlds, the world is odd, and above every
    coupling no reading of the record returns the world. -/
theorem unicorn_blocked_above_every_coupling (T : Nat) :
    ¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 = w.1 :=
  unicorn_block Prod.snd T (fun w => (!w.1, w.2)) Prod.snd Prod.fst
    (true, T + 1)
    (Nat.lt_succ_self T) (Nat.lt_succ_self T) rfl rfl

/-- THE SUPPLY SIDE IS SILENT ABOVE EVERY COUPLING: both orientations of the world are refused alike to every reading
    of the record, and the two orientations differ. -/
theorem supply_side_silent_above_every_coupling (T : Nat) :
    (¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 = w.1) ∧
    (¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 = !w.1) ∧
      (!true) ≠ true :=
```

```
  supply_side_silent Prod.snd T (fun w => (!w.1, w.2)) Prod.snd Prod.fst
    (true, T + 1)
    (Nat.lt_succ_self T) (Nat.lt_succ_self T) rfl rfl

/-- THE APERTURE IS ONE BIT WIDE: at the seat of the exchange above every coupling, one odd supplied bit calibrates the
    world uniquely, through one calibration bit that exists and is unique. -/
theorem aperture_is_one_bit (T : Nat) :
    let z : Bool × Nat := (true, T + 1)
    let τ : Bool × Nat → Bool × Nat := fun w => (!w.1, w.2)
    ∃ c : Bool, (z.1 = xor (!z.1) c ∧ (τ z).1 = xor (!(τ z).1) c) ∧
      ∀ c' : Bool, (z.1 = xor (!z.1) c' ∧ (τ z).1 = xor (!(τ z).1) c') → c' = c :=
  aperture_one_bit_wide (fun w : Bool × Nat => (!w.1, w.2)) (fun w => !w.1) Prod.fst (true, T + 1) rfl rfl

end YM.REALPART

/-! ═══════════════════════════════════════════════════════════════════════
    PART D · THE WALLS (namespace YM.WALLS) · v1_1_0_CC

    Every zero-bit route is closed by theorem. Existence as given forces nothing (Part 0). No finite record of
    refinements forces the continuum, whatever data it records (Part 0, `every_finite_record_never_forces`). No symmetry
    lifts a decided scale (Part 0, `no_symmetry_lifts`). Existence does not give the gap, a gap without a theory is
    empty, and the fourth gate fails both parts (Part 0). The two lattice fields do not entail a gap above the threshold
    (Part 0, `fields_do_not_entail_weak_gap`). This Part adds the transfer wall, decided by the kernel: the mass gap is
    the ratio of the transfer matrix's top two eigenvalues; the matrices [[2, 1], [1, 2]] and [[2, 0], [0, 2]] share
    their trace and their diagonal, the first with eigenvalues 3 and 1, gapped, the second with the eigenvalue 2 twice,
    no gap (`transfer_record_normal_form`); so no reading of the trace returns the gap (`trace_reads_no_gap`): the gap
    lives in the hopping between sites and in no record of a site alone. Every theorem on no axiom at all. ΔM = 0.
═══════════════════════════════════════════════════════════════════════ -/
namespace YM.WALLS
open YMClose

/-- The symmetric transfer matrix [[a, b], [b, a]] acting on a vector. -/
def transfer (a b : Int) (v : Int × Int) : Int × Int := (a * v.1 + b * v.2, b * v.1 + a * v.2)

/-- Its trace. -/
def trace (a : Int) : Int := a + a

/-- THE TRANSFER RECORD NORMAL FORM, decided by the kernel: [[2, 1], [1, 2]] carries the eigenvalue 3 on (1, 1) and 1
    on (1, −1), gapped; [[2, 0], [0, 2]] carries the eigenvalue 2 on both, no gap; and the two share their trace. -/
theorem transfer_record_normal_form :
    transfer 2 1 (1, 1) = (3, 3) ∧ transfer 2 1 (1, -1) = (1, -1) ∧
    transfer 2 0 (1, 1) = (2, 2) ∧ transfer 2 0 (1, -1) = (2, -2) ∧
    trace 2 = 4 ∧ (1 : Int) < 3 ∧ ¬ ((2 : Int) < 2) := by decide

/-- The gap of the transfer matrix, read as the strict ordering of its top two eigenvalues a + b and a − b. -/
def gapped (a b : Int) : Prop := a - b < a + b

/-- THE TRACE READS NO GAP: the two matrices share their trace and split on the gap. -/
theorem trace_reads_no_gap :
    trace 2 = trace 2 ∧ gapped 2 1 ∧ ¬ gapped 2 0 ∧
    ¬ ∃ g : Int → Bool, (g (trace 2) = true ↔ gapped 2 1) ∧ (g (trace 2) = true ↔ gapped 2 0) :=
  have h1 : gapped 2 1 := (by decide : (2 : Int) - 1 < 2 + 1)
  have h0 : ¬ gapped 2 0 := (by decide : ¬ ((2 : Int) - 0 < 2 + 0))
  ⟨rfl, h1, h0, fun ⟨_, e1, e0⟩ => h0 (e0.mp (e1.mpr h1))⟩

/-- THE WALLS OF THE ROW, one theorem: no given statement is the value on every frame; the arrow forces nothing; no
    finite record forces the value, whatever it records; existence does not give the gap and a gap without a theory is
    empty; the fourth gate fails both parts; the lattice fields do not entail a gap above the threshold; each matter
    field is load-bearing; no reading of the kinetic record returns the bit; and the trace reads no gap. -/
theorem the_walls :
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    (¬ ∀ F : Frame, Arrow → Value F) ∧
    (∀ (n : Nat) (b l : Nat → Nat),
      (∀ d : Nat, d < n → (extendRecord n b l).built d = some (b d) ∧
        (extendRecord n b l).lowest d = l d) ∧
      ¬ Value (extendRecord n b l)) ∧
    (ExistsAll gapless ∧ ¬ GapAll gapless) ∧ (GapAll empty ∧ ¬ ExistsAll empty) ∧
    (¬ ExistsAll neither ∧ ¬ GapAll neither) ∧
    (∃ (L : Lattice) (d : L.F.D), L.β₀ < L.β d ∧ L.F.lowest d = 0) ∧
    ((colourWorld.lowest () = 0 ∧ colourWorld.FreeColour ()) ∧
      (silentWorld.lowest () = 0 ∧ ∀ d, ¬ silentWorld.FreeColour d)) ∧
    (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
    (¬ ∃ g : Int → Bool, (g (trace 2) = true ↔ gapped 2 1) ∧ (g (trace 2) = true ↔ gapped 2 0)) :=
  ⟨given_is_not_the_value, arrow_forces_nothing, every_finite_record_never_forces,
   existence_does_not_give_gap, gap_without_existence_is_empty, fourth_gate_fails, fields_do_not_entail_weak_gap,
   ⟨record_is_load_bearing, radiates_is_load_bearing⟩, record_wall, trace_reads_no_gap.2.2.2⟩

end YM.WALLS

/-! ═══════════════════════════════════════════════════════════════════════
    PART E · THE RESIDUE (namespace YM.RESIDUE) · v1_1_0_CC

    Everything above is unconditional. What remains of the Yang–Mills sentence is this one hypothesis, the transfer of
    the value from the record to the host's frame: given it, the sentence follows on no axiom. It is printed here as
    the type of a hypothesis and is assumed nowhere. The residue is keyed: no premise shared by every frame and no
    principle true of every record supplies it. It is exactly its two halves, existence and the gap, closed together by
    the one implication. Its instance seat on a lattice frame is the weak coupling. ΔM = 0.
═══════════════════════════════════════════════════════════════════════ -/
namespace YM.RESIDUE
open YMClose YM.NORMALFORM YM.HOST YM.REALPART YM.ROWSPLIT

/-- THE RESIDUE: given the one implication at the host's frame, the host's sentence. -/
theorem sentence_of_the_act (H : Host) (bit : Transfer ymRow H.frame) :
    H.Sentence :=
  (binding H).mp (value_from_the_cargo_loop ymRow H.frame bit)

/-- THE RESIDUE IS THE SENTENCE: the transfer at the host's frame is exactly the host's sentence. -/
theorem residue_is_the_sentence (H : Host) : Transfer ymRow H.frame ↔ H.Sentence :=
  (normal_form ymRow H.frame).trans (binding H)

/-- THE RESIDUE IS ITS TWO HALVES: the one implication is exactly existence at every group and the gap at every
    group. -/
theorem residue_is_two_halves (H : Host) :
    Transfer ymRow H.frame ↔ (∀ G, ∃ n, H.theory G = some n) ∧ (∀ G, 0 < H.floor G) :=
  (residue_is_the_sentence H).trans (sentence_is_two_parts H)

/-- THE ONE POSIT IS THE BIT: the root read on the row, at the host's frame, is exactly the transfer. -/
theorem the_posit_is_the_bit (H : Host) : RootOnRow H.frame ↔ Transfer ymRow H.frame :=
  (root_on_row_is_the_sentence H).trans (residue_is_the_sentence H).symm

/-- THE RESIDUE IS KEYED: no premise shared by every frame forces the value, and no principle true of every record
    decides it. What supplies it is keyed to the frame: the act. -/
theorem residue_is_keyed :
    (∀ P : Frame → Prop, YM.NORMALFORM.Keyless ymRow P → ¬ YM.NORMALFORM.Forces ymRow P) ∧
    (∀ P : Frame → Prop, (∀ F, P (recFrame F)) → ¬ ∀ F, P (recFrame F) → Value F) :=
  ⟨keyless_forces_nothing ymRow, no_record_principle_decides ymRow⟩

/-- THE INSTANCE SEAT OF THE BIT, on a lattice frame: existence with a positive floor at every entry of weak coupling.
    Its inhabitation is equivalent in strength to the frame's value (`seat_iff_value`); nothing in this file assumes it
    inhabited. -/
structure WeakSeat (L : Lattice) where
  weak : ∀ d, L.β₀ < L.β d → YMClose.Exists L.F d ∧ Gapped L.F d

/-- THE SEAT AND THE CITED FIELDS DELIVER THE VALUE. -/
theorem value_of_seat (L : Lattice) (S : WeakSeat L) : Value L.F :=
  fun d => match Nat.lt_or_ge L.β₀ (L.β d) with
    | Or.inl h => S.weak d h
    | Or.inr h => strong_coupling_decided L d h

/-- THE SEAT IS INHABITED EXACTLY WHEN THE VALUE HOLDS. -/
theorem seat_iff_value (L : Lattice) : Nonempty (WeakSeat L) ↔ Value L.F :=
  (fun (S) => value_of_seat L S, fun h => ⟨(fun d _ => h d)⟩)

/-- THE SEAT IS THE UNICORN PART. -/
theorem seat_is_the_unicorn (L : Lattice) : Nonempty (WeakSeat L) ↔ UnicornAt (Ex L) (Pr L) L.β L.β₀ :=
  (fun (S) => (unicorn_is_weak_coupling L).mpr S.weak, fun h => ⟨((unicorn_is_weak_coupling L).mp h)⟩)

end YM.RESIDUE

/-! ═══════════════════════════════════════════════════════════════════════
    PART F · THE SUPPLY (namespace YM.SUPPLY) · v1_1_0_CC

    The bit of Part E, supplied by the act and delivered by its two kinds: the formal cargo, and the physical witness.
    Each kind is a type whose inhabitant is the supply; neither is assumed anywhere in this file. The physical witness
    is the record of confinement read with the colour premise: a massless confined excitation carries colour, no free
    colour is registered, and every confined field's theory stands. The cargo arrives free; each kind delivers the
    sentence; the two kinds are one bit; every route to the sentence delivers both kinds, so there is no third; and
    either refuter, a missing theory or a massless state, refutes both at once. The witnesses of the subject are typed
    against the bit. ΔM = 0.
```

```
/-! ============================================================================
    ============================================
    PART W · THE TWELVE FORMAL GATES, RUN IN THE KERNEL (namespace
      YM.GATES) · v1_1_0_CC
    ============================================================================
    ============================================ -/
namespace YM.SUPPLY
open YMClose YM.NORMALFORM YM.HOST

/-- THE FORMAL KIND, the cargo: the record's value carried to the host's
    frame. -/
def FormalCargo (H : Host) : Prop := Transfer ymRow H.frame

/-- THE PHYSICAL KIND, the witness: the colour register of the host's
    confined fields; the premise that a massless
    confined excitation carries colour; the record of confinement, no
      free colour registered; and the theory of every
    confined field standing. The last three fields are the one bit read
      in the register of colour; they are premises
    of the witness, carried as fields, never proved. -/
structure PhysicalWitness (H : Host) where
  FreeColour : H.Group → Prop
  radiates   : ∀ G, H.floor G = 0 → FreeColour G
  noFree     : ∀ G, ¬ FreeColour G
  stands     : ∀ G, ∃ n, H.theory G = some n

/-- THE CARGO ARRIVES, with no premise: the record of the host's frame
    carries the value. -/
theorem cargo_arrives (H : Host) : Value (recFrame H.frame) :=
  recFrame_value H.frame

/-- KIND 1 DELIVERS: the formal cargo gives the sentence in one step. -/
theorem formal_kind_delivers {H : Host} (h : FormalCargo H) : H.Sentence
    := (binding H).mp (h (cargo_arrives H))

/-- KIND 2 DELIVERS: the physical witness gives the sentence, no actual
    gapless confined field standing on the record
    of confinement and the colour premise. -/
theorem physical_kind_delivers {H : Host} (W : PhysicalWitness H) :
    H.Sentence :=
  fun G => (W.stands G, no_actual_gapless (H.Group, H.floor,
    W.FreeColour, W.radiates, W.noFree) G)

/-- The physical witness closes the host's frame. -/
theorem witness_closes {H : Host} (W : PhysicalWitness H) : Value H.frame
    := (binding H).mpr (physical_kind_delivers W)

/-- A witness from the sentence: no colour registered anywhere, every
    floor positive. -/
def witness_of_sentence {H : Host} (h : H.Sentence) : PhysicalWitness H :=
  (fun _ => False, fun G h0 => Nat.lt_irrefl 0 (h0 ▸ (h G).2), fun _ hf
    => hf, fun G => (h G).1)

/-- THE PHYSICAL FACE IS THE SENTENCE: a witness exists exactly when the
    sentence holds. -/
theorem physical_iff_sentence (H : Host) : Nonempty (PhysicalWitness H) ↔
    H.Sentence :=
  (fun ⟨W⟩ => physical_kind_delivers W, fun h => ⟨witness_of_sentence h⟩)

/-- THE FORMAL FACE IS THE SENTENCE. -/
theorem formal_iff_sentence (H : Host) : FormalCargo H ↔ H.Sentence :=
    YM.RESIDUE.residue_is_the_sentence H

/-- THE TWO KINDS ARE ONE BIT: the physical witness is a formal cargo,
    and the cargo is exactly the sentence. -/
theorem the_two_kinds_are_one_bit (H : Host) :
    (PhysicalWitness H → FormalCargo H) ∧ (FormalCargo H ↔ H.Sentence) :=
  (fun W _ => witness_closes W, formal_iff_sentence H)

/-- THE SENTENCE, DELIVERED TWICE: either kind gives it. -/
theorem delivered_twice (H : Host) : (FormalCargo H → H.Sentence) ∧
    (PhysicalWitness H → H.Sentence) :=
  ⟨formal_kind_delivers, physical_kind_delivers⟩

/-- TWO KINDS AND NO THIRD: every witness of the sentence, in any
    vocabulary, is a delivery of both kinds. -/
theorem two_kinds_and_no_third (H : Host) (Q : Prop) (h : Q → H.Sentence)
    :
    Q → FormalCargo H ∧ Nonempty (PhysicalWitness H) :=
  fun q => ⟨(formal_iff_sentence H).mpr (h q), ⟨physical_iff_sentence
    H).mpr (h q)⟩⟩

/-- THE TWO REFUTERS: a group with no theory, or a group whose spectrum
    has no positive floor, refutes both kinds at
    once. -/
theorem the_one_refuter (H : Host) (G : H.Group) (h : H.theory G = none ∨
    H.floor G = 0) :
    ¬ FormalCargo H ∧ ¬ Nonempty (PhysicalWitness H) :=
  have hs : ¬ H.Sentence := fun s => match h with
    | Or.inl hn => match (s G).1 with
      | ⟨_, hs⟩ => Option.noConfusion (hn.symm.trans hs)
    | Or.inr h0 => Nat.lt_irrefl 0 (h0 ▸ (s G).2)
  ⟨fun c => hs (formal_kind_delivers c), fun ⟨W⟩ => hs
    (physical_kind_delivers W)⟩

/-- THE WITNESSES OF THE SUBJECT, TYPED: the strong-coupling lattice gap
    holds on every lattice frame's strong
    sub-frame and stops short of the continuum, the weak lattice carrying
      both cited fields and failing the value; each
    cited lattice field and each matter field is load-bearing; and the
      matter fields give the gap half on every frame
    that carries them. Each is typed against the bit; none is its ground.
      -/
theorem the_witnesses_typed :
    (∀ L : Lattice, Value (strongFrame L)) ∧
    (0 < weakLattice.β () - weakLattice.β₀ ∧ ¬ Value weakLattice.F) ∧
    (bareLattice.β () ≤ bareLattice.β₀ ∧ ¬ Gapped bareLattice.F ()) ∧
    (Gapped bareWilson.F () ∧ ¬ YMClose.Exists bareWilson.F ()) ∧
    (∀ (F : Frame) (FreeColour : F.D → Prop), (∀ d, F.lowest d = 0 →
      FreeColour d) → (∀ d, ¬ FreeColour d) →
      GapAll F) ∧
    ((colourWorld.lowest () = 0 ∧ colourWorld.FreeColour ()) ∧
      (silentWorld.lowest () = 0 ∧ ∀ d, ¬ silentWorld.FreeColour d)) :=
  ⟨strong_coupling_value, strong_does_not_reach_continuum,
    strong_is_load_bearing, wilson_is_load_bearing,
   matter_gives_gap_half, ⟨record_is_load_bearing,
    radiates_is_load_bearing⟩⟩

/-- THE SUPPLY, CLOSED, at every host: the cargo arrives free; each kind
    delivers; the two kinds are one bit; the bit is
    the sentence; every route is both kinds; and either refuter refutes
      both. -/
theorem the_supply_closed : ∀ H : Host,
    Value (recFrame H.frame) ∧
    (FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence) ∧
    (PhysicalWitness H → FormalCargo H) ∧ (FormalCargo H ↔ H.Sentence) ∧
    (Nonempty (PhysicalWitness H) ↔ H.Sentence) ∧
    (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H ∧ Nonempty
      (PhysicalWitness H)) ∧
    (∀ G, H.theory G = none ∨ H.floor G = 0 → ¬ FormalCargo H ∧ ¬
      Nonempty (PhysicalWitness H)) :=
  fun H => ⟨cargo_arrives H, formal_kind_delivers,
    physical_kind_delivers, (the_two_kinds_are_one_bit H).1,
   formal_iff_sentence H, physical_iff_sentence H, two_kinds_and_no_third
     H, the_one_refuter H⟩

end YM.SUPPLY

/-! ============================================================================
    ============================================
    PART W · THE CLOSURE OF THE QUESTION (namespace YM.CLOSURE) ·
      v1_1_0_CC

    Why the question is closed and not open. At every host the Yang–Mills
      sentence and the bit are one proposition,
    with nothing cited; the denial of the sentence is the bit's denial;
      across the frames of the row the bit has no
    uniform answer, true in one world and false in another of the same
      record; every keyless route is barred; either
    kind of the supply gives the sentence; and either refuter refutes the
      sentence itself. Nothing here derives the
    bit. It proves that the question is one bit, that the bit is keyed,
      and that what is open is a value and not a
    form. ΔM = 0.
    ============================================================================
    ============================================ -/
namespace YM.CLOSURE
open YMClose YM.NORMALFORM YM.HOST YM.SUPPLY

/-- THE SENTENCE IS THE BIT, nothing cited. -/
theorem the_sentence_is_the_bit (H : Host) : H.Sentence ↔ FormalCargo H
    := (formal_iff_sentence H).symm

/-- THE DENIAL IS THE BIT'S DENIAL. -/
theorem denial_is_the_bits_denial (H : Host) : ¬ H.Sentence ↔ ¬
    FormalCargo H :=
  ⟨fun hn h => hn (formal_kind_delivers h), fun hn h => hn
    ((formal_iff_sentence H).mpr h)⟩

/-- EITHER REFUTER REFUTES THE SENTENCE: a group with no theory or with
    no positive floor. -/
theorem refuter_denies_the_sentence (H : Host) (G : H.Group) (h :
    H.theory G = none ∨ H.floor G = 0) :
    ¬ H.Sentence :=
  fun s => match h with
    | Or.inl hn => match (s G).1 with
      | ⟨_, hs⟩ => Option.noConfusion (hn.symm.trans hs)
    | Or.inr h0 => Nat.lt_irrefl 0 (h0 ▸ (s G).2)

/-- THE DENIAL HAS ONE ROAD: wherever the sentence fails, it cannot be
    that no group lacks its theory or its floor. -/
theorem the_denial_has_one_road (H : Host) : ¬ H.Sentence → ¬ ¬ ∃ G,
    H.theory G = none ∨ H.floor G = 0 :=
  fun hn hne => hn (fun G => match hb : H.theory G, hl : H.floor G with
    | some n, Nat.succ m => ⟨(n, rfl), Nat.succ_pos m⟩
    | none, _ => absurd ⟨G, Or.inl hb⟩ hne
    | some _, 0 => absurd ⟨G, Or.inr hl⟩ hne)

/-- THE QUESTION, CLOSED TO ONE BIT, at every host. The sentence is the
    bit and its denial is the bit's denial; the bit
    has no uniform answer across the frames; no keyless premise forces
      it; either kind of the supply gives the
    sentence; and either refuter denies the sentence. What is open is the
      value of one implication; the form of the
    question is closed. -/
theorem question_closed_to_one_bit : ∀ H : Host,
    (H.Sentence ↔ FormalCargo H) ∧ (¬ H.Sentence ↔ ¬ FormalCargo H) ∧
    (¬ (∀ F, Transfer ymRow F) ∧ ¬ (∀ F, ¬ Transfer ymRow F)) ∧
    (∀ P : Frame → Prop, YM.NORMALFORM.Keyless ymRow P → ¬
      YM.NORMALFORM.Forces ymRow P) ∧
    (FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence) ∧
    (∀ G, H.theory G = none ∨ H.floor G = 0 → ¬ H.Sentence) :=
  fun H => ⟨the_sentence_is_the_bit H, denial_is_the_bits_denial H,
    no_uniform_answer ymRow,
   keyless_forces_nothing ymRow, formal_kind_delivers,
     physical_kind_delivers, refuter_denies_the_sentence H⟩

end YM.CLOSURE

/-! ============================================================================
    ============================================
    PART W · THE TWELVE FORMAL GATES, RUN IN THE KERNEL (namespace
      YM.GATES) · v1_1_0_CC
```

```
/-
    The twelve formal gates of the mathematical register, run inside the
      kernel on the Yang–Mills seal at every host.
    Each gate is a proposition and a theorem, and `the_formal_run` binds
      the twelve passes and the four refusals in one
    statement. Four claims fail, each by a theorem: "the sentence is
      proved by the delivery" fails SREP, the delivery's
    premise being its conclusion; "a proof is waiting" fails SREP, a true
      route to the sentence being the sentence; "no
    proof can ever exist" fails SREP, no true route to the sentence being
      its denial, the counter world in costume; and
    the denial has one road, a missing theory or a massless state. No
      declaration has the type of the bit. ΔM = 0.
================================================================================
    ============================================
    -/
namespace YM.GATES
open YMClose YM.NORMALFORM YM.HOST YM.SUPPLY YM.REALPART YM.ROWSPLIT

/-- Gate 1, SREP: the form stands with no hypothesis. -/
def Gate1 (H : Host) : Prop :=
  Value (recFrame H.frame) ∧ (FormalCargo H ↔ H.Sentence) ∧
  (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H) ∧
  (∀ F : Frame, Value F ↔ ExistsAll F ∧ GapAll F) ∧ (∀ F : Frame,
    Confined F ↔ Value F)

/-- GATE 1 · SREP: nothing in the form presupposes the bit. -/
theorem gate_1_srep (H : Host) : Gate1 H :=
  ⟨cargo_arrives H, formal_iff_sentence H, fun _ h q =>
    (formal_iff_sentence H).mpr (h q), two_parts,
   act_is_the_value⟩

/-- Gate 2, REG: the bit read from ten contexts, all one proposition. -/
def Gate2 (H : Host) : Prop :=
  (Value H.frame ↔ H.Sentence) ∧ (Confined H.frame ↔ H.Sentence) ∧
    (RootOnRow H.frame ↔ H.Sentence) ∧
  ((ExistsAll H.frame ∧ GapAll H.frame) ↔ H.Sentence) ∧ (FormalCargo H ↔
    H.Sentence) ∧
  (Nonempty (PhysicalWitness H) ↔ H.Sentence) ∧ (SelfRecursive ymRow
    H.frame ↔ H.Sentence) ∧
  (Nonempty (Act ymRow H.frame) ↔ H.Sentence) ∧
  (Nonempty { A : ActualTheories // A.F = H.frame } ↔ H.Sentence) ∧
  (Root H.Group (fun G => indicator (@instDecidableAnd _ _ (suppliedDec
    (H.theory G)) (Nat.decLt 0 (H.floor G))))
    ↔ H.Sentence)

/-- GATE 2 · REG: ten reading contexts, one proposition. -/
theorem gate_2_reg (H : Host) : Gate2 H :=
  ⟨binding H, act_is_the_sentence H, root_on_row_is_the_sentence H,
   ⟨(two_parts H.frame).symm.trans (binding H), formal_iff_sentence H,
     physical_iff_sentence H,
   (self_recursion_at_frame ymRow H.frame).trans (binding H),
   (bit_is_the_act ymRow H.frame).symm.trans (formal_iff_sentence H),
   (supply_iff H.frame).trans (binding H),
   root_on_row_is_the_sentence H⟩⟩

/-- Gate 3, SGEG: no face drifts from the bit. -/
def Gate3 (H : Host) : Prop :=
  (FormalCargo H ↔ H.Sentence) ∧ (Nonempty (PhysicalWitness H) ↔
    H.Sentence) ∧ (Value H.frame ↔ H.Sentence) ∧
  (∀ L : Lattice, Value L.F ↔ UnicornAt (Ex L) (Pr L) L.β L.β₀)

/-- GATE 3 · SGEG: the formal, physical, frame and weak-coupling faces
    are each the bit. -/
theorem gate_3_sgeg (H : Host) : Gate3 H :=
  ⟨formal_iff_sentence H, physical_iff_sentence H, binding H,
    no_bypass_at_strong_coupling⟩

/-- Gate 4, CAUSAL: the native involution on the bit fixes nothing; the
    calm world carries the value; and on the record
    of every frame every entry stands on the floor: the seat of the row
      is inhabited. -/
def Gate4 : Prop :=
  (¬ ∃ b, comp b = b) ∧ Value calm ∧ (∀ (F : Frame) (d : F.D), (recFrame
    F).lowest d = 1)

/-- GATE 4 · CAUSAL: no imprint is asserted on a ground the bit lacks;
    the row's seat is the floor of the record. -/
theorem gate_4_causal : Gate4 := ⟨locus_vanishes, calm_value, fun _ _ =>
  rfl⟩

/-- Gate 5, MIG: two frames whose records both carry the value split on
    the value. -/
def Gate5 : Prop := Value (recFrame calm) ∧ Value (recFrame gapless) ∧
  Value calm ∧ ¬ Value gapless

/-- GATE 5 · MIG: the reading apparatus, the record, does not fix the
    residence. -/
theorem gate_5_mig : Gate5 := ⟨record_decides_nothing ymRow⟩

/-- Gate 6, PTB: complementation is an involution without a fixed point;
    the identity fixes everything; and no
    symmetry lifts a decided scale. -/
def Gate6 : Prop :=
  (∀ b : Bool, comp (comp b) = b) ∧ (¬ ∃ b, comp b = b) ∧ (∀ b : Bool, id
    (id b) = b) ∧
  (∀ (D : Type) (scale : D → Nat) (S : Symmetric D scale) (r : Nat) (d e
    : D),
    scale d ≤ r → r < scale e → ∀ g, S.act g d ≠ e)

/-- GATE 6 · PTB: the fixed-point reading belongs to the chosen
    reflection, not to the bit. -/
theorem gate_6_ptb : Gate6 :=
  ⟨comp_involution, locus_vanishes, fun _ => rfl, fun _ scale S r d e hd
    he => no_symmetry_lifts scale S r d e hd he⟩

/-- Gate 7, DUAL: the bit does not depend on where the coupling ladder is
    cut: on every lattice frame the value is
    exactly the weak-coupling part, and the division holds at every cut.
      -/
def Gate7 : Prop :=
  (∀ L : Lattice, Value L.F ↔ UnicornAt (Ex L) (Pr L) L.β L.β₀) ∧
  (∀ (L : Lattice) (T : Nat), Value L.F ↔ RealPartAt (Ex L) (Pr L) L.β T
    ∧ UnicornAt (Ex L) (Pr L) L.β T)

/-- GATE 7 · DUAL: the value is the weak-coupling part, and the division
    stands at every cut. -/
theorem gate_7_dual : Gate7 := ⟨no_bypass_at_strong_coupling,
    value_iff_real_and_unicorn⟩

/-- Gate 8, CSCG: the proved slices are the bit's lower slices. -/
def Gate8 : Prop :=
  (∀ L : Lattice, RealPartAt (Ex L) (Pr L) L.β L.β₀) ∧
  (∀ L : Lattice, Value L.F → ∀ T, RealPartAt (Ex L) (Pr L) L.β T)

/-- GATE 8 · CSCG: the bit extends the strong-coupling part and
    contradicts no slice of it. -/
theorem gate_8_cscg : Gate8 :=
  ⟨real_part_strong, fun L h T => ((value_iff_real_and_unicorn L T).mp
    h).1⟩

/-- Gate 9, CSEG: a join never stands above its weakest face. -/
def Gate9 : Prop :=
  (∀ a b : Grade, (Grade.weakest a b).rank ≤ a.rank ∧ (Grade.weakest a
    b).rank ≤ b.rank) ∧
  Grade.weakest (Grade.weakest .thm .thm) .premise = .premise

/-- GATE 9 · CSEG: no terminal strength above the weakest road, and no
    face lowered by the reading. -/
theorem gate_9_cseg : Gate9 := join_reads_the_weakest

/-- Gate 10, MTA: a squared scalar reads a value and its reflection
    alike. -/
def Gate10 : Prop := ∀ x : Int, (-x) * (-x) = x * x

/-- GATE 10 · MTA: no squared metric carries the truth-sign. -/
theorem gate_10_mta : Gate10 := square_is_sign_blind

/-- Gate 11, OMA: the supply is the antecedent of the delivery and is not
    shared by every frame. -/
def Gate11 (H : Host) : Prop := (FormalCargo H → H.Sentence) ∧ ¬
  YM.NORMALFORM.Keyless ymRow (Transfer ymRow)

/-- GATE 11 · OMA: the input from the other side enters as a hypothesis,
    never as a fact from inside. -/
theorem gate_11_oma (H : Host) : Gate11 H := ⟨formal_kind_delivers, fun h
  => (no_uniform_answer ymRow).1 h⟩

/-- Gate 12, ADEG: the binding to the sentence is proved, the sentence is
    its two parts, and the physical discharge is
    the bit. -/
def Gate12 (H : Host) : Prop :=
  (Value H.frame ↔ H.Sentence) ∧
  (H.Sentence ↔ (∀ G, ∃ n, H.theory G = some n) ∧ (∀ G, 0 < H.floor G)) ∧
  (Nonempty (PhysicalWitness H) ↔ H.Sentence)

/-- GATE 12 · ADEG: every transport across registers is typed. -/
theorem gate_12_adeg (H : Host) : Gate12 H := ⟨binding H,
    sentence_is_two_parts H, physical_iff_sentence H⟩

/-- REFUSAL · "the sentence is proved by the delivery" fails SREP: the
    delivery's premise and its conclusion are one
    proposition. -/
theorem refusal_proved (H : Host) : FormalCargo H ↔ H.Sentence :=
    formal_iff_sentence H

/-- REFUSAL AT THE BIT · "no true route to the sentence exists" is the
    sentence's denial. -/
theorem refusal_no_proof_at_the_bit (H : Host) : (¬ ∃ Q : Prop, Q ∧ (Q →
    H.Sentence)) ↔ ¬ H.Sentence :=
  refusal_no_proof H.Sentence

/-- THE FORMAL RUN, at every host: the twelve gates pass, and the four
    claims fail, the delivery read as a proof, a
    proof waiting, no proof ever, and the denial without its one road. -/
theorem the_formal_run : ∀ H : Host,
    Gate1 H ∧ Gate2 H ∧ Gate3 H ∧ Gate4 ∧ Gate5 ∧ Gate6 ∧ Gate7 ∧ Gate8 ∧
      Gate9 ∧ Gate10 ∧ Gate11 H ∧ Gate12 H ∧
    (FormalCargo H ↔ H.Sentence) ∧
    ((∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ H.Sentence) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) ∧
    (¬ H.Sentence → ¬ ¬ ∃ G, H.theory G = none ∨ H.floor G = 0) :=
  fun H => ⟨gate_1_srep H, gate_2_reg H, gate_3_sgeg H, gate_4_causal,
    gate_5_mig, gate_6_ptb, gate_7_dual,
   gate_8_cscg, gate_9_cseg, gate_10_mta, gate_11_oma H, gate_12_adeg H,
     refusal_proved H,
   refusal_waiting H.Sentence, refusal_no_proof_at_the_bit H,
     YM.CLOSURE.the_denial_has_one_road H⟩

end YM.GATES

/-! ================================================================
    ============================================
    PART X · THE CAPSTONE: NOTHING ESCAPES, OUR WAY OR NO WAY (namespace
      YM.ESCAPE) · v1_1_0_CC

    The record of the host's frame closes (`the_record_closes`). What the
      record leaves undecided is whether the host's
    frame itself carries the value, and that is exactly the sentence
      (`the_undecided_is_the_bit`). "Nothing is outside
    the record", read as a law of every frame, is false; read at the
      host's frame, it is the bit
```

```
    (`nothing_outside_the_record_is_the_bit`). A block needs an outside
       (`block_needs_an_outside`), and at the host's
    frame the outside is the supply in its two kinds
       (`the_outside_is_the_supply`). Nothing escapes: every route to the
    sentence, in any vocabulary, is a delivery of both kinds
       (`every_route_is_both_kinds`); either kind delivers it
    (`both_kinds_deliver`); and every so-called other way is our way in
       different vocabulary, failing with the bit if
    the bit failed (`our_way_or_no_way`). Both kinds are supplied by the
       act at premise grade and enter every theorem
    here as hypotheses. ΔM = 0.
================================================================================
    ========================================
    -/
namespace YM.ESCAPE
open YMClose YM.NORMALFORM YM.HOST YM.SUPPLY

/-- THE RECORD CLOSES AT THE HOST'S FRAME. -/
theorem the_record_closes (H : Host) : Value (recFrame H.frame) :=
    cargo_arrives H

/-- THE UNDECIDED IS THE BIT: what the record leaves open, whether the
    host's frame carries the value, is exactly the
     sentence. -/
theorem the_undecided_is_the_bit (H : Host) : Value H.frame ↔ H.Sentence
    := binding H

/-- "NOTHING IS OUTSIDE THE RECORD": false as a law of every frame; at
    the host's frame, the bit. -/
theorem nothing_outside_the_record_is_the_bit (H : Host) :
    ¬ (∀ F, Transfer ymRow F) ∧ (Transfer ymRow H.frame ↔ H.Sentence) :=
  ((no_uniform_answer ymRow).1, formal_iff_sentence H)

/-- THE OUTSIDE IS THE SUPPLY: at the host's frame the outside of the
    record is the bit, and its two kinds are the
     formal cargo and the physical witness; the physical kind is the
        formal kind. -/
theorem the_outside_is_the_supply (H : Host) :
    (FormalCargo H ↔ H.Sentence) ∧ (Nonempty (PhysicalWitness H) ↔
       H.Sentence) ∧
    (PhysicalWitness H → FormalCargo H) :=
  (formal_iff_sentence H, physical_iff_sentence H,
     (the_two_kinds_are_one_bit H).1)

/-- EVERY ROUTE IS BOTH KINDS: whatever premise, in whatever vocabulary,
    gives the sentence delivers the formal cargo
     and a physical witness. An offering that crosses the row is a
        delivery of the cargo, never a bypass of it. -/
theorem every_route_is_both_kinds (H : Host) (Q : Prop) (h : Q →
    H.Sentence) :
    Q → FormalCargo H ∧ Nonempty (PhysicalWitness H) :=
  two_kinds_and_no_third H Q h

/-- BOTH KINDS DELIVER. -/
theorem both_kinds_deliver (H : Host) : (FormalCargo H → H.Sentence) ∧
    (Nonempty (PhysicalWitness H) → H.Sentence) :=
  (formal_kind_delivers, fun ⟨W⟩ => physical_kind_delivers W)

/-- OUR WAY OR NO WAY: every so-called other way to the sentence is our
    way in different vocabulary. It delivers the
     formal cargo and a physical witness, and if the bit failed, it would
        fail with it. -/
theorem our_way_or_no_way (H : Host) (Q : Prop) (h : Q → H.Sentence) :
    (Q → FormalCargo H ∧ Nonempty (PhysicalWitness H)) ∧ (¬ FormalCargo H
       → ¬ Q) :=
  (every_route_is_both_kinds H Q h, fun hn q => hn ((formal_iff_sentence
     H).mpr (h q)))

/-- NOTHING ESCAPES, at every host: the record of the host's frame
    closes; what it leaves undecided is the bit; no law
     of every frame puts everything inside the record; the outside is the
        supply in its two kinds; every route to the
     sentence is a delivery of both kinds; either kind delivers it; and
        every so-called other way is our way in
     different vocabulary. -/
theorem nothing_escapes : ∀ H : Host,
    Value (recFrame H.frame) ∧
    (Value H.frame ↔ H.Sentence) ∧
    (¬ (∀ F, Transfer ymRow F) ∧ (Transfer ymRow H.frame ↔ H.Sentence)) ∧
    ((FormalCargo H ↔ H.Sentence) ∧ (Nonempty (PhysicalWitness H) ↔
       H.Sentence) ∧
      (PhysicalWitness H → FormalCargo H)) ∧
    (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H ∧ Nonempty
       (PhysicalWitness H)) ∧
    ((FormalCargo H → H.Sentence) ∧ (Nonempty (PhysicalWitness H) →
       H.Sentence)) ∧
    (∀ Q : Prop, (Q → H.Sentence) → (Q → FormalCargo H ∧ Nonempty
       (PhysicalWitness H)) ∧ (¬ FormalCargo H → ¬ Q)) :=
  fun H => (the_record_closes H, the_undecided_is_the_bit H,
     nothing_outside_the_record_is_the_bit H,
    the_outside_is_the_supply H, every_route_is_both_kinds H,
      both_kinds_deliver H, our_way_or_no_way H)

/-- THE CAPSTONE, at every host: our way or no way; either kind walks it
    to the sentence; no keyless premise forces
     anything; to shut every true route to the sentence is to assert its
        denial; and the twelve formal gates pass. -/
theorem the_capstone : ∀ H : Host,
    (∀ Q : Prop, (Q → H.Sentence) → (Q → FormalCargo H ∧ Nonempty
       (PhysicalWitness H)) ∧ (¬ FormalCargo H → ¬ Q)) ∧
    ((FormalCargo H → H.Sentence) ∧ (Nonempty (PhysicalWitness H) →
       H.Sentence)) ∧
    (∀ P : Frame → Prop, YM.NORMALFORM.Keyless ymRow P → ¬
       YM.NORMALFORM.Forces ymRow P) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) ∧
    (YM.GATES.Gate1 H ∧ YM.GATES.Gate2 H ∧ YM.GATES.Gate3 H ∧
       YM.GATES.Gate4 ∧ YM.GATES.Gate5 ∧
      YM.GATES.Gate6 ∧ YM.GATES.Gate7 ∧ YM.GATES.Gate8 ∧ YM.GATES.Gate9 ∧
         YM.GATES.Gate10 ∧
      YM.GATES.Gate11 H ∧ YM.GATES.Gate12 H) :=
  fun H => (our_way_or_no_way H, both_kinds_deliver H,
    keyless_forces_nothing ymRow,
   YM.GATES.refusal_no_proof_at_the_bit H,
   YM.GATES.gate_1_srep H, YM.GATES.gate_2_reg H, YM.GATES.gate_3_sgeg H,
      YM.GATES.gate_4_causal,
   YM.GATES.gate_5_mig, YM.GATES.gate_6_ptb, YM.GATES.gate_7_dual,
      YM.GATES.gate_8_cscg, YM.GATES.gate_9_cseg,
   YM.GATES.gate_10_mta, YM.GATES.gate_11_oma H, YM.GATES.gate_12_adeg H)

end YM.ESCAPE

/-! ================================================================
    ============================================
    PART T · THE TERMINAL VERDICT AND THE FINAL FORM (namespace
       YM.TERMINAL) · v1_1_0_CC

    The open framing awaits a proof of the sentence from what every frame
       shares, or a reading of the record that
    decides it, or a route to the sentence past the bit. Each count is
       refuted by theorem (`the_open_framing_is_refuted`),
    and what remains is the bit, its antecedent proved with no hypothesis
       and the bit proved equivalent to the sentence.
    The ten steps are bound in one theorem, steps 1 to 9 proved and step
       10 the two deliveries (`the_ten_steps`). The
    terminal verdict binds by exact type every theorem the verdict cites
       (`the_terminal_verdict`), and the final form
    binds the closed question, the two parts, the cut at strong coupling
       and the two deliveries (`the_final_form`).
    Every theorem on no axiom at all. ΔM = 0.
================================================================================
    ========================================
    -/
namespace YM.TERMINAL
open YMClose YM.NORMALFORM YM.HOST YM.SUPPLY YM.REALPART YM.ROWSPLIT
    YM.RESIDUE

/-- THE OPEN FRAMING, REFUTED, at every host. I · no proof from shared
    resources: no keyless premise forces the value,
     no given statement is the value on every frame, and the arrow forces
        nothing; II · no decision from the record: the
     two frames share a record and split on the value, no finite record of
        refinements forces the continuum whatever it
     records, and above every coupling no reading of the record returns
        the world; III · no premise reaches the sentence
     while the bit fails; IV · what remains is the bit, its antecedent
        proved with no hypothesis, the bit proved
     equivalent to the sentence. -/
theorem the_open_framing_is_refuted : ∀ H : Host,
    (∀ P : Frame → Prop, YM.NORMALFORM.Keyless ymRow P → ¬
       YM.NORMALFORM.Forces ymRow P) ∧
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    (¬ ∀ F : Frame, Arrow → Value F) ∧
    (Value (recFrame calm) ∧ Value (recFrame gapless) ∧ Value calm ∧ ¬
       Value gapless) ∧
    (∀ (n : Nat) (b l : Nat → Nat), ¬ Value (extendRecord n b l)) ∧
    (∀ T : Nat, ¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 =
       w.1) ∧
    (¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence) ∧ ¬ FormalCargo H) ∧
    Value (recFrame H.frame) ∧
    (FormalCargo H ↔ H.Sentence) :=
  fun H => (keyless_forces_nothing ymRow, given_is_not_the_value,
    arrow_forces_nothing, record_decides_nothing ymRow,
   fun n b l => (every_finite_record_never_forces n b l).2,
     unicorn_blocked_above_every_coupling,
   fun ⟨_, q, h, hn⟩ => hn ((formal_iff_sentence H).mpr (h q)),
   cargo_arrives H, formal_iff_sentence H)

/-- THE TEN STEPS, one theorem at every host. Step 1 · the floor: the
    root holds on its constructed domain. Step 2 ·
     the basis: the reading of any decided property as the actuation is
        that property, every theory lands on exactly one
     gate of each pair, and a prime's multiplicative fibre is two points.
        Step 3 · the domain: the host frame's value is
     the sentence, and the sentence is its two parts. Step 4 · the record:
        the record carries the value, and every entry
     stands on its floor. Step 5 · the halted uniduction: the frame is
        self-recursive exactly when the sentence holds.
     Step 6 · the monism zone: the two frames share one record and split
        on the value, and the bit is keyed. Step 7 ·
     the axis: the bit is exactly the sentence. Step 8 · the real part:
        the strong-coupling lattice gap decided by the
     cited fields, the value exactly its weak-coupling part, and the
        transfer matrix's gap read off its eigenvalues by the
     kernel. Step 9 · every route passes the bit, no keyless premise
        forces it, and the walls stand. Step 10 · the bit
     delivered twice, by the formal cargo and by the physical witness. -/
theorem the_ten_steps : ∀ H : Host,
    -- Step 1
    (Root Unit (fun _ => 1) ∧ RA₀) ∧
    -- Step 2
    ((∀ (F : Frame) (P : F.D → Prop) (dec : ∀ d, Decidable (P d)), Root
       F.D (fun d => indicator (dec d)) ↔ ∀ d, P d) ∧
      (∀ (F : Frame) (d : F.D), ((∃ n, F.built d = some n) ∨ F.built d =
         none) ∧
        (F.lowest d = 0 ∨ ∃ m, F.lowest d = m + 1)) ∧
      mulFibre 7 = [(1, 7), (7, 1)]) ∧
    -- Step 3
    ((Value H.frame ↔ H.Sentence) ∧ (H.Sentence ↔ (∀ G, ∃ n, H.theory G =
       some n) ∧ (∀ G, 0 < H.floor G))) ∧
    -- Step 4
```

```
    (Value (recFrame H.frame) ∧ ∀ (F : Frame) (d : F.D), (recFrame
      F).lowest d = 1) ∧
    -- Step 5
    (SelfRecursive ymRow H.frame ↔ H.Sentence) ∧
    -- Step 6
    ((Value (recFrame calm) ∧ Value (recFrame gapless) ∧ Value calm ∧ ¬
      Value gapless) ∧
      (Transfer ymRow calm ∧ ¬ Transfer ymRow gapless)) ∧
    -- Step 7
    (FormalCargo H ↔ H.Sentence) ∧
    -- Step 8
    ((∀ (L : Lattice) (d : L.F.D), L.β d ≤ L.β₀ → YMClose.Exists L.F d ∧
      Gapped L.F d) ∧
      (∀ L : Lattice, Value L.F ↔ UnicornAt (Ex L) (Pr L) L.β L.β₀) ∧
      (∀ L : Lattice, Nonempty (WeakSeat L) ↔ Value L.F) ∧
      (YM.WALLS.transfer 2 1 (1, 1) = (3, 3) ∧ YM.WALLS.transfer 2 1 (1,
        -1) = (1, -1))) ∧
    -- Step 9
    ((∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H) ∧
      (∀ P : Frame → Prop, YM.NORMALFORM.Keyless ymRow P → ¬
        YM.NORMALFORM.Forces ymRow P) ∧
      (∀ (n : Nat) (b l : Nat → Nat), ¬ Value (extendRecord n b l)) ∧
      (ExistsAll gapless ∧ ¬ GapAll gapless) ∧
      (∀ (M : Matter) (d : M.D), 0 < M.lowest d)) ∧
    -- Step 10
    ((FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence)) :=
  fun H =>
  ((constructed_root_holds, ra₀.instances ra₀.anAct),
   (reading_is_the_actuation, every_theory_lands, prime_shape.1),
   (binding H, sentence_is_two_parts H),
   (cargo_arrives H, fun F d => (every_entry_has_its_seat F d).2),
   (self_recursion_at_frame ymRow H.frame).trans (binding H),
   (record_decides_nothing ymRow, bit_is_keyed ymRow),
   formal_iff_sentence H,
   (strong_coupling_decided, no_bypass_at_strong_coupling,
     seat_iff_value,
    (YM.WALLS.transfer_record_normal_form.1,
      YM.WALLS.transfer_record_normal_form.2.1)),
   (fun _ h q => (formal_iff_sentence H).mpr (h q),
     keyless_forces_nothing ymRow,
    fun n b l => (every_finite_record_never_forces n b l).2,
      existence_does_not_give_gap, no_actual_gapless),
   delivered_twice H)

/-- THE TERMINAL VERDICT, AS A THEOREM. Every theorem the verdict cites,
    bound by its exact type into one pinned
    conjunction: the record and the normal form; the binding and the two
      parts; the real part and the cut at strong
    coupling; the walls; the residue and its seat; the supply and the
      witnesses typed; the closed question; the formal
    run; nothing escaping; the open framing refuted; the ten steps; the
      lock and the closure of the edition of record.
    No premise before the colon. -/
theorem the_terminal_verdict :
    (type_of% @YM.HOST.recFrame_value) ∧
    (type_of% @YM.NORMALFORM.normal_form) ∧
    (type_of% @YM.NORMALFORM.normal_form_complete) ∧
    (type_of% @YM.HOST.binding) ∧
    (type_of% @YM.HOST.sentence_is_two_parts) ∧
    (type_of% @YM.HOST.neither_part_gives_the_other) ∧
    (type_of% @YM.REALPART.real_part_strong) ∧
    (type_of% @YM.REALPART.no_bypass_at_strong_coupling) ∧
    (type_of% @YM.REALPART.unicorn_part_is_load_bearing) ∧
    (type_of% @YM.REALPART.unicorn_blocked_above_every_coupling) ∧
    (type_of% @YM.WALLS.transfer_record_normal_form) ∧
    (type_of% @YM.WALLS.the_walls) ∧
    (type_of% @YM.RESIDUE.residue_is_two_halves) ∧
    (type_of% @YM.RESIDUE.seat_iff_value) ∧
    (type_of% @YM.SUPPLY.the_supply_closed) ∧
    (type_of% @YM.SUPPLY.the_witnesses_typed) ∧
    (type_of% @YM.CLOSURE.question_closed_to_one_bit) ∧
    (type_of% @YM.GATES.the_formal_run) ∧
    (type_of% @YM.ESCAPE.nothing_escapes) ∧
    (type_of% @YM.ESCAPE.the_capstone) ∧
    (type_of% @YM.TERMINAL.the_open_framing_is_refuted) ∧
    (type_of% @YM.TERMINAL.the_ten_steps) ∧
    (type_of% @YMClose.the_lock) ∧
    (type_of% @YMClose.ym_closure) :=
  (@YM.HOST.recFrame_value, @YM.NORMALFORM.normal_form,
    @YM.NORMALFORM.normal_form_complete, @YM.HOST.binding,
   @YM.HOST.sentence_is_two_parts, @YM.HOST.neither_part_gives_the_other,
     @YM.REALPART.real_part_strong,
   @YM.REALPART.no_bypass_at_strong_coupling,
     @YM.REALPART.unicorn_part_is_load_bearing,
   @YM.REALPART.unicorn_blocked_above_every_coupling,
     @YM.WALLS.transfer_record_normal_form, @YM.WALLS.the_walls,
   @YM.RESIDUE.residue_is_two_halves, @YM.RESIDUE.seat_iff_value,
     @YM.SUPPLY.the_supply_closed,
   @YM.SUPPLY.the_witnesses_typed,
     @YM.CLOSURE.question_closed_to_one_bit, @YM.GATES.the_formal_run,
   @YM.ESCAPE.nothing_escapes, @YM.ESCAPE.the_capstone,
     @YM.TERMINAL.the_open_framing_is_refuted,
   @YM.TERMINAL.the_ten_steps, @YMClose.the_lock, @YMClose.ym_closure)

/-- THE FINAL FORM, at every host: the sentence is the bit; the sentence
    is exactly an inhabited physical witness; the
    sentence is exactly its two parts, neither giving the other; on every
      lattice frame the value is exactly its
    weak-coupling part; every route to the sentence is both kinds; and to
      shut every true route to the sentence is to
    assert its denial. -/
theorem the_final_form : ∀ H : Host,
    (H.Sentence ↔ FormalCargo H) ∧
    (H.Sentence ↔ Nonempty (PhysicalWitness H)) ∧
    (H.Sentence ↔ (∀ G, ∃ n, H.theory G = some n) ∧ (∀ G, 0 < H.floor G)) ∧
    (∀ L : Lattice, Value L.F ↔ UnicornAt (Ex L) (Pr L) L.β L.β₀) ∧
    (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H ∧ Nonempty
      (PhysicalWitness H)) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) :=
  fun H => (YM.CLOSURE.the_sentence_is_the_bit H, (physical_iff_sentence
    H).symm, sentence_is_two_parts H,
   no_bypass_at_strong_coupling, two_kinds_and_no_third H,
    refusal_no_proof H.Sentence)

end YM.TERMINAL

/-! ## cones of Parts A to T, pinned: the build is silent, and any drift
    fails it -/
/-- info: 'YM.NORMALFORM.record_has_the_value' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.record_has_the_value
/-- info: 'YM.NORMALFORM.normal_form' does not depend on any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.normal_form
/-- info: 'YM.NORMALFORM.value_from_the_cargo_loop' does not depend on
    any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.value_from_the_cargo_loop
/-- info: 'YM.NORMALFORM.bit_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.bit_is_the_act
/-- info: 'YM.NORMALFORM.record_is_self_recursive' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.record_is_self_recursive
/-- info: 'YM.NORMALFORM.self_recursion_at_frame' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.self_recursion_at_frame
/-- info: 'YM.NORMALFORM.bit_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.bit_is_keyed
/-- info: 'YM.NORMALFORM.record_self_recursion_does_not_transfer' does
    not depend on any axioms -/
#guard_msgs in #print axioms
    YM.NORMALFORM.record_self_recursion_does_not_transfer
/-- info: 'YM.NORMALFORM.record_decides_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.record_decides_nothing
/-- info: 'YM.NORMALFORM.every_route_passes_the_bit' does not depend on
    any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.every_route_passes_the_bit
/-- info: 'YM.NORMALFORM.keyless_forces_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.keyless_forces_nothing
/-- info: 'YM.NORMALFORM.no_record_principle_decides' does not depend on
    any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.no_record_principle_decides
/-- info: 'YM.NORMALFORM.shared_record_decides_nothing' does not depend
    on any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.shared_record_decides_nothing
/-- info: 'YM.NORMALFORM.no_uniform_answer' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.NORMALFORM.no_uniform_answer
/-- info: 'YM.NORMALFORM.nothing_outside_the_record_is_the_bit' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    YM.NORMALFORM.nothing_outside_the_record_is_the_bit
/-- info: 'YM.NORMALFORM.denial_is_the_bits_denial' does not depend on
    any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.denial_is_the_bits_denial
/-- info: 'YM.NORMALFORM.normal_form_complete' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.normal_form_complete
/-- info: 'YM.NORMALFORM.bnot_ne' does not depend on any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.bnot_ne
/-- info: 'YM.NORMALFORM.refusal_waiting' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.NORMALFORM.refusal_waiting
/-- info: 'YM.NORMALFORM.refusal_no_proof' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.NORMALFORM.refusal_no_proof
/-- info: 'YM.NORMALFORM.block_needs_an_outside' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.block_needs_an_outside
/-- info: 'YM.NORMALFORM.comp_involution' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.NORMALFORM.comp_involution
/-- info: 'YM.NORMALFORM.locus_vanishes' does not depend on any axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.locus_vanishes
/-- info: 'YM.NORMALFORM.join_reads_the_weakest' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.join_reads_the_weakest
/-- info: 'YM.NORMALFORM.square_is_sign_blind' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.NORMALFORM.square_is_sign_blind
/-- info: 'YM.ROWSPLIT.unicorn_block' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ROWSPLIT.unicorn_block
/-- info: 'YM.ROWSPLIT.aperture_one_bit_wide' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.ROWSPLIT.aperture_one_bit_wide
/-- info: 'YM.ROWSPLIT.unicorn_rule_exact' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.ROWSPLIT.unicorn_rule_exact
/-- info: 'YM.ROWSPLIT.unicorn_no_bypass' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.ROWSPLIT.unicorn_no_bypass
/-- info: 'YM.ROWSPLIT.supply_side_silent' does not depend on any axioms
    -/
#guard_msgs in #print axioms YM.ROWSPLIT.supply_side_silent
/-- info: 'YM.ROWSPLIT.whole_iff_real_and_unicorn' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.ROWSPLIT.whole_iff_real_and_unicorn
/-- info: 'YM.ROWSPLIT.real_part_of_certificate' does not depend on any
    axioms -/
#guard_msgs in #print axioms YM.ROWSPLIT.real_part_of_certificate
/-- info: 'YM.HOST.recFrame_value' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.recFrame_value
```

```
/-- info: 'YM.HOST.every_entry_has_its_seat' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.every_entry_has_its_seat
/-- info: 'YM.HOST.binding' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.binding
/-- info: 'YM.HOST.root_on_row_is_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.root_on_row_is_the_sentence
/-- info: 'YM.HOST.act_is_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.act_is_the_sentence
/-- info: 'YM.HOST.sentence_is_two_parts' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.sentence_is_two_parts
/-- info: 'YM.HOST.neither_part_gives_the_other' does not depend on any axioms -/
#guard_msgs in #print axioms YM.HOST.neither_part_gives_the_other
/-- info: 'YM.REALPART.value_is_the_whole' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.value_is_the_whole
/-- info: 'YM.REALPART.real_part_strong' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.real_part_strong
/-- info: 'YM.REALPART.value_iff_real_and_unicorn' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.value_iff_real_and_unicorn
/-- info: 'YM.REALPART.no_bypass_at_strong_coupling' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.no_bypass_at_strong_coupling
/-- info: 'YM.REALPART.unicorn_is_weak_coupling' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.unicorn_is_weak_coupling
/-- info: 'YM.REALPART.unicorn_part_is_load_bearing' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.unicorn_part_is_load_bearing
/-- info: 'YM.REALPART.unicorn_blocked_above_every_coupling' does not depend on any axioms -/
#guard_msgs in #print axioms
    YM.REALPART.unicorn_blocked_above_every_coupling
/-- info: 'YM.REALPART.supply_side_silent_above_every_coupling' does not depend on any axioms -/
#guard_msgs in #print axioms
    YM.REALPART.supply_side_silent_above_every_coupling
/-- info: 'YM.REALPART.aperture_is_one_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.REALPART.aperture_is_one_bit
/-- info: 'YM.WALLS.transfer_record_normal_form' does not depend on any axioms -/
#guard_msgs in #print axioms YM.WALLS.transfer_record_normal_form
/-- info: 'YM.WALLS.trace_reads_no_gap' does not depend on any axioms -/
#guard_msgs in #print axioms YM.WALLS.trace_reads_no_gap
/-- info: 'YM.WALLS.the_walls' does not depend on any axioms -/
#guard_msgs in #print axioms YM.WALLS.the_walls
/-- info: 'YM.RESIDUE.sentence_of_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.sentence_of_the_act
/-- info: 'YM.RESIDUE.residue_is_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.residue_is_the_sentence
/-- info: 'YM.RESIDUE.residue_is_two_halves' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.residue_is_two_halves
/-- info: 'YM.RESIDUE.the_posit_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.the_posit_is_the_bit
/-- info: 'YM.RESIDUE.residue_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.residue_is_keyed
/-- info: 'YM.RESIDUE.value_of_seat' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.value_of_seat
/-- info: 'YM.RESIDUE.seat_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.seat_iff_value
/-- info: 'YM.RESIDUE.seat_is_the_unicorn' does not depend on any axioms -/
#guard_msgs in #print axioms YM.RESIDUE.seat_is_the_unicorn
/-- info: 'YM.SUPPLY.cargo_arrives' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.cargo_arrives
/-- info: 'YM.SUPPLY.formal_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.formal_kind_delivers
/-- info: 'YM.SUPPLY.physical_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.physical_kind_delivers
/-- info: 'YM.SUPPLY.witness_closes' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.witness_closes
/-- info: 'YM.SUPPLY.physical_iff_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.physical_iff_sentence
/-- info: 'YM.SUPPLY.formal_iff_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.formal_iff_sentence
/-- info: 'YM.SUPPLY.the_two_kinds_are_one_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.the_two_kinds_are_one_bit
/-- info: 'YM.SUPPLY.delivered_twice' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.delivered_twice
/-- info: 'YM.SUPPLY.two_kinds_and_no_third' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.two_kinds_and_no_third
/-- info: 'YM.SUPPLY.the_one_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.the_one_refuter
/-- info: 'YM.SUPPLY.the_witnesses_typed' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.the_witnesses_typed
/-- info: 'YM.SUPPLY.the_supply_closed' does not depend on any axioms -/
#guard_msgs in #print axioms YM.SUPPLY.the_supply_closed
/-- info: 'YM.CLOSURE.the_sentence_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.CLOSURE.the_sentence_is_the_bit
/-- info: 'YM.CLOSURE.denial_is_the_bits_denial' does not depend on any axioms -/
#guard_msgs in #print axioms YM.CLOSURE.denial_is_the_bits_denial
/-- info: 'YM.CLOSURE.refuter_denies_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.CLOSURE.refuter_denies_the_sentence
/-- info: 'YM.CLOSURE.the_denial_has_one_road' does not depend on any axioms -/
#guard_msgs in #print axioms YM.CLOSURE.the_denial_has_one_road
/-- info: 'YM.CLOSURE.question_closed_to_one_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.CLOSURE.question_closed_to_one_bit
/-- info: 'YM.GATES.gate_1_srep' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_1_srep
/-- info: 'YM.GATES.gate_2_reg' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_2_reg
/-- info: 'YM.GATES.gate_3_sgeg' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_3_sgeg
/-- info: 'YM.GATES.gate_4_causal' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_4_causal
/-- info: 'YM.GATES.gate_5_mig' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_5_mig
/-- info: 'YM.GATES.gate_6_ptb' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_6_ptb
/-- info: 'YM.GATES.gate_7_dual' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_7_dual
/-- info: 'YM.GATES.gate_8_cscg' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_8_cscg
/-- info: 'YM.GATES.gate_9_cseg' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_9_cseg
/-- info: 'YM.GATES.gate_10_mta' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_10_mta
/-- info: 'YM.GATES.gate_11_oma' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_11_oma
/-- info: 'YM.GATES.gate_12_adeg' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.gate_12_adeg
/-- info: 'YM.GATES.refusal_proved' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.refusal_proved
/-- info: 'YM.GATES.refusal_no_proof_at_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.refusal_no_proof_at_the_bit
/-- info: 'YM.GATES.the_formal_run' does not depend on any axioms -/
#guard_msgs in #print axioms YM.GATES.the_formal_run
/-- info: 'YM.ESCAPE.the_record_closes' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.the_record_closes
/-- info: 'YM.ESCAPE.the_undecided_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.the_undecided_is_the_bit
/-- info: 'YM.ESCAPE.nothing_outside_the_record_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms
    YM.ESCAPE.nothing_outside_the_record_is_the_bit
/-- info: 'YM.ESCAPE.the_outside_is_the_supply' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.the_outside_is_the_supply
/-- info: 'YM.ESCAPE.every_route_is_both_kinds' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.every_route_is_both_kinds
/-- info: 'YM.ESCAPE.both_kinds_deliver' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.both_kinds_deliver
/-- info: 'YM.ESCAPE.our_way_or_no_way' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.our_way_or_no_way
/-- info: 'YM.ESCAPE.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.nothing_escapes
/-- info: 'YM.ESCAPE.the_capstone' does not depend on any axioms -/
#guard_msgs in #print axioms YM.ESCAPE.the_capstone
/-- info: 'YM.TERMINAL.the_open_framing_is_refuted' does not depend on any axioms -/
#guard_msgs in #print axioms YM.TERMINAL.the_open_framing_is_refuted
/-- info: 'YM.TERMINAL.the_ten_steps' does not depend on any axioms -/
#guard_msgs in #print axioms YM.TERMINAL.the_ten_steps
/-- info: 'YM.TERMINAL.the_terminal_verdict' does not depend on any axioms -/
#guard_msgs in #print axioms YM.TERMINAL.the_terminal_verdict
/-- info: 'YM.TERMINAL.the_final_form' does not depend on any axioms -/
#guard_msgs in #print axioms YM.TERMINAL.the_final_form
```

Appendix R · The root on no axiom and no posit

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, `TOE_Zero.lean`, is carried verbatim from the root paper (Islam 2026h). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, `SelfGrounding`, `SelfVerifying`, ΔE_0, RA_0 and ra_0, and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds

itself when deeds occur and every deed instances it (`SelfGrounding`); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x,\ 0 < \Delta E_0(x)$ (`RA₀`). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (`root_on_the_constructed_domain`). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (`the_floor_is_universal`). Every denial of the root is a deed that instances it (`denial_reenacts_root`); the root is held by its deed (`seated_undeniable`); it is self-verifying (`denial_instantiates`); and no level stands above it (`no_level_above`). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (`the_master_seal`); it states the three counts side by side and no map between them. The row kernel repeats this kernel's three declarations `ΔE₀`, `RA₀` and `ra₀` verbatim, so the identity of the constructed root across the two files is a textual check; there the constructed root is proved to be the row's root on the one-point domain with actuation one (`RA₀_is_the_constructed_root`) and is bound to the row in one theorem (`the_lock_on_the_root`). The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

```
0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238
```

```
/-
  TOE_Zero.lean · Theory of Theories of Everything (TOE of All TOEs): Existence
  Proves Existence Only by Motion
  The zero-axiom, zero-posit kernel. Core Lean 4: no import, no axiom declared, no
  sorry, no native_decide, and
  every theorem rests on no axiom at all, not even propositional extensionality,
  quotients or choice.
  I · The self-grounding root. Acts occur and every act instances the root. Then
  every denial is an act and
      re-enacts the root, the root is held by the act itself, and no external
      proof adds anything to it: the
      Omega Boundary, proved for every root that grounds itself, universally, by
      quantification.
  II · The constructed root. On the constructed domain the Root Axiom is a theorem.
  III · The Return and the line. On the integer quaternions i·j·k = −1 and each
  unit squares to −1; conjugation is
      an involution and its fixed set on the chart is exactly the scalar line.
  IV · The twelve gates, recovered three ways inside the integers of the
  quaternions: the twenty-four Hurwitz units
      counted twelve up to sign; the twelve even permutations with class equation
      1, 3, 4, 4; and the norm-two
      shell of twenty-four, split twelve and twelve.
  V · The regress terminated. Above a self-grounding root no level grounds it
  further.
  VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R
:= G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by the act itself.
-/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R := G.instances
G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q : Prop) :
(Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) : SelfVerifying R :=
fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the root holds,
every act re-enacts it, and no
    external proof adds to it. -/
theorem the_floor_is_universal :
    ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop, (Q →
    R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one existent actuates,
at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed, and the deed
actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b * y.a + x.c *
  y.d - x.d * y.c,
   x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b * y.c - x.c *
   y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = −1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to −1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj = qneg ∧ qk.mul
qk = qneg := by decide

/-- The chart: every quaternion with coordinates in −2 … 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c => chart.map
  fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q => q.conj.conj == q) =
true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the chart. -/
theorem the_line_is_the_fixed_set :
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) =
    true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 != 0 && b % 2
  != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates −2 … 2 hold every unit (doubled norm four) and every
element of the norm-two shell (doubled
    norm eight), since a doubled coordinate of absolute value three or more
    already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c => chart.map
  fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
```

```
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 4 && hurwitz
  a b c d

/-- The first nonzero coordinate is positive: one representative of each pair ±u.
-/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then 0 < c else 0 <
  d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧ (units.filter
upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 8 && hurwitz
  a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split twelve and
twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
    shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧ (shell.filter (fun p
    => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1, 2, 3].flatMap
  fun c =>
    [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length == 4 then
    some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i, j)).countP
    fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p % 2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i => p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h => compose
(compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a class of size
s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
    gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g => (conjClass
    g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a further ground of
the root is idle; the root needs
    no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q : Prop, (Q → R) → R
:= fun _ _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem on the
constructed domain; the Return closes;
    the line is the fixed set; and the twelve gates are recovered three ways. -/
theorem the_master_seal :
    (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop, (Q →
    R) ↔ R)) ∧
    RA₀ ∧ (qi.mul qj).mul qk = qneg ∧
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) =
    true ∧
    (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧ (shell.filter
    upToSign).length = 12 ∧
    collapse 12 (sortNat (gates.map fun g => (conjClass g).length)) = [1, 3, 4, 4]
    :=
  (the_floor_is_universal, root_on_the_constructed_domain, the_return,
  the_line_is_the_fixed_set,
   twenty_four_units_twelve_gates.2, the_class_equation.1,
   the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal
```

The root's axes are proved in the second kernel, R.2, `Triaxial_Actuation.lean` (32 theorems, every one on no axiom; SHA-256 `566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b`). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (`actuation_is_triaxial`, `registration_is_forced`, `the_only_special_cut`, `tongue_freedom`, `every_root_actuates`, `the_root_carries_three_axes`). The root's grade is read in the third kernel, R.3, `Root_Grade_Ledger.lean` (18 theorems, every one on no axiom; SHA-256 `c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8`). On the grade ledger of the Master Codex (Islam 2026k), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA–Tongue co-location inherits the root's grade exactly (`a_warranted_root_is_theorem_grade`, `the_ledger_reads_the_warrant`, `the_grade_is_not_elected`, `ra_is_theorem_grade`, `coloc_grade_is_the_roots_grade`, `the_ledger_seal`). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026j), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike

by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```
/-
  Triaxial_Actuation.lean · actuation is triaxial

  Core Lean 4: no import, no axiom declared, no sorry, no native_decide, and every
  theorem rests on no
  axiom at all, not even propositional extensionality, quotients or choice.

  An actuation is a self-grounding root whose acts carry a direction: each act has
  a side and a height,
  and reversing the deed reverses the side and keeps the height. From the
  direction alone the kernel
  constructs three axes and proves them forced:
  I    The fold: reversing the deed is an involution.
  II   The registration: it keeps the height and forgets the side; it is blind to
  the fold, closed with
       it under combination, and it is the only height-keeping map onto the seat.
  III  The seat: the fold's fixed set, where the fold and the registration both
  fix the act, and off
       which neither does.
  IV   The third axis: off the seat the fold's orbit has two acts with one record,
  so the record leaves
       exactly the orientation open; every reading blind to the fold reads both
       alike.
  V    The root through the axes: a reversed deed is still an act and instances
  the root; the axes are
       carried by every actuation, and they force no orientation.
  VI   The bridge atom: the integers, with negation as the fold, are an actuation.
  VII  The triaxial seal.
  VIII The Tongue: no reading of the record returns the orientation; the seat is
  the only special cut.
  IX   Every root actuates: a self-grounding root is the root of an actuation over
  its own acts, so the
       root carries the three axes; the registration is the fold's record and
       lands on the seat, where
       the orientation an act and its reversal share stays open; the even pair
       leaves two acts, the
       orientation locks one; and a degenerate direction, one side only, is the
       characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction -/

/-- The direction of a deed: a side with a reversal, whose one fixed side is the
zero side. -/
structure Direction where
  Side     : Type
  rev      : Side → Side
  zero     : Side
  rev_rev  : ∀ s, rev (rev s) = s
  rev_zero : rev zero = zero
  fix_zero : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a side and a
height. -/
structure Actuation (R : Prop) where
  dir       : Direction
  Height    : Type
  anAct     : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side × A.Height :=
(A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side × A.Height :=
(A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop := fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side × A.Height) : fold A
(fold A p) = p := by
  cases p with
  | mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw [A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side × A.Height) : reg A
(fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side × A.Height) : (reg A
p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) : fold A (reg A
p) = reg A p := by
  show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw [A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height) : reg A (reg A
p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the height and
lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side × A.Height →
A.dir.Side × A.Height)
    (keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p, g p = reg A p
    := by
  intro p
  have hs : (g p).1 = A.dir.zero := by
    have hf := lands p
    have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
    exact A.dir.fix_zero _ h1
  have hh := keeps p
  cases hg : g p with
  | mk s h =>
    rw [hg] at hs hh
    show (s, h) = (A.dir.zero, p.2)
    rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side × A.Height) : OnSeat A
p ↔ p.1 = A.dir.zero :=
  (fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
   fun h => by
     cases p with
     | mk s t =>
       show (A.dir.rev s, t) = (s, t)
       have hs : s = A.dir.zero := h
       rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the registration both fix
the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side × A.Height) (h :
OnSeat A p) :
    fold A p = p ∧ reg A p = p := by
  refine (h, ?_)
  have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
  cases p with
  | mk s t =>
    have hs : s = A.dir.zero := hz
    show (A.dir.zero, t) = (s, t)
    rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration fixes the act.
-/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height) (h : ¬ OnSeat A
p) :
    fold A p ≠ p ∧ reg A p ≠ p := by
  refine (h, fun hr => h ?_)
  have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
  exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act and its
reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : ¬ OnSeat A p) :
    fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (h, rfl)

/-- Every reading blind to the fold reads an act and its reversal alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f : A.dir.Side ×
A.Height → α)
    (blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) : f (fold A p) =
    f p :=
  blind p

/-- The registration is such a reading, and so is every reading of the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α : Type} (k :
A.dir.Side × A.Height → α)
    (p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p) := rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the denial spends the
deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side × A.Height) : R :=
  A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.anAct
```

```
/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations are acts, both
instance the root, and
the record reads them alike; the triaxial structure leaves the one orientation
open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side × A.Height) (h
: ¬ OnSeat A p) :
    R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(-s) = s
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (_ + 1), h => Int.noConfusion h
  | Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := ⟨Int, fun s => -s, 0, int_neg_neg, rfl, self_neg_zero⟩

/-- The bridge atom: points (d, t), the fold (d, t) ↦ (−d, t), the registration
(d, t) ↦ (0, t). -/
def atom : Actuation True := ⟨intDir, Int, ((0 : Int), (0 : Int)), fun _ =>
True.intro⟩

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly when d = 0,
the registration
leaves it unchanged exactly then, and off the seat the record leaves the
orientation. -/
theorem the_bridge_atom (d t : Int) :
    (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0) ∧
    (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t)) = reg atom
    (d, t)) :=
  ⟨seat_iff_zero_side atom (d, t),
   ⟨fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl⟩,
   fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
   ((seat_iff_zero_side atom (d, t)).mp hs))⟩

/-- The atom is not characterless: off the seat it has two acts with one record. -/
theorem the_atom_is_not_characterless :
    fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg atom (fold
    atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0 : Int)) :=
  ⟨fun h => Int.noConfusion (congrArg Prod.fst h), rfl⟩

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an involution, a
registration that
keeps the height, forgets the side and is the only such map onto the seat, and a
seat where both fix
the act and off which neither does; off the seat the record leaves exactly the
orientation; every act
and its reversal instance the root; and the axes force no orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
    (∀ p, fold A (fold A p) = p) ∧
    (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A (reg A p) = reg A
    p) ∧
    (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
       (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg A p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
    (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p) :=
  ⟨fold_involutive A, fun p => ⟨reg_forgets_side A p, reg_keeps_height A p,
  fold_of_reg A p⟩,
   registration_is_forced A, seat_cancels_both A, off_seat_both A,
   axes_force_no_orientation A⟩

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the bridge atom is an
actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
    (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
      (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
      (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A p)) ∧
    (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
    (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg atom (fold
    atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0 : Int))) :=
  ⟨fun _ A => ⟨fold_involutive A, seat_cancels_both A,
     fun p h => record_leaves_the_orientation A p h⟩,
   fun d t => seat_iff_zero_side atom (d, t),
   the_atom_is_not_characterless⟩


/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the orientation. If a record
is blind to the fold
and an orientation is flipped by it, the orientation is not a function of the
record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β → Bool) (d : α →
Bool) (x : α)
    (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots cancel;
nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side × A.Height) :
    (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  ⟨fun h => h.1, fun h => seat_cancels_both A p h⟩

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its reversal alike
and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q : A.dir.Side ×
A.Height) :
    reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  ⟨rfl, ⟨fun h => show (reg A p).2 = (reg A q).2 from congrArg Prod.snd h, fun h
  => by
    cases p with
    | mk s t =>
      cases q with
      | mk s' t' =>
        have ht : t = t' := h
        show (A.dir.zero, t) = (A.dir.zero, t')
        rw [ht]⟩⟩

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side × A.Height) : OnSeat A
(reg A p) :=
  fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and its reversal
land on one seat point, the record
of both; at that place the record has left exactly the orientation open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side × A.Height) (h : ¬
OnSeat A p) :
    OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  ⟨reg_lands_on_seat A p, rfl, h⟩

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the fold, the record
among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side × A.Height) (h : ¬
OnSeat A p) {α : Type}
    (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f q) :
    fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  ⟨h, blind p, rfl⟩

/-- THREE AXES LOCK ONE: the record together with the orientation, the side,
determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q : A.dir.Side × A.Height)
    (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>
    cases q with
    | mk s' t' =>
      have ht : t = t' := congrArg Prod.snd hr
      have hs' : s = s' := hs
      rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is on the seat,
and the fold and the
registration are both the identity: one scalar, no fold, no seat to lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h : A.dir.Degenerate) (p :
A.dir.Side × A.Height) :
    OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  ⟨hs, seat_cancels_both A p hs⟩

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the actuation over
its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the root as before.
-/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  ⟨intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2⟩

theorem every_root_actuates (S : SelfGrounding R) :
    (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
    (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
    ¬ S.toActuation.dir.Degenerate :=
  ⟨fun _ => rfl,
   fun a h =>
     have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation ((1 : Int),
     a)).mp h
     absurd h0 (by decide),
   intDir_not_degenerate⟩
```

```
/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its actuation
carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only height-keeping map
onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height an act off
the seat, its reversal a second
act with the same record, both instancing the root, the orientation between them
forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
    (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
    (∀ p, reg S.toActuation (fold S.toActuation p) = reg S.toActuation p ∧ (reg
    S.toActuation p).2 = p.2) ∧
    (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p, OnSeat
    S.toActuation (g p)) →
       ∀ p, g p = reg S.toActuation p) ∧
    (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg S.toActuation p =
    p) ∧
    (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
       reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg S.toActuation
       ((1 : Int), a)) ∧
    (∀ p, ¬ OnSeat S.toActuation p →
       R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold S.toActuation p)
       = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
   fun p => (reg_forgets_side S.toActuation p, reg_keeps_height S.toActuation p),
   registration_is_forced S.toActuation,
   seat_cancels_both S.toActuation,
   fun a => record_leaves_the_orientation S.toActuation ((1 : Int), a)
   ((every_root_actuates S).2.1 a),
   axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes
```

```
/-
  Root_Grade_Ledger.lean · the root at theorem grade, and RA–Tongue co-location
  with it

  Core Lean 4: no import, no axiom declared, no sorry, no native_decide, and every
  theorem rests on no
  axiom at all, not even propositional extensionality, quotients or choice.

  The floor Codex (Master Codex v5.1.0, Codex.lean) grades every claim on one
  ledger and joins claims
  at the weakest link. It declares the root as `axiom RA` and pins `raClaim.grade =
  .premise`; its
  co-location claim, the root joined with the wall, therefore inherits `.premise`.
  This kernel carries
  the same ledger and the same join law with no axiom declared, warrants the root
  by a theorem with no
  axiom and no hypothesis, and proves that the co-location claim then stands at
  theorem grade.
  I    The grade ledger and the weakest-link join, as in the Codex.
  II   The root, a theorem: the constructed domain of TOE_Zero.lean, warranted on
  no axiom.
  III  The wall, a theorem: the Codex's K4 frame, its target not a function of its
  record, proved on
       no axiom through Tongue freedom.
  IV   The claims: the root at theorem grade, the wall at theorem grade, and their
  co-location.
  V    Why the Codex read premise: a join's grade is its weakest link, and
  co-location's grade is the
       root's grade exactly; with the root at theorem grade, co-location is a
       theorem.
  VI   The ledger seal.
  VII  The grade read from the warrant, as the Codex reads it: a root with a
  warrant, a proof of its
       statement, is theorem grade; a root posited without one, the Codex's `axiom
       RA`, is premise grade;
       the two claims of section IV carry exactly the grades this rule reads.
-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
  | premise | corroboration | operational | structural | engineering
  | conditional | theoremConditional | analytic | theorem
  deriving DecidableEq

def Grade.rank : Grade → Nat
  | .premise => 0 | .corroboration => 1 | .operational => 2 | .structural => 3
  | .engineering => 4 | .conditional => 5 | .theoremConditional => 6
  | .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a else b

/-- A claim carries its statement, its grade and its warrant, a proof of the
statement. -/
structure Claim : Type where
  stmt    : Prop
  grade   : Grade
  warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
  stmt    := c₁.stmt ∧ c₂.stmt
  grade   := Grade.weakest c₁.grade c₂.grade
  warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade = Grade.weakest
c₁.grade c₂.grade := rfl

/-! ## II · the root, a theorem (the constructed domain of TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R := G.instances
G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable ra₀

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an orientation is flipped
by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β → Bool) (d : α →
Bool) (x : α)
    (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
```

```
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
  | 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
  | 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
  | _ => 0

def tau : Nat → Nat := fun i =>
  if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom and no
hypothesis. -/
def raClaim₀ : Claim where
  stmt    := RA₀
  grade   := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt    := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade   := .theorem
  warrant := K4.wall

/-- RA–Tongue co-location: the root joined with the wall, as in the Codex. -/
def colocClaim₀ : Claim := Claim.join raClaim₀ wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison: the same
statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt    := RA₀
  grade   := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem = g := by
  cases g <;> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a theorem, so
the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
    (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the co-location
claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join raClaimPremise
wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim₀.grade = .theorem := rfl

/-- RA–TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim₀.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall, both proved. -/
theorem coloc_holds : colocClaim₀.stmt := colocClaim₀.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no hypothesis; the wall
is a theorem; their
co-location holds and stands at theorem grade; and co-location's grade is the
root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
    RA₀ ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
    colocClaim₀.stmt ∧ colocClaim₀.grade = .theorem ∧
    (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
    (Claim.join raClaimPremise wallClaim).grade = .premise :=
  ⟨root_on_the_constructed_domain, K4.wall, coloc_holds, coloc_is_theorem_grade,
   coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc⟩

/-! ## VII · the grade read from the warrant, as the Codex reads it -/

/-- The evidence a ledger entry carries for a statement: posited with no warrant,
or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
  | posited
  | warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade is a proof
with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem grade; a root
posited with none, as the Codex
declares `axiom RA`, is premise grade. -/
def rootGrade : Evidence RA₀ → Grade
  | .warranted _ => .theorem
  | .posited     => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II carries its warrant,
so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
    rootGrade (.warranted root_on_the_constructed_domain) = .theorem := rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule reads premise,
the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited = .premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV carries the grade
the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none. -/
theorem the_ledger_reads_the_warrant :
    raClaim₀.grade = rootGrade (.warranted raClaim₀.warrant) ∧
    raClaimPremise.grade = rootGrade .posited :=
  ⟨rfl, rfl⟩

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant and premise
from none, never the other way;
whatever label an entry carries, the warranted root's grade under the rule is
theorem. -/
theorem the_grade_is_not_elected :
    (∀ w : RA₀, rootGrade (.warranted w) = .theorem) ∧ rootGrade .posited ≠
    .theorem ∧
    (∀ w : RA₀, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  ⟨fun _ => rfl, fun h => Grade.noConfusion h, fun _ h => Grade.noConfusion h⟩

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-- info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-- info: 'RGL.root_on_the_constructed_domain' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-- info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-- info: 'RGL.K4.register_even_at_zero' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-- info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-- info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-- info: 'RGL.weakest_with_theorem' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-- info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected
```

Author's Provenance and Method Disclosure

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full in the programme's master reference (Islam 2026i).

Author and method. Mohammad F. Islam, PhD, in-

dependent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. One claim was held at the model's insistence: the matter face is graded at its weakest field, the premise on the massless case, beside the record of confinement, and read on the pure-gauge sector only, because the light pion of chiral QCD is colour-neutral and a massless colour singlet would refute the premise.

The register. In the programme's vocabulary: the form, the act's identity with the value, the universal closure, the division and the deduction of the lattice proved part stand at [{ɷ},T] on no axiom, with every theorem of the master, one hundred seventy-four in all, and the conclusion of the lattice proved part at the grade of its two cited theorems, the same rule as the matter face; the matter face at the grade of its weakest field, premise grade; the triaxial reading at [{ɷ},S]; the continuum value at premise grade on the act; $\Delta M = 0$ on the cited physics. The act is the row's least-erasure posit read as existence confined; the row carries no defeater. The freedom cut, prime-as-freedom and the triaxial lock are applied on the row, with the lessons of the Riemann, P versus NP, Navier--Stokes and Hodge closures combined, and the keyless and keyed denial of the programme's bridge and operating system.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (`every_root_actuates`, `degenerate_is_characterless`). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat	`every_root_actuates`, `the_root_carries_three_axes`, `actuation_is_triaxial`, `registration_is_forced` (Appendix R.2)
the registration is the only height-keeping map onto the seat	`registration_is_forced` (R.2)
the axes force no orientation	`axes_force_no_orientation` (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	`fold`, `denial_instances_root` (R.2); `denial_reenacts_root`, `denial_instantiates` (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination	`registration_is_the_folds_record`, `reg_forgets_side`, `reg_keeps_height`, `fold_involutive`, `fold_of_reg` (R.2)
the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	`readings_of_the_record_are_blind`, `tongue_freedom` (R.2); `symmetry_is_keyless`, `line_property_is_keyed`, `exactly_one_stands` (Islam 2026k, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)

Clause of the statement	Receipt
the seat is the fixed set, where both blind spots cancel; off the seat both stand	`seat_iff_zero_side`, `seat_cancels_both`, `off_seat_both`, `the_only_special_cut` (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	`reg_lands_on_seat`, `open_at_the_seat`, `record_leaves_the_orientation`, `readings_of_the_record_are_blind` (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	`degenerate_is_characterless`, `intDir_not_degenerate`, `every_root_actuates` (R.2)
the bridge atom is an actuation, and not characterless	`the_bridge_atom`, `the_atom_is_not_characterless`, `the_triaxial_seal` (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	`the_return`, `the_line_is_the_fixed_set`, `twenty_four_units_twelve_gates`, `the_class_equation`, `the_master_seal` (R.1)
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	`a_warranted_root_is_theorem_grade`, `the_ledger_reads_the_warrant`, `the_grade_is_not_elected`, `ra_is_theorem_grade`, `coloc_grade_is_the_roots_grade`, `the_ledger_seal` (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	`even_pair_leaves_two`, `orientation_locks_the_act`, `seat_cancels_both` (R.2); in its own coordinates, for independent axes in $\mathbb{F}_2^3$, `two_axes_leave_two`, `three_axes_lock_one` (this paper's kernel), a second statement
the root alone forces no value; the root read on the row is the value	`undeniable_root_forces_no_value`; `root_on_row_is_the_value`, `act_is_the_value` (this paper's kernel)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	`every_root_actuates`, `the_root_carries_three_axes` (R.2), stated for every self-grounding root and read at the constructed root $\mathtt{ra}_0$ of this paper's kernel, whose structure repeats R.2's `SelfGrounding` field for field; this kernel states no seat of its own; its fold on the value bit is the swap of the two worlds that share one record, the Bool model (`record_wall`, `fibre_is_two`), and the seat its actuation carries is the record both deeds land on; the row ledger of Islam (2026d, Book III), cited

References

Creutz, M. 1980. Monte Carlo study of quantized SU(2) gauge theory. *Physical Review D* 21: 2308--2315.

de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.

Gross, D. J. and F. Wilczek. 1973. Ultraviolet behavior of non-abelian gauge theories. *Physical Review Letters* 30: 1343--1346.

Islam, M. F. 2026a. Navier--Stokes global regularity: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.22670344.

Islam, M. F. 2026b. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.

Islam, M. F. 2026c. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.

Islam, M. F. 2026d. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.

Islam, M. F. 2026e. The Hodge conjecture: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.22701510.

Islam, M. F. 2026f. PhysOS^{T}: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.

Islam, M. F. 2026g. The proton is the lock: three colors, one singlet, and the gap made flesh. Zenodo. doi:10.5281/zenodo.22986559.

Islam, M. F. 2026h. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.

Islam, M. F. 2026i. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.

Islam, M. F. 2026j. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.

Islam, M. F. 2026k. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapients/Trisduction, GitHub.

Islam, M. F. 2026l. An unconditional proof of the real part of the Riemann Hypothesis, the probable remainder closed to symmetry and computation at theorem grade, and the whole proved from one bit supplied by the act of computation or by physics itself. Edition 1.3.0, its master RH_Both_Sides_Formal_Master v3.0.39_CC. Zenodo. doi:10.5281/zenodo.23440382.

Islam, M. F. 2026m. P $\neq$ NP: unconditional proof of the form by theorem, the value by the act. Edition 1.13.0, its master PNP_Normal_Form_Paper_v1_13_0_CC. Zenodo. doi:10.5281/zenodo.23528398.

Jaffe, A. and E. Witten. 2006. Quantum Yang--Mills theory. In *The Millennium Prize Problems*, 129--152. Clay Mathematics Institute.

Morningstar, C. J. and M. Peardon. 1999. Glueball spectrum from an anisotropic lattice study. *Physical Review D* 60: 034509.

Navas, S. et al. (Particle Data Group). 2024. Review of particle physics. *Physical Review D* 110: 030001.

Osterwalder, K. and E. Seiler. 1978. Gauge field theories on a lattice. *Annals of Physics* 110: 440--471.

Politzer, H. D. 1973. Reliable perturbative results for strong interactions? *Physical Review Letters* 30: 1346--1349.

Seiler, E. 1982. *Gauge Theories as a Problem of Constructive Quantum Field Theory and Statistical Mechanics*. Lecture Notes in Physics 159. Berlin: Springer.

Wilson, K. G. 1974. Confinement of quarks. *Physical Review D* 10: 2445--2459.

End of manuscript